

第三章

数据链路层

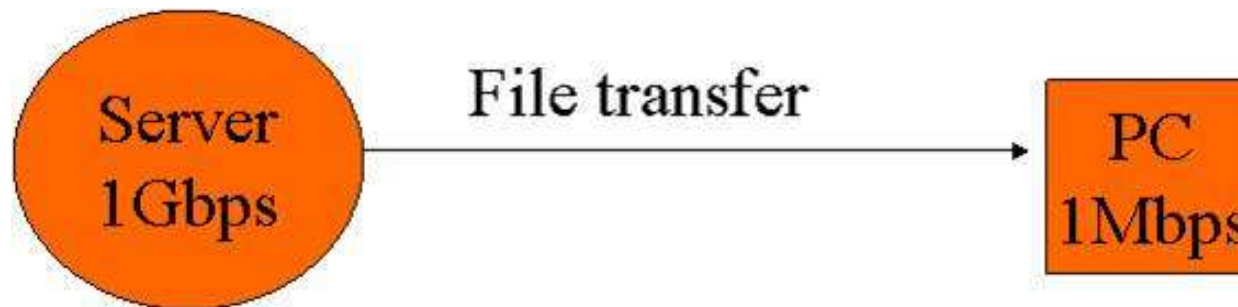
主要内容

- 数据链路层的位置、功能及服务
- 相关问题的设计
 - ◆ 成帧
 - ◆ 差错控制
 - ◆ 流量控制
- 协议示例
 - ◆ HDLC
 - ◆ PPP

(8-9学时)

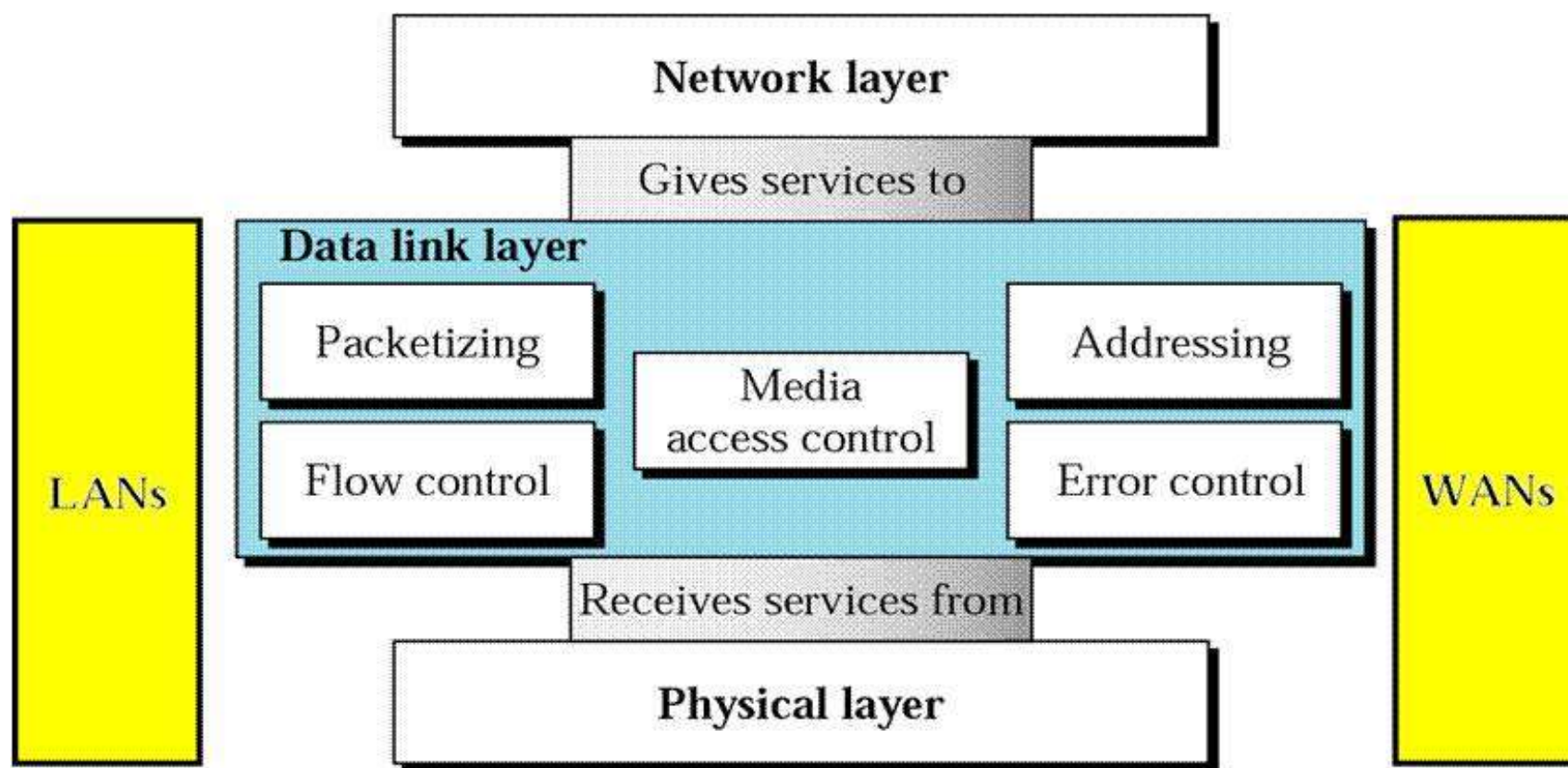
为什么需要数据链路层?

- 物理层传输可能出现位差错（误码率）
- 接收方可能要防止数据来得太快太多
- 物理层的同步和接口技术可能不够



数据链路层的位置和功能

- 数据链路层负责将数据（帧）以可靠、高效的方式从一个节点传送到链路上相邻的另一个节点



[Forouzan]

数据链路层设计（涉及）的相关问题

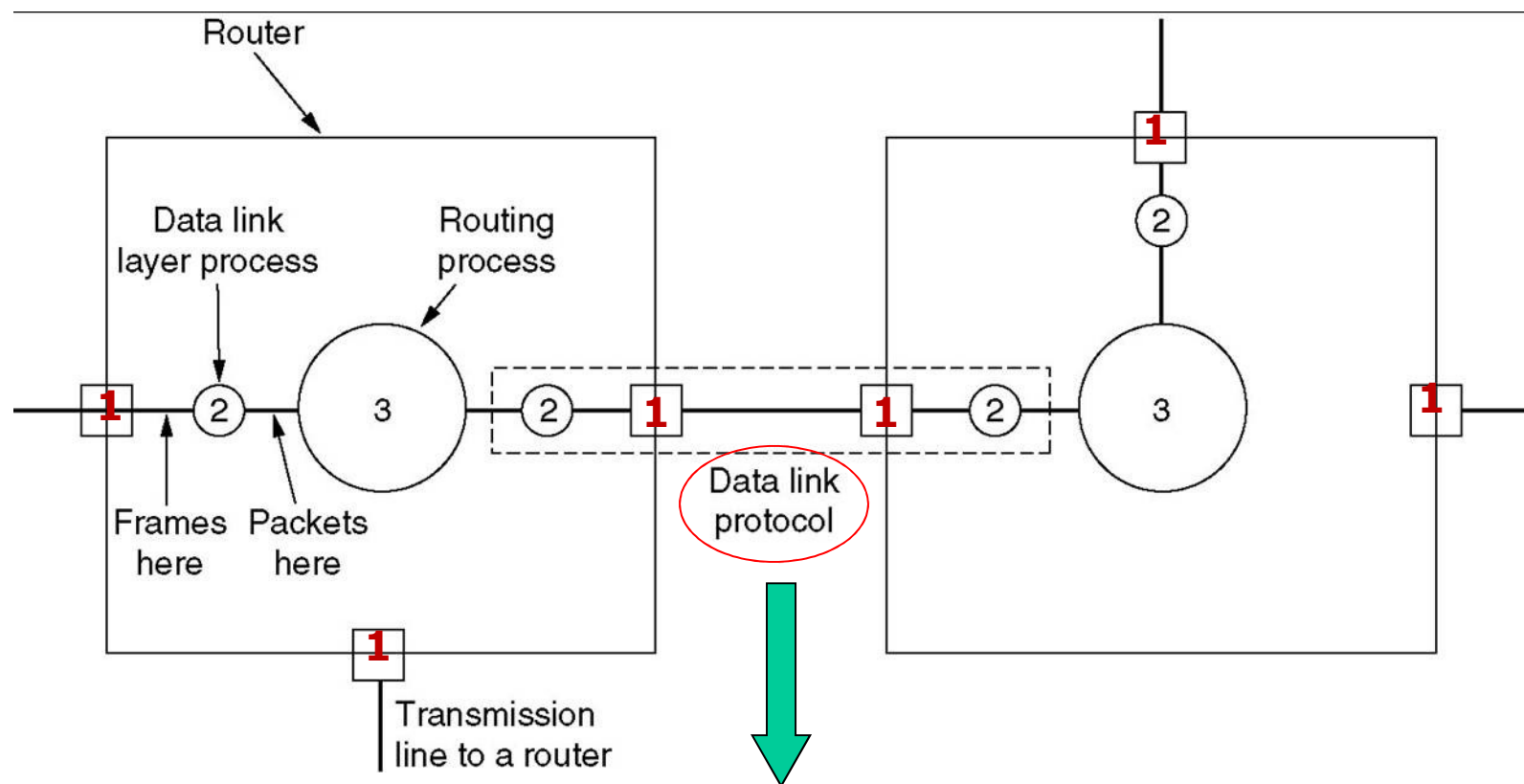
■ 目标

- ◆ 研究在相邻节点之间实现可靠高效通信的算法

■ 相关设计问题

- ◆ 向网络层提供什么服务？
- ◆ 成帧
 - 从物理层收到的位流中分离出帧
- ◆ 差错控制
 - 确保收到的数据与发出的一致
- ◆ 流量控制
 - 不能让慢速的接收方被快速的发送方淹没

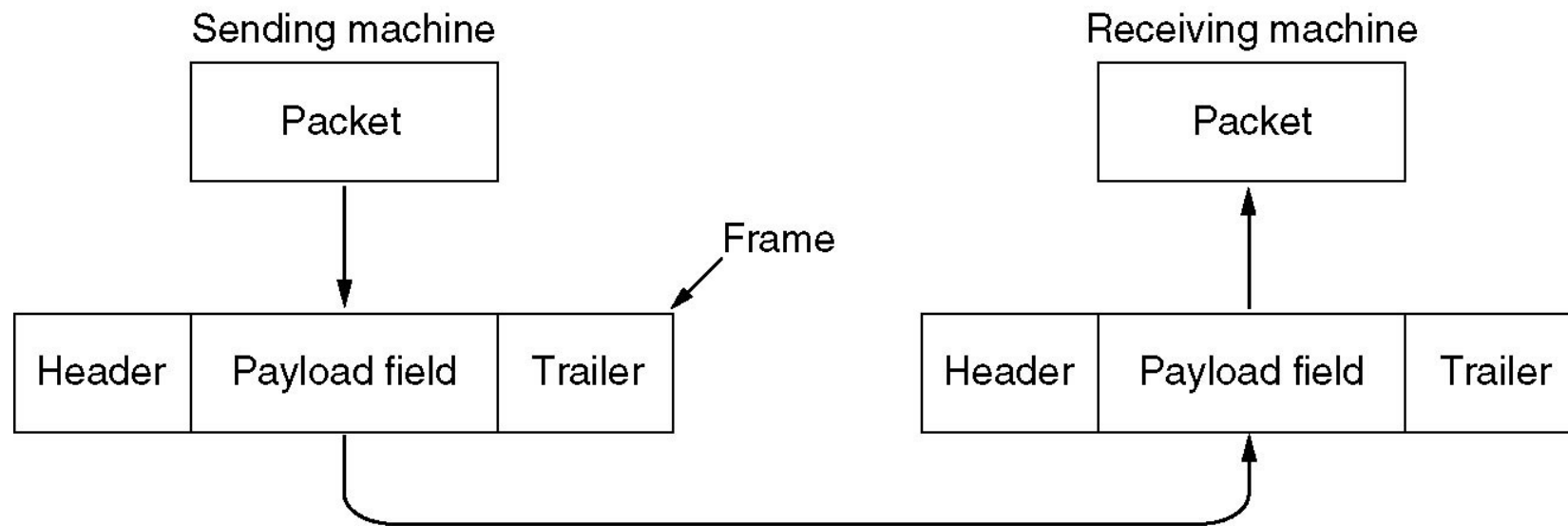
数据链路层的功能示意图



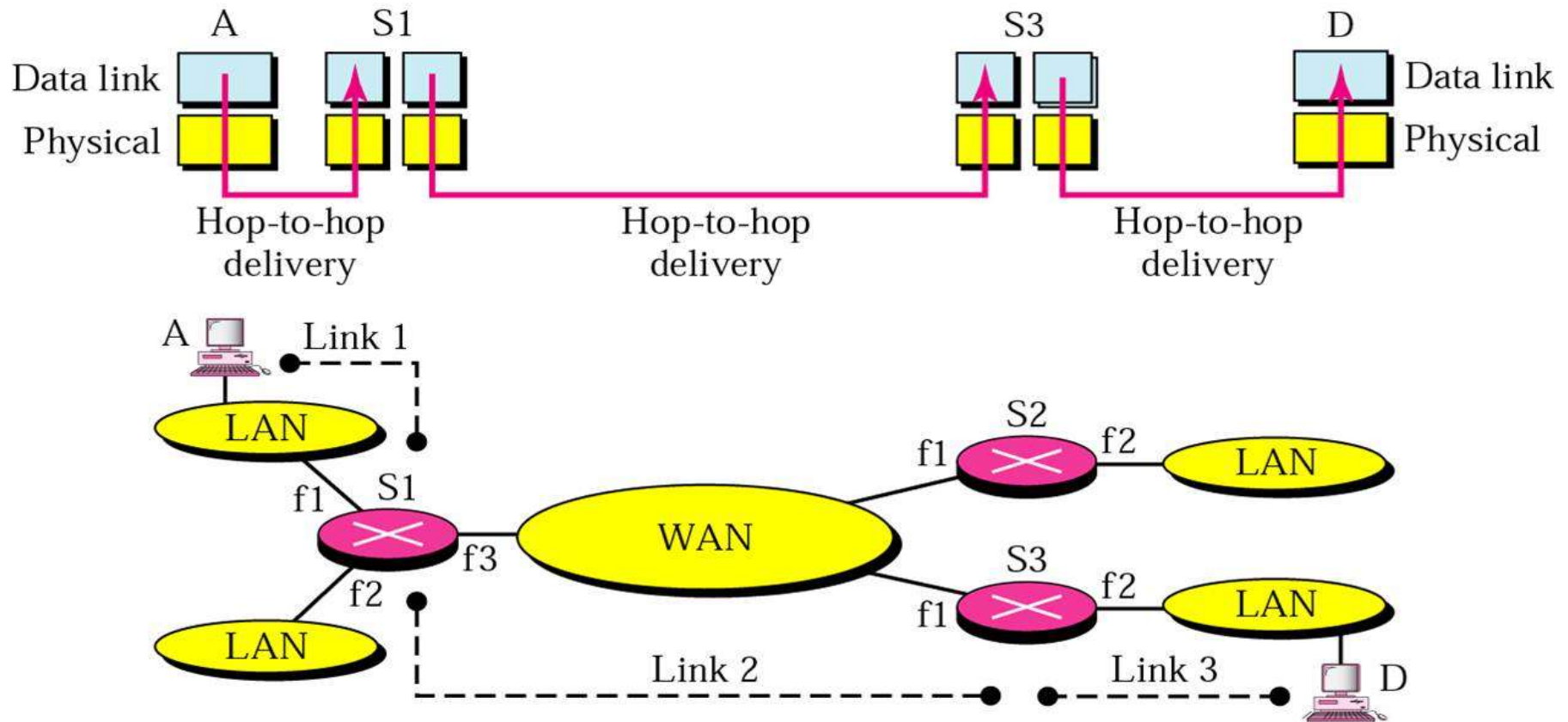
- ➡流量控制：能发送多少数据？
- ➡差错控制：差错如何纠正？
- ➡接入控制：该谁发送？→ 第4章学习

分组（包） vs. 帧

Encapsulation(封装)



数据链路层：逐跳通信



逐跳：两个设备之间没有其他数据链路层及以上的设备，只有传输介质和物理接口。

[Forouzan]

数据链路层：适配器通信

■ 发送方:

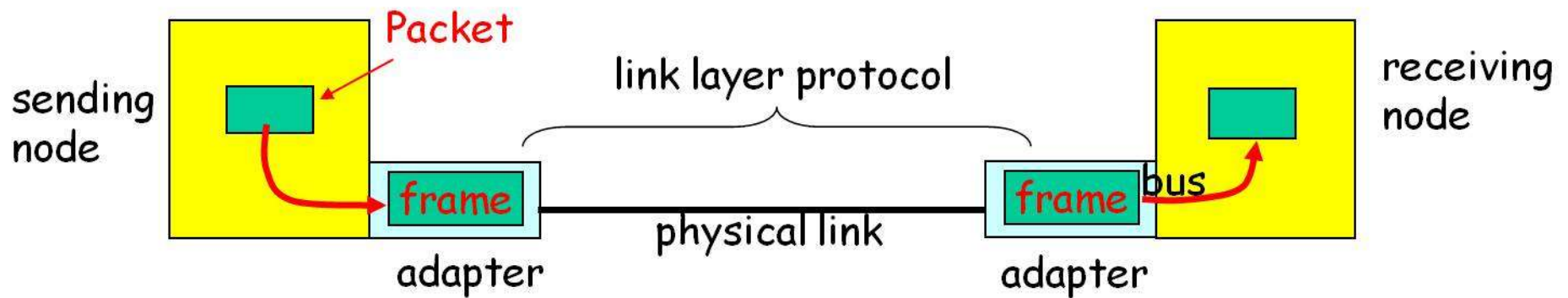
- ◆ 将网络层数据包（分组）封装到帧中
- ◆ 增加差错校验位、可靠传送、流量控制等

■ 接收方

- ◆ 检查差错、可靠传送、流量控制等
- ◆ 提取出包（分组），交给接收节点

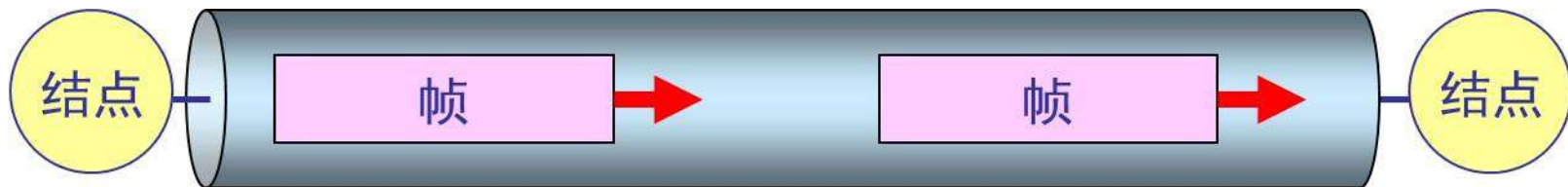
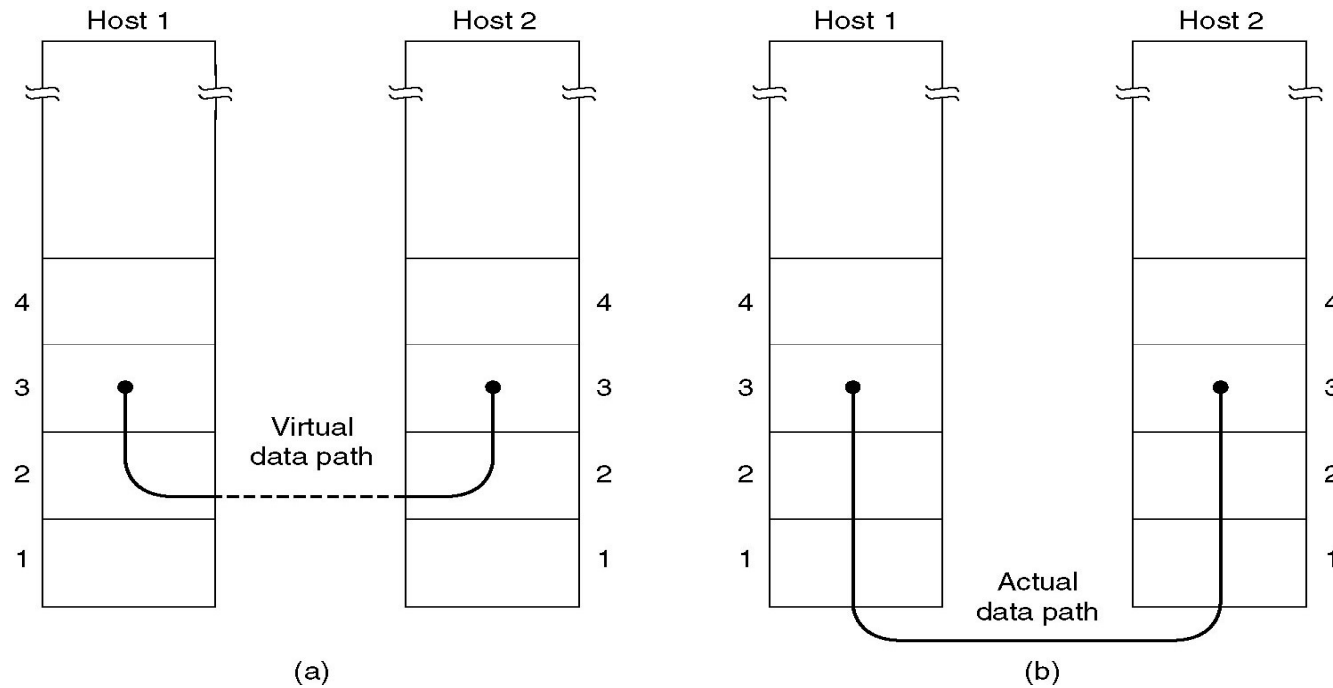
■ 数据链路层在适配器（NIC网卡）中实现

- ◆ 以太网卡、PCMCIA卡、802.11网卡



提供给网络层的服务 (1)

虚拟通信 vs. 实际通信



数据链路层像个数字管道

提供给网络层的服务（2）

■ 无连接服务

◆ 无确认的无连接服务

- 没有确认、没有逻辑连接
- 多数LAN提供这种服务

◆ 有确认的服务

- 确认（收到的）每一帧，在不可靠信道上使用，例如无线系统

■ 有确认的面向连接的服务

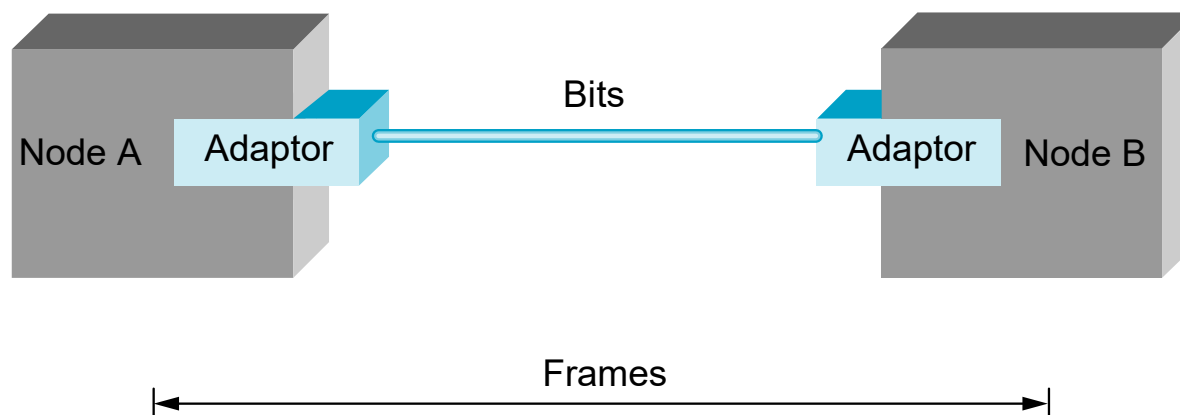
◆ （预先）建立一个连接、帧编号、确保可靠传输

◆ 例如ATM

主要内容

- 数据链路层的位置、功能及服务
- 相关问题的设计
 - ◆ 成帧
 - ◆ 差错控制
 - ◆ 流量控制
- 协议示例
 - ◆ HDLC
 - ◆ PPP

成帧 (Framing)



- 在点到点链路上从A节点向B节点发送一串比特（位）给主机B
- B节点必须能够准确识别哪些比特（位）组成一帧，即B必须确定一帧从哪里开始，到哪里结束

成帧的要求

- 要求: 拆分比特流

- 约束条件

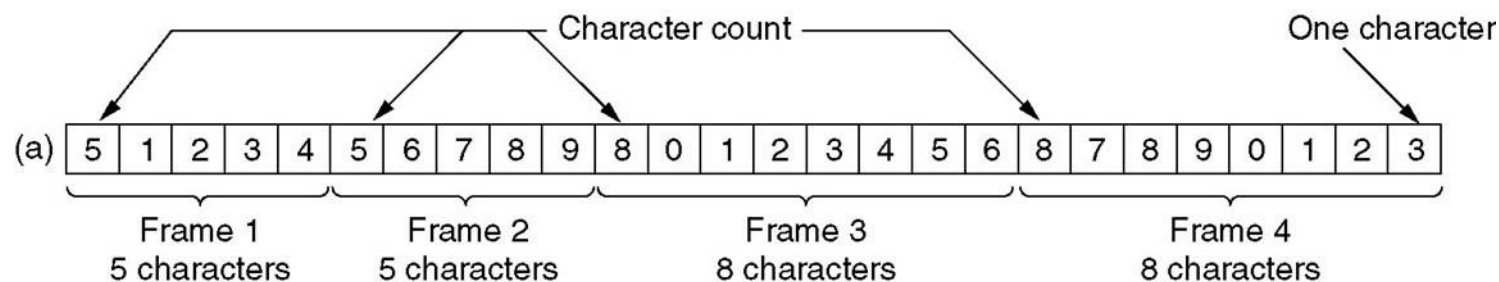
- ◆ 简单: 接收方容易实现
- ◆ 独立于代码: 与传输的消息内容无关
- ◆ 高效: 使用的信道带宽尽量少
- ◆ 帧的长度与所传输的数据的内容无关
- ◆ 健壮性: 不易出错, 出错后易重新同步

成帧的主要方法

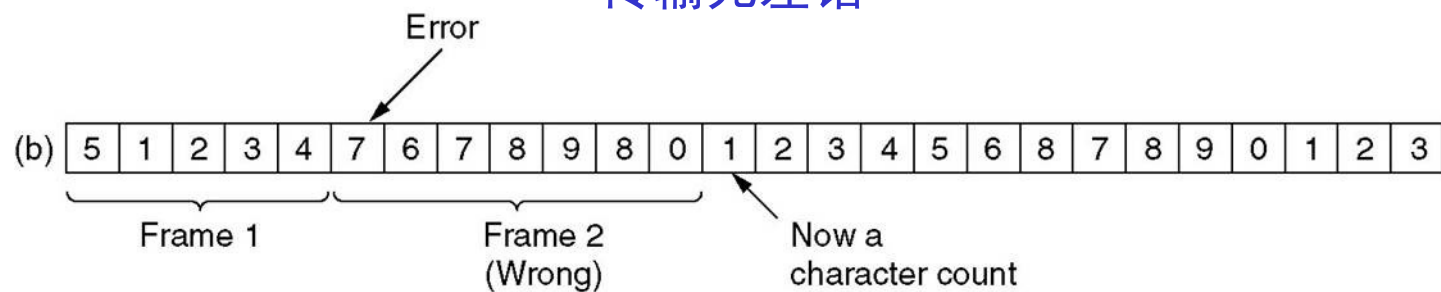
- 字符计数法: Character count
- 字节/字符填充法: Flag byte with byte stuffing
- 比特填充法: Starting and ending flags, with bit stuffing
- 物理层编码违例法: Physical layer coding violations

字符计数法

- 在帧头用一个字段（域）来说明帧内的字符个数
- 传输差错可能导致计数值被篡改



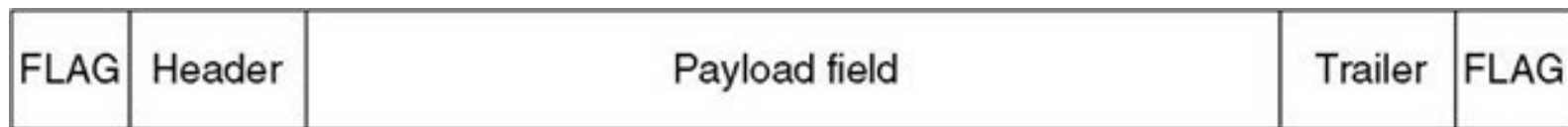
传输无差错



有一位传输差错

字节/字符填充

- 用一个标志字节来标记帧的开始和结束
- 和所使用的8位字符集密切相关
- 协议示例：BISYNC(IBM)



例如,

帧开始符

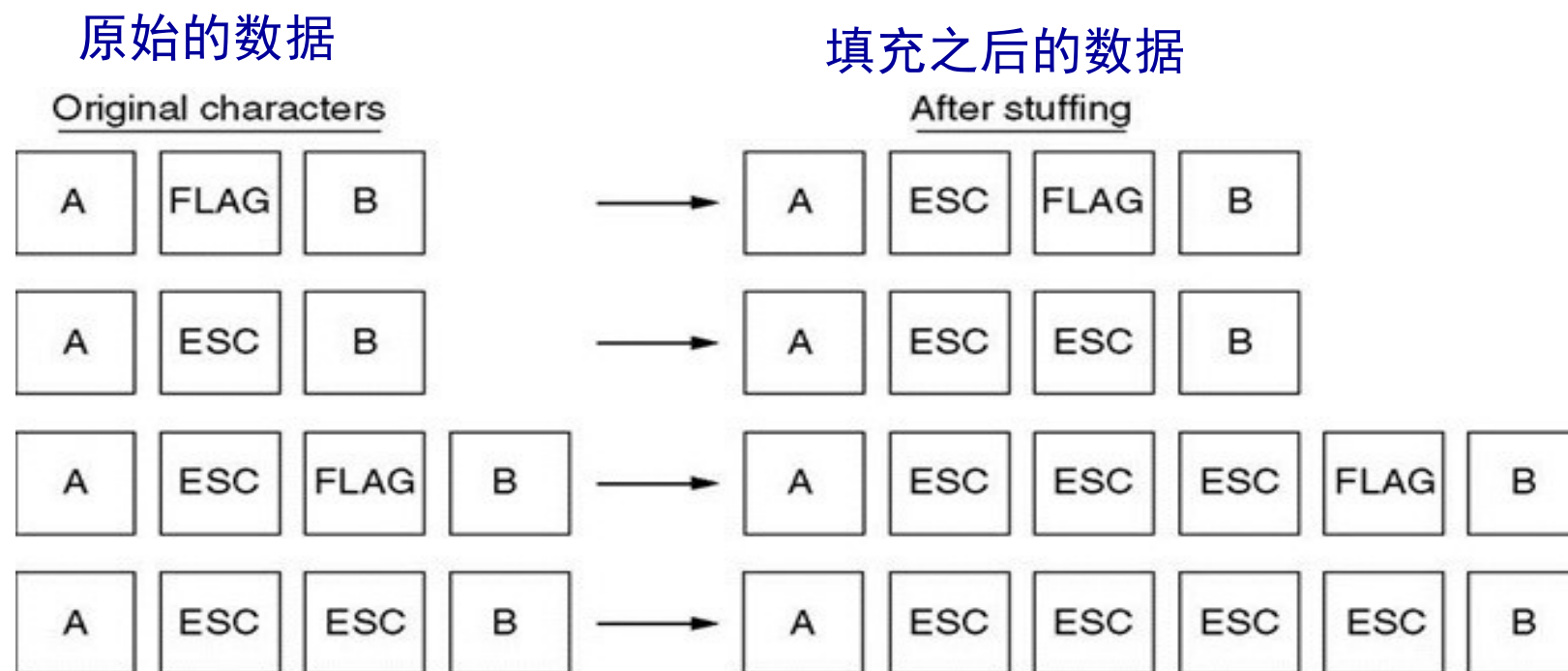
帧结束符



← 帧 →

字节填充：透明传输

- 如果数据中包含和标志相同的字节怎么办？
 - ◆ 使用特殊的填充字节：**ESC** (如 0x1B)
 - ◆ 插在数据中和标志相同的字节前面



比特填充

- 帧中包含的比特数是任意的（不需要是8的整数倍）
- 帧开始/结束标志：0111,1110
- 填充：在连续五个'1'之后填充一个
- 应用在HDLC中

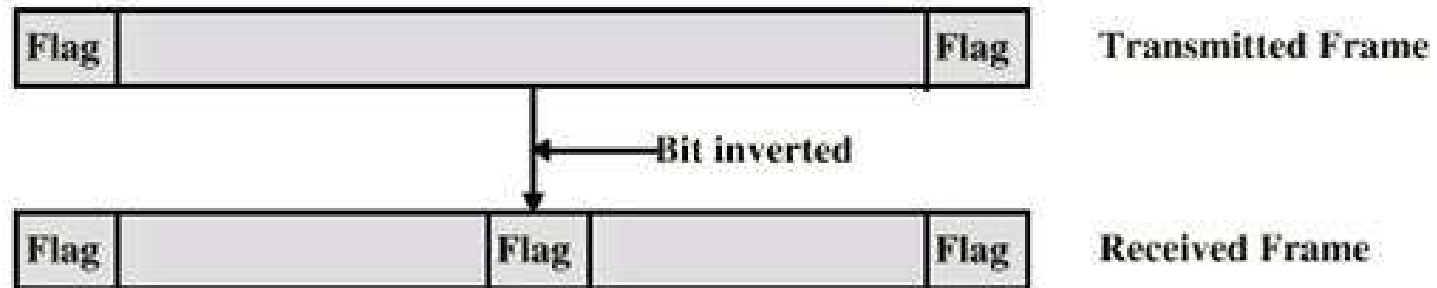
原始数据 (a) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

填充之后 (b) 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 0

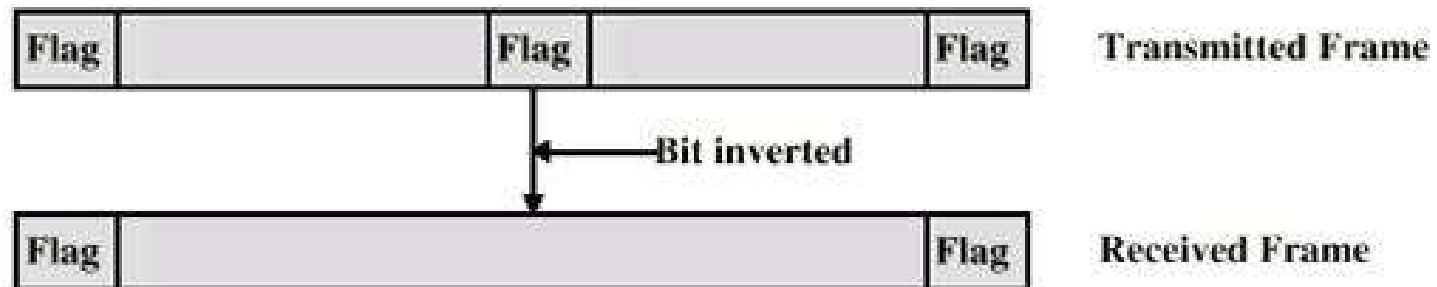
Stuffed bits

去填充之后 (c) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

比特填充可能出现的问题



(b) An inverted bit splits a frame in two



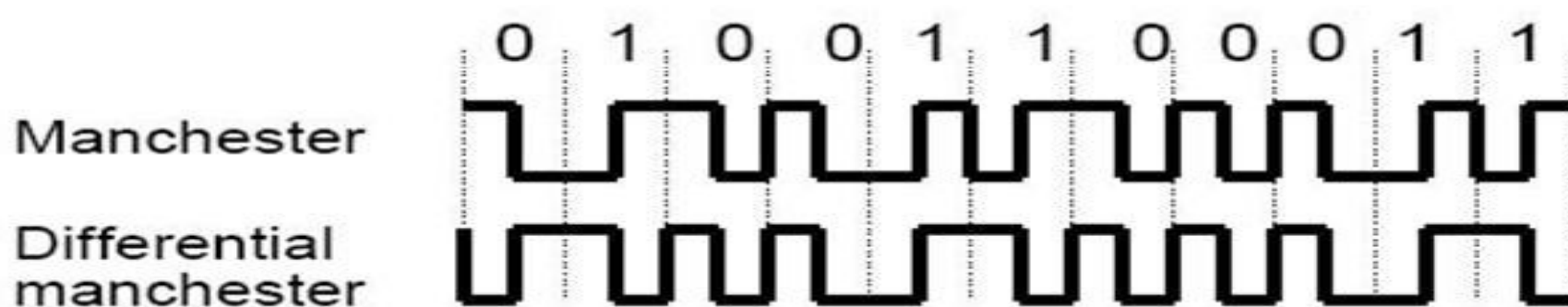
(c) An inverted bit merges two frames

比特填充练习

- 从信道上收到比特序列： 1101 0111 1110
0111 1110 1101 1011 1110 0010 1100
0101 1111 0101 1001 1111 1001，该比特序列中包含一个完整的帧，用十六进制写出该帧的内容（不包含帧的首尾标志）。

物理层编码违例法

- 编码方案有冗余时，可以采用
 - ◆ 例如，LAN使用的曼彻斯特编码
 - ◆ 每个有效的二进制位在信号中间都有跳变
 - ◆ 信号中间没有跳变的编码，如 高-高 和 低-低 用于帧定界（标记帧的开始和结束）



Review

- 数据链路层的功能：相邻节点间可靠地传输数据帧
- 实现可靠传输的功能
 - ◆ 差错控制：检测差错、纠正差错
 - ◆ 流量控制：防止发送方速度过快而淹没接收方（接收方来不及接收而丢失数据）
 - ◆ 介质访问控制(MAC)：对于共享信道，如何防止因两个站点同时发送而导致冲突？ ---在下一章学习
- 向网络层提供的服务：取决于不同的物理网络（数据链路层+物理层）
 - ◆ Ethernet：无连接服务
 - ◆ Wifi：带确认的无连接服务
 - ◆ ATM：面向连接服务

数据链路层的第一个功能：成帧

- 目标：让接收方能够从来自物理层的一串二进制位流中提取出完整的一帧
- 成帧方法
 - ◆ 字符计数法：前面加一个帧长度字段，不实用
 - ◆ 字节(字符)填充法：前面加一个帧首标志字符，后面加一个帧尾标志字符
 - 透明传输：用添加转义字符来实现
 - ◆ 比特(位)填充法：用0111 1110标记帧头和帧尾
 - 透明传输：零比特填充，在帧中连续的五个1之后添加一个0
 - ◆ 物理层编码违例法：使用物理层中的非法编码标记帧的开始和结束，自然解决透明传输，无需填充

主要内容

- 数据链路层的位置、功能及服务
- 相关问题的设计
 - ◆ 成帧
 - ◆ 差错控制
 - 纠错码 (ECC)
 - 检错码 (EDC)
 - ◆ 流量控制
- 协议示例
 - ◆ HDLC
 - ◆ PPP

差错控制

■ 差错类型

- ◆ 帧丢失：一帧没有到达另一端
 - 由于噪声影响、或者被从队列中丢弃
- ◆ 帧被毁坏：某些比特出错（收到的与发送的不一致）

■ 差错检测方法

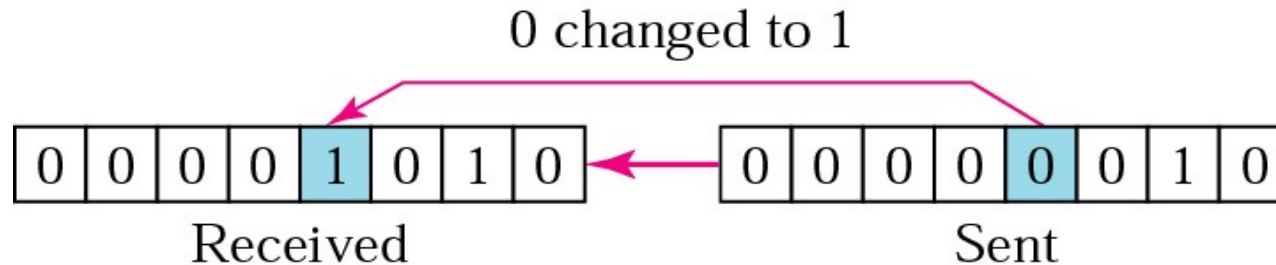
- ◆ 奇偶校验：能检测出单比特错误
- ◆ 循环冗余校验(CRC)：可检测出一些突发错误
- ◆ 检查和(Checksum)，在网络层学习

■ 差错纠正方法

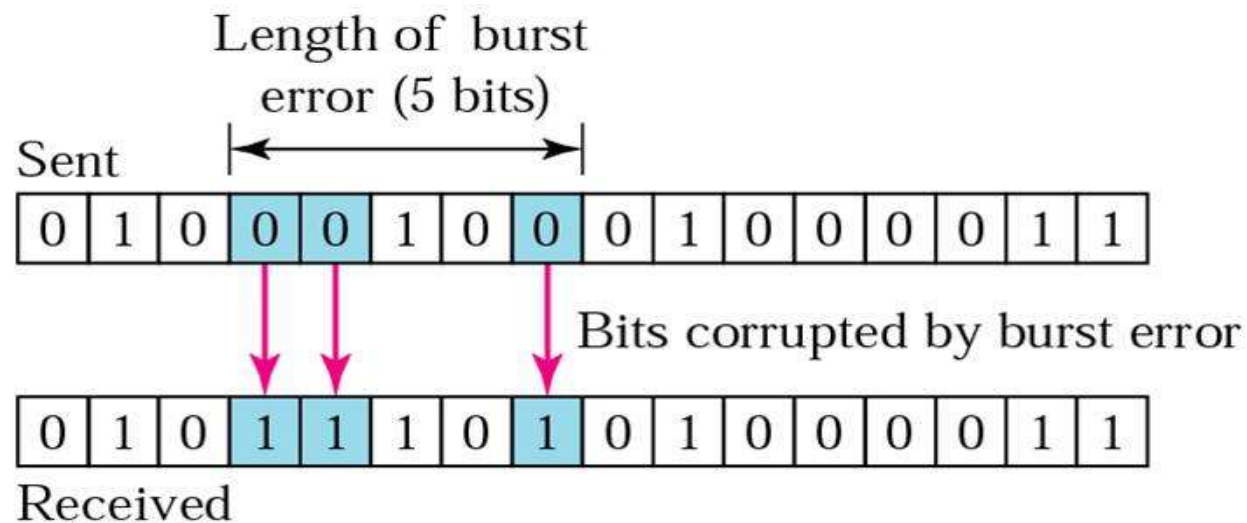
- ◆ 前向纠错FEC
- ◆ 重传：自动重发请求 (ARQ)

单比特差错 & 突发差错

- 单比特差错：只有一个比特被改变



- 突发差错：2个或更多的比特被改变



奇偶校验

■ 奇偶校验

1	0	1	1	1	0	1	0	?
---	---	---	---	---	---	---	---	---

■ 奇校验

1	0	1	1	1	0	1	0	0
0	0	1	1	1	0	1	0	0
0	0	0	1	0	0	1	0	0
0	0	1	1	1	1	1	0	0

■ 偶校验

1	0	1	1	1	0	1	0	1
---	---	---	---	---	---	---	---	---

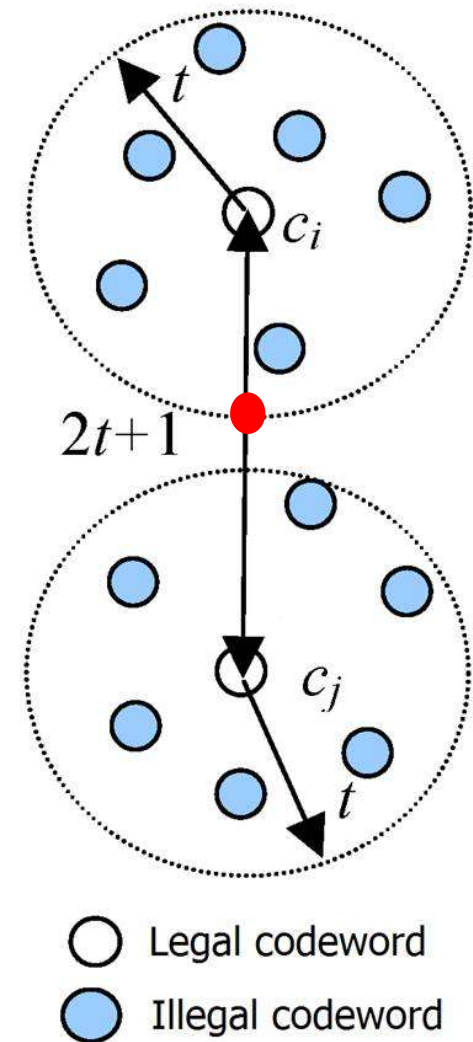
单个奇偶校验位性能

- 可检出奇数个位翻转错误
- 其它情况漏检率50%

[Forouzan]

纠错码：汉明/海明距离

- 汉明距离：两个码字中对应比特的值不同的数目
 - ◆ 例如. 1000 1001 和 1011 0001, 汉明距离 = 3
- 要检测出 t -bit 差错, 编码的汉明距离至少为 $t+1$
- 要纠正 t -bit 差错, 编码的汉明距离至少是 $2t+1$



纠错码示例

- ◆ 对于下列只有4个有效码字的编码：
0000000000, 00000 11111
11111 00000, 11111 11111
- ◆ 汉明距离是5，这意味着可以纠正 2-bit 差错
 - 若收到码字 00000 00111，接收方知道原来的码字一定是 00000 11111
 - 若发生了三比特差错，00000 00000 变成了 00000 00111，将无法正确地纠错。

校验位的个数

- 要纠正单比特差错，需要几个校验位？
- 编码长度
 - ◆ m : 消息位的位数
 - ◆ r : 校验位的位数
 - ◆ n : 码字长度, $n=m+r$
- 一共有 2^m 个合法消息
- 与一个合法消息距离为1的非法码字有 n 个，因此一共有 $(n+1)2^m$ 个码字

$$(n+1)2^m \leq 2^n$$

$$\longrightarrow (m+r+1) \leq 2^r.$$

例如: $m=7, r \geq 4$

汉明码：纠正单比特差错

将码字的各比特编号，最左端为1号，其右边为2号，...
以此类推

■ 校验位

- ◆ 编号为2的幂次（1,2,4,8,16...）的位置对应的比特
- ◆ 每个校验位负责对一些比特（数据位）（包括校验位本身）进行奇偶校验

■ 数据位

- ◆ 非二进制幂次的编号位置（3,5,6,...）放数据位

汉明码示例 (1)

- 待校验的消息: 'A' (7-bit ASCII 100,0001)
数据位的位置编号: 3,5,6,7,9,10,11
- 4个校验位: 位置编号是 1,2,4,8 (采用偶校验)
位编码: 1 2 3 4 5 6 7 8 9 10 11
码字: 0 0 1 0 0 0 0 1 0 0 1

发送前编码:

$$3=1+2, \quad 5=1+4, \quad 6=2+4, \quad 7=1+2+4$$

$$9=1+8, \quad 10=2+8, \quad 11=1+2+8$$

接收者纠错:

$$b1 \oplus b3 \oplus b5 \oplus b7 \oplus b9 \oplus b11 = 0 \quad (k=1)$$

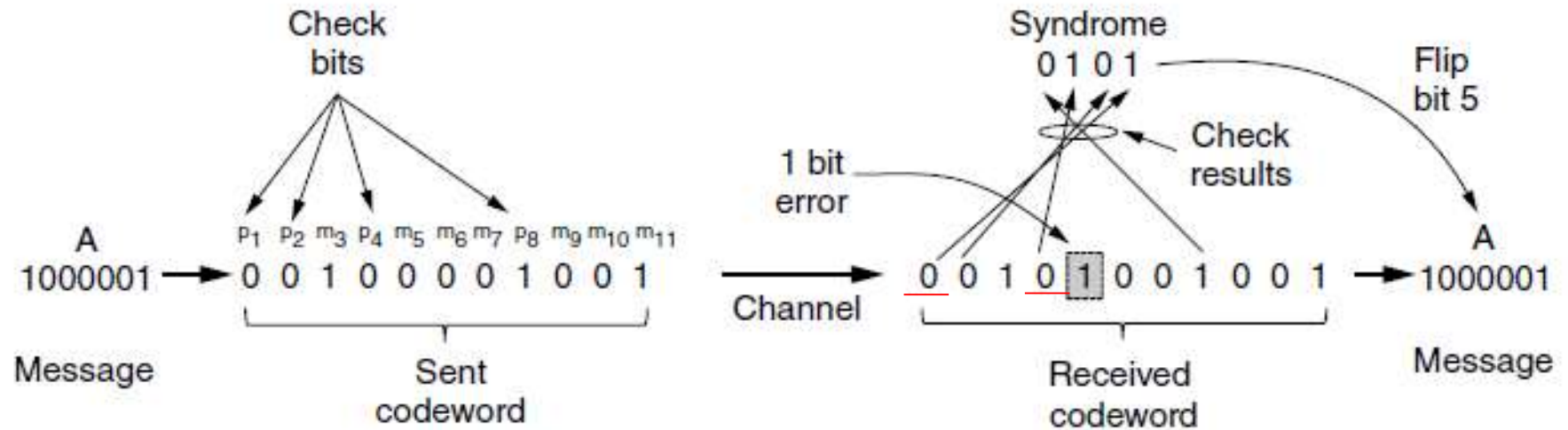
$$b2 \oplus b3 \oplus b6 \oplus b7 \oplus b10 \oplus b11 = 0 \quad (k=2)$$

$$b4 \oplus b5 \oplus b6 \oplus b7 = 0 \quad (k=4)$$

$$b8 \oplus b9 \oplus b10 \oplus b11 = 0 \quad (k=8)$$

汉明码示例(2)

采用偶校验来计算校验位



- 汉明码的汉明距离: 3
- 可以检测出2比特差错, 纠正单比特差错

偶校验

海明编码解码—简便法

例：数据 = 1011,

b1	b2	b3	b4	b5	b6	b7
P1	P2	1	P3	0	1	1

编码简便法：将码字中为1的各位码字位号表示为二进制码，再按模2求和，所得结果就是校验码。

$$\begin{array}{r} b3 = 011 \\ b6 = 110 \\ \oplus \quad b7 = 111 \\ \hline P3P2P1 = 010 \end{array}$$

发送的码字：→

b1	b2	b3	b4	b5	b6	b7
0	1	1	0	0	1	1

收到的码字：→

b1	b2	b3	b4	b5	b6	b7
0	1	1	0	0	1	0

差错位

解码简便法：将码字中为1的各位码字位号表示为二进制码，再按模2求和，若和为0，则无差错。若和不为0，则指明差错的位号。

$$\begin{array}{r} b2 = 010 \\ b3 = 011 \\ \oplus \quad b6 = 110 \\ \hline S4S2S1 = 111 \end{array}$$

汉明码的示例

Char.	ASCII	Check bits
H	1001000	00110010000
a	1100001	10111001001
m	1101101	11101010101
m	1101101	11101010101
i	1101001	01101011001
n	1101110	01101010110
g	1100111	01111001111
	0100000	10011000000
c	1100011	11111000011
o	1101111	10101011111
d	1100100	11111001100
e	1100101	00111000101

Order of bit transmission

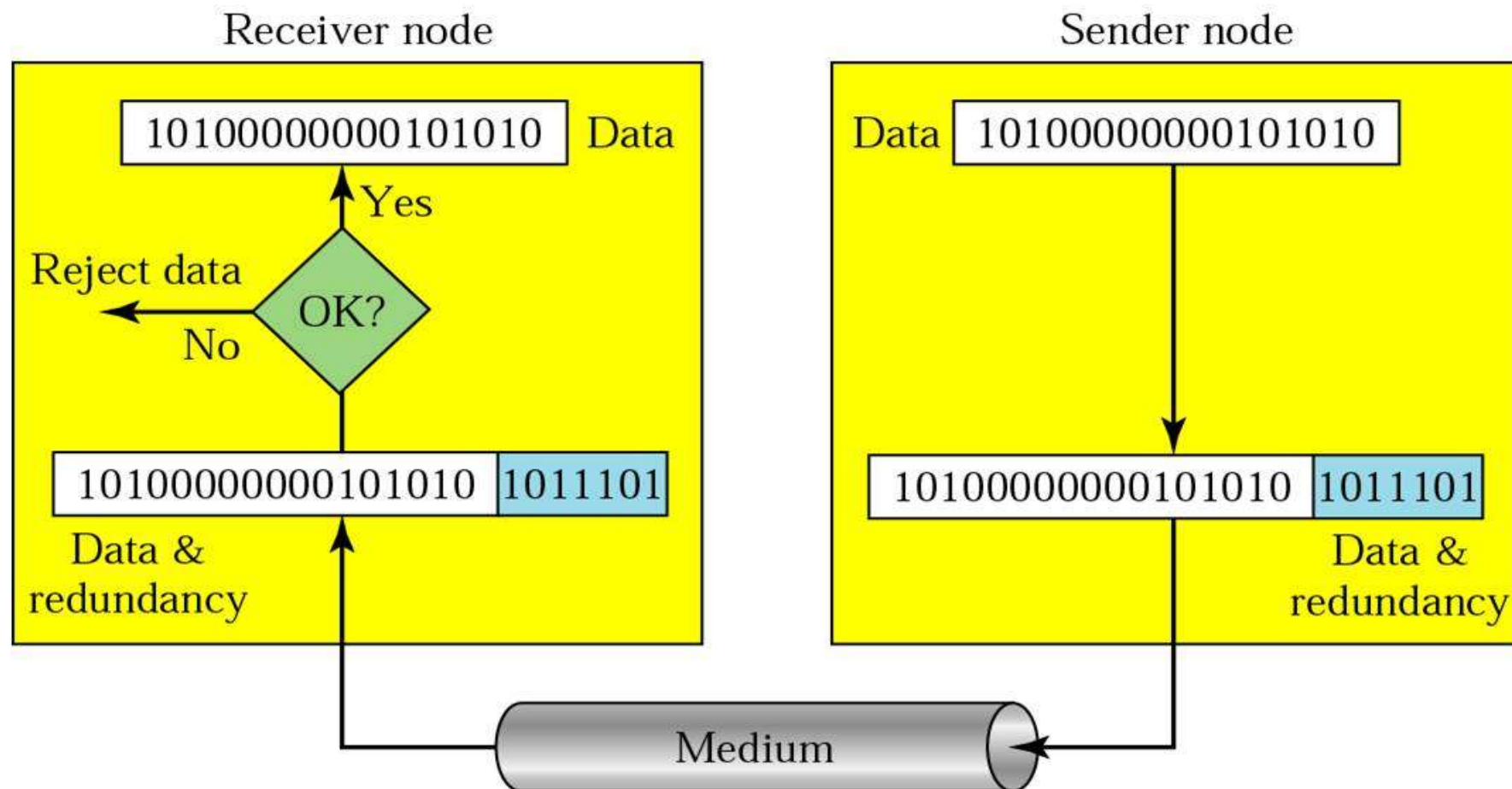
主要内容

- 数据链路层的位置、功能及服务
- 相关问题的设计
 - ◆ 成帧
 - ◆ 差错控制
 - 纠错码 (ECC)
 - 检错码 (EDC)
 - ◆ 流量控制
- 协议示例
 - ◆ HDLC
 - ◆ PPP

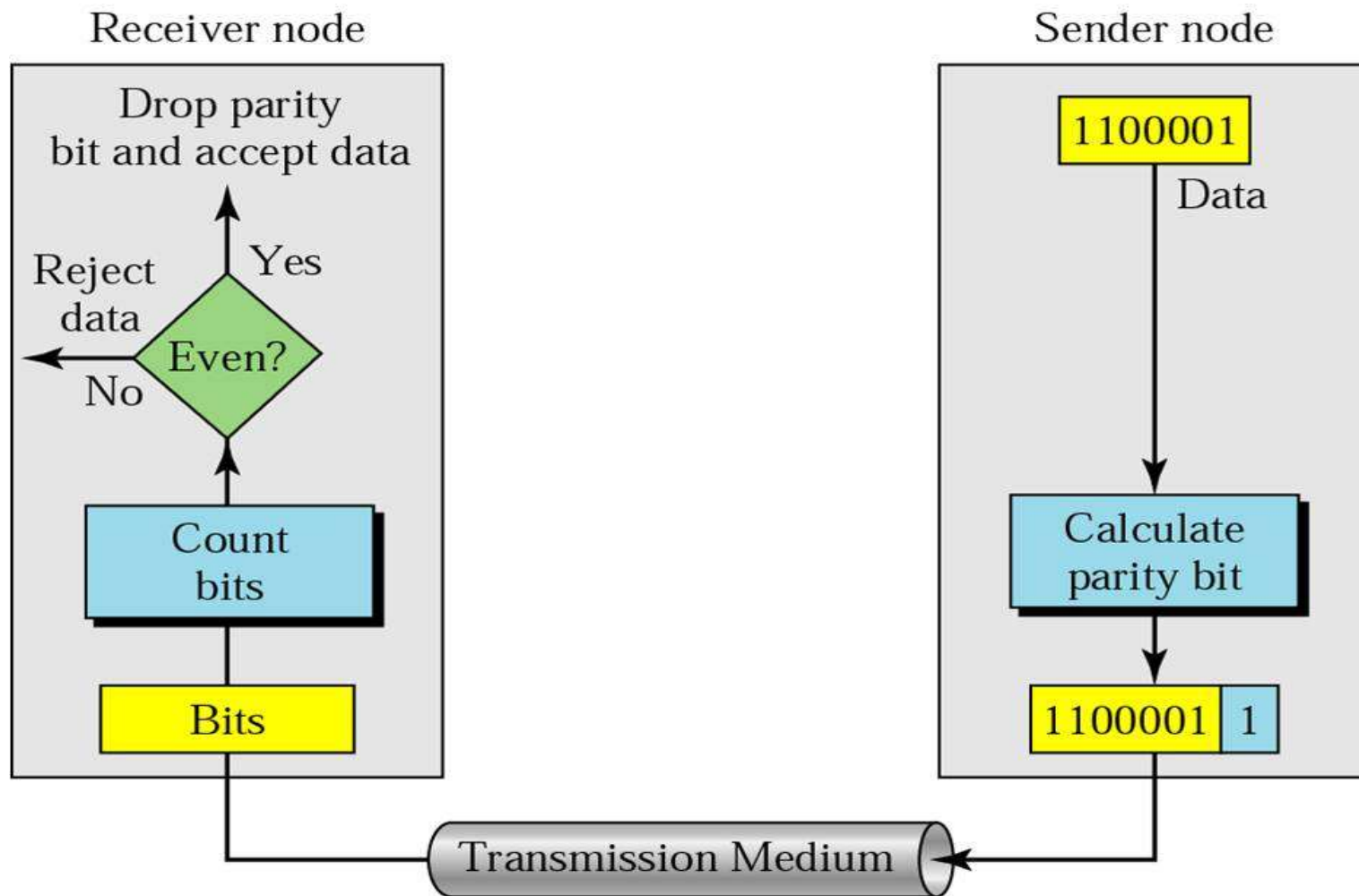
纠错码与检错码

- 在可靠的信道上，如光纤，使用检错码价格比较低，对于偶尔出错的数据块进行重传
- 在差错较多的无线链路上，较好的方法是为每个数据块增加足够的冗余，以便接收方能够计算出原始的数据块——使用纠错码
- 示例：
 - ◆ 位差错率(BER)= 10^{-6} , 1块=1000位, 数据=1M位
 - ◆ (成功) 发送1000块的开销
 - 差错检测 (奇偶校验) + 重传: 2001 位
 - 汉明码: 10,000 位, 每个数据块需要10个校验位

差错检测



偶校验

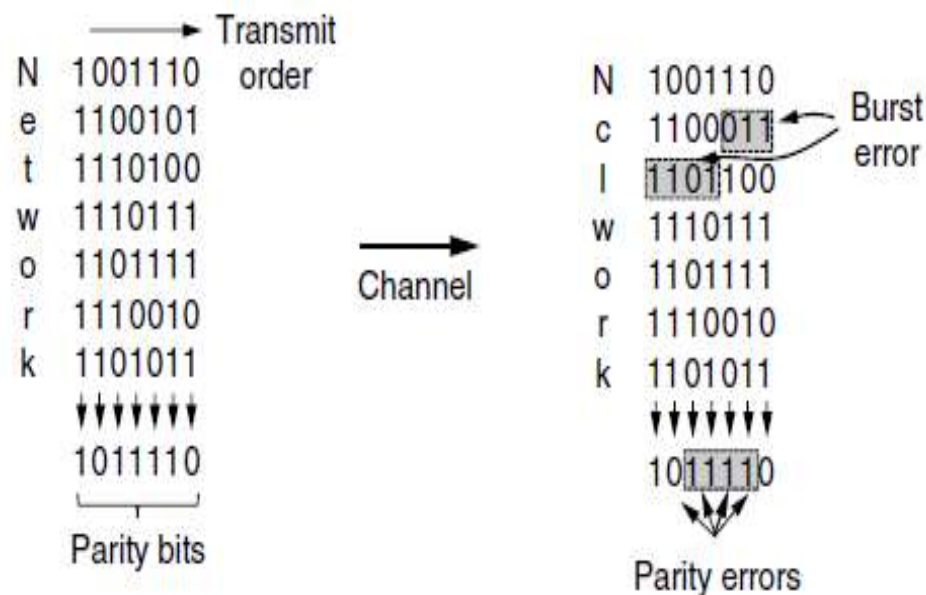


对数据块的奇偶校验 (交错校验)

- 编码 (n 个校验位 check bits)
 - ◆ 每个数据块是一个 $k \times n$ 矩阵
 - ◆ 一次发送一行
 - ◆ 对每一列计算一个奇偶校验位

- 性能

- ◆ 可检测出长度为 n 的突发差错
- ◆ 对于长度为 $n+1$ 的突发差错，如果恰好第一个比特被翻转，最后一个比特被翻转，而中间的所有比特都是正确的，这种情况将无法检测出来
- ◆ 如果发生更长的突发差错，无法检测出（即被接受）的概率是 2^{-n}



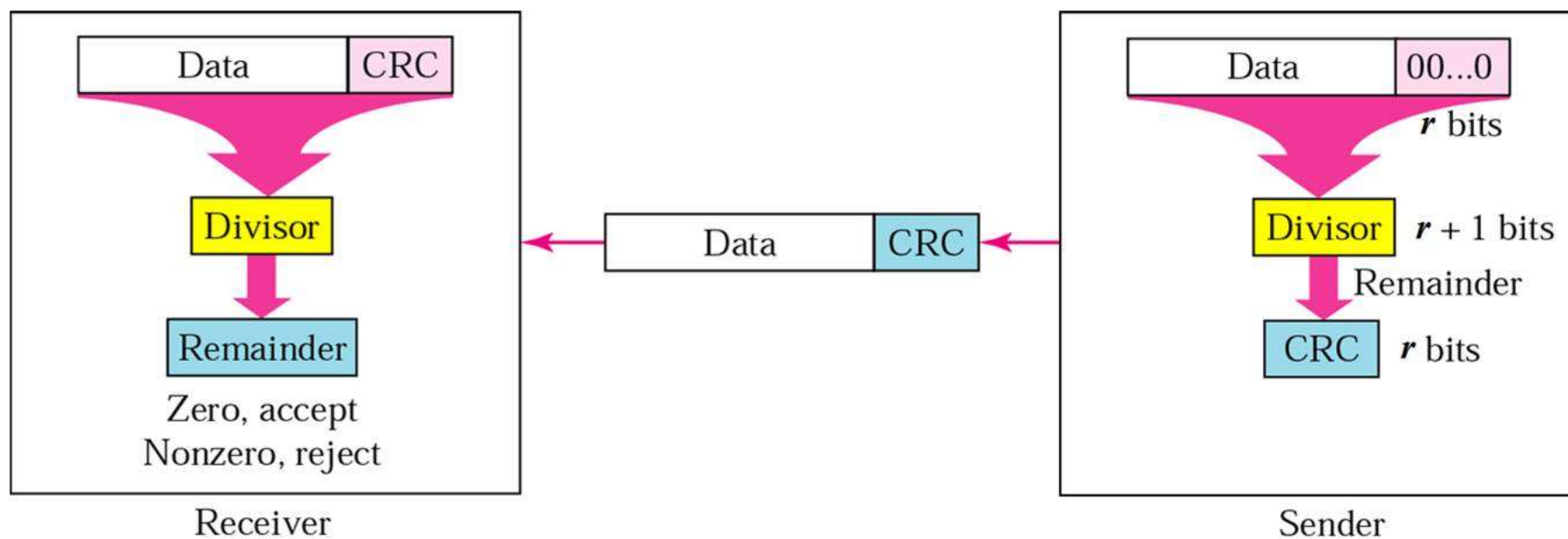
多项式编码

也称为 **CRC** (循环冗余校验), **FCS** (帧校验序列)

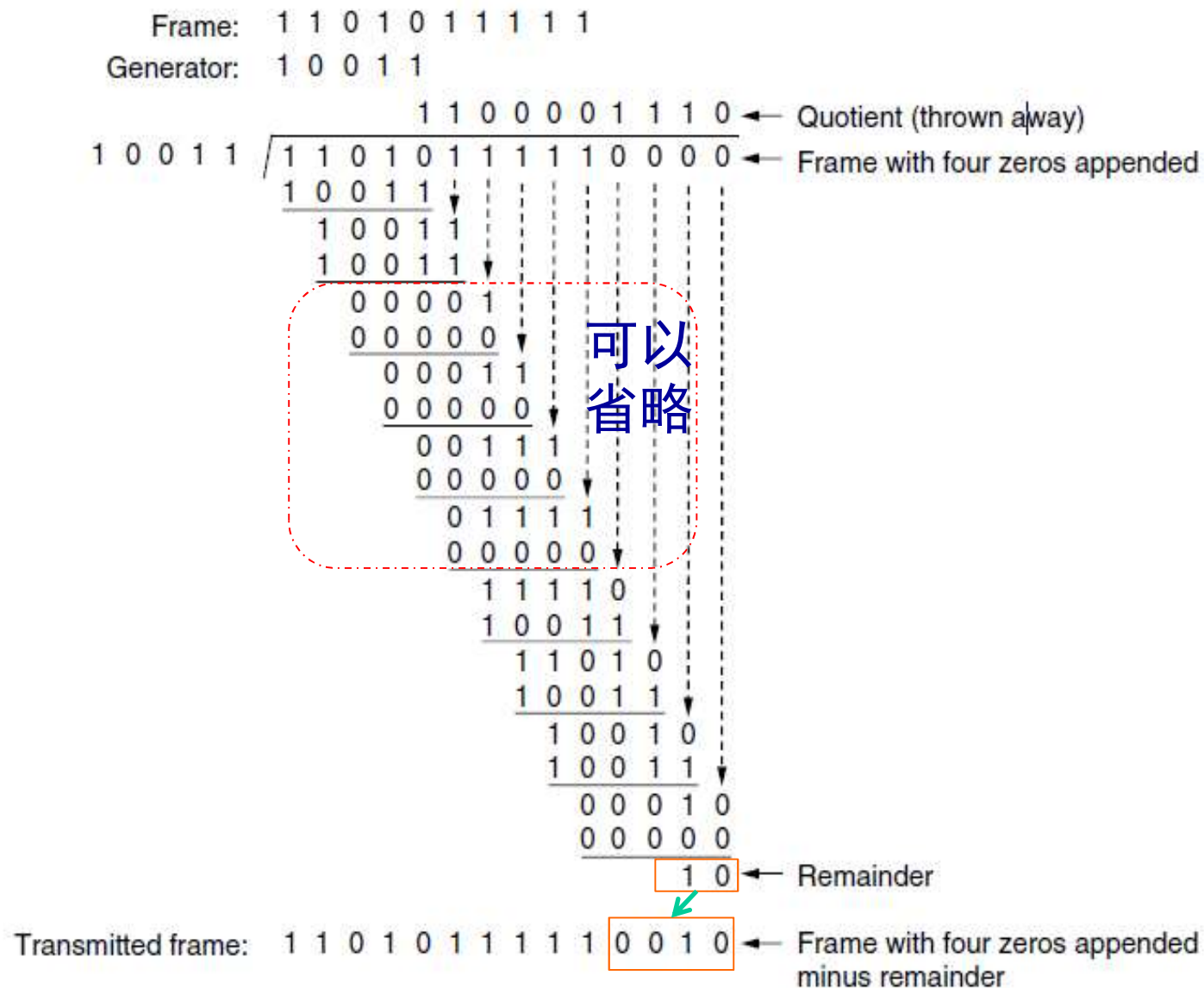
- **$M(x)$** : 把比特串看成多项式的系数 (**0**和**1**)
 - ◆ 将一个**k-bit** 的帧看成是**k** 阶多项式 (从 x^{k-1} 到 x^0) 的系数序列, 如 **110001** $\Rightarrow x^5 + x^4 + x^0$
- 多项式算法按照模**2**运算, 即加和减都是异或 (**XOR**)
- **$G(x)$** : 生成多项式

发送方：计算校验位

- 计算校验位的算法如下：
 - ◆ 在帧的最低阶后面增加 r 个0
 - ◆ 用构成的位串除以 $G(x)$
 - ◆ $T(x)$: 从位串中减去余数



CRC计算示例



接收方：验证校验位

- 如果出现传输差错，到达的不是比特串 $T(x)$ ，而是 $T(x) + E(x)$
- 接收方计算 $[T(x) + E(x)]/G(x)$ ，由于 $T(x)/G(x) = 0$ ，校验结果取决于 $E(x)/G(x)$
- ◆ 如果差错恰好对应于包含 $G(x)$ 因子的多项式，将无法检测出来；其他差错则能被检测出来。

CRC的检错能力

- r 个校验位能检测出长度 $\leq r$ 的全部突发差错
- 能够检测出两个独立的单比特差错
- 能够检测出所有奇数个翻转差错
- 其他长度 $>r$ 的突发差错
- ◆ 无法检测出坏帧的概率是 2^{-r}

生成多项式

- 标准规定的一些生成多项式:

- CRC-12(Telecomm system)

$$x^{12} + x^{11} + x^3 + x^2 + x + 1$$

- CRC-16(Bisync, Modbus, USB, ANSI X3.28, SIA DC-07, many others)

$$x^{16} + x^{15} + x^2 + 1$$

- CRC-CCITT (HDLC, ITU X.25, V.34/V.41/V.42, PPP-FCS, ZigBee)

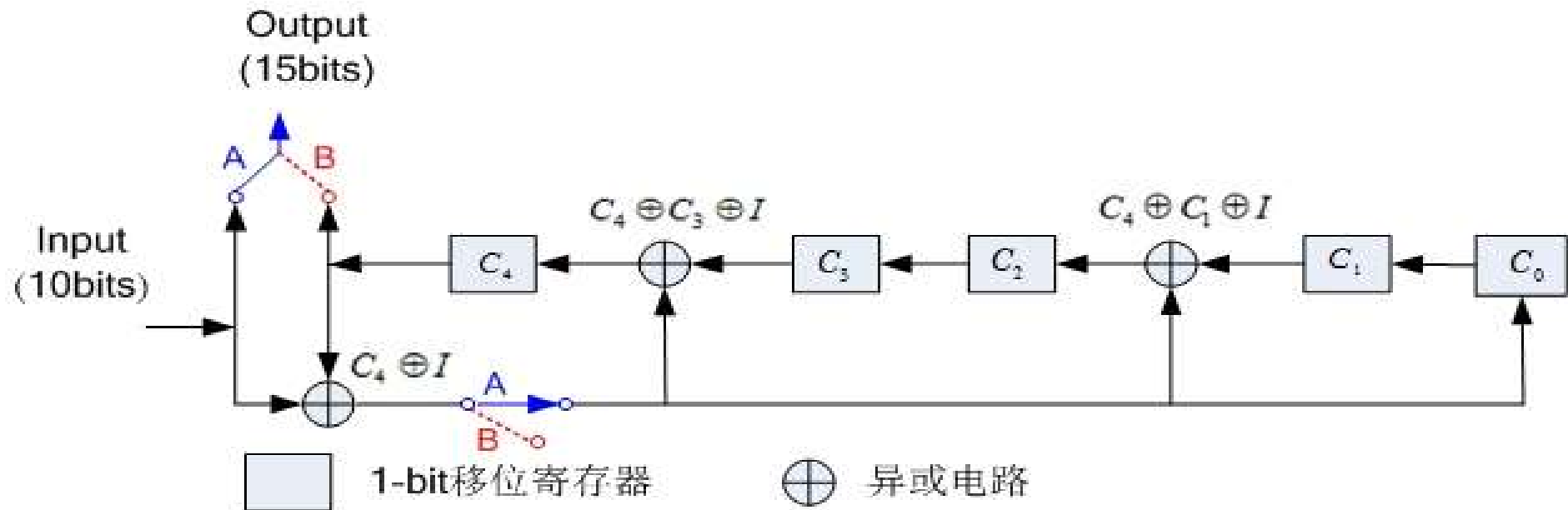
$$x^{16} + x^{12} + x^5 + 1$$

- CRC-32 (ZIP, RAR, IEEE 802 LAN/FDDI, IEEE 1394, PPP-FCS)

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

CRC的计算 (硬件实现)

- 一个简单的移位寄存电路可以实现于计算和验证校验位



- ◆ $G(x)=x^5+x^4+x^2+1$, $D=1010001101$; $D(X)=x^9+x^7+x^3+x^2+1$
- ◆ 初始时移位寄存器 $C_0 \sim C_4$ 清0
- ◆ 10个信息位发送之后, 断开A接通B, 发送5位校验和, 并将 $C_0 \sim C_4$ 清零, 为下次发送做好准备

硬件计算CRC的示例

	C_4	C_3	C_2	C_1	C_0	$C_4 \oplus C_3 \oplus I$	$C_4 \oplus C_1 \oplus I$	$C_4 \oplus I$	$I(\text{Input})$
Initial	0	0	0	0	0	1	1	1	1
Step 1	1	0	1	0	1	1	1	1	0
Step 2	1	1	1	1	1	1	1	0	1
Step 3	1	1	1	1	0	0	0	1	0
Step 4	0	1	0	0	1	1	0	0	0
Step 5	1	0	0	1	0	1	0	1	0
Step 6	1	0	0	0	1	0	0	0	1
Step 7	0	0	0	1	0	1	0	1	1
Step 8	1	0	0	0	1	1	1	1	0
Step 9	1	0	1	1	1	0	1	0	1
Step10	0	1	1	1	0				

2020春

计算机网络, 数据链路层

小结

■ 差错控制方法

◆ 纠错码：汉明码（海明码），纠正1比特翻转错误

- 序号是2的幂次的位置放校验位，其它位置放数据位
- 任何一个数据位的序号都可以写成2的幂次的和，即由对应的校验位进行奇偶校验；该位置的数据位出错时，必然会影响到对应校验位的校验结果

◆ 检错码+重传纠错

- 奇偶校验：检测单比特错误/奇数个比特错误，ASCII码使用
- 交错校验：块数据使用， $n*k$ 矩阵，可检测出长度 $\leq n$ 的错误
- CRC（循环冗余校验）

(1)发送方：待校验数据补充 r 个0之后，对生成多项式进行模2除法，余数（ r 位）即为校验信息

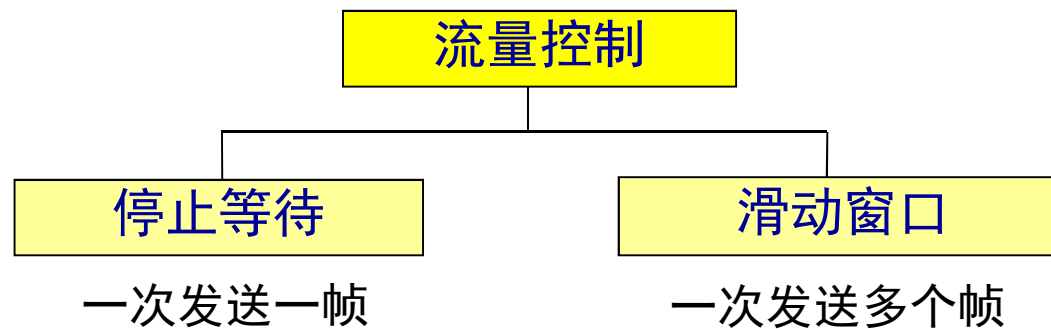
(2)接收方：待校验数据和校验信息一起，对生成多项式进行模2除法，余数不为0即认为有错

主要内容

- 数据链路层的位置、功能及服务
- 相关问题的设计
 - ◆ 成帧
 - ◆ 差错控制
 - ◆ 流量控制
 - 基本流量控制协议
 - 滑动窗口协议的性能
- 协议示例
 - ◆ HDLC
 - ◆ PPP

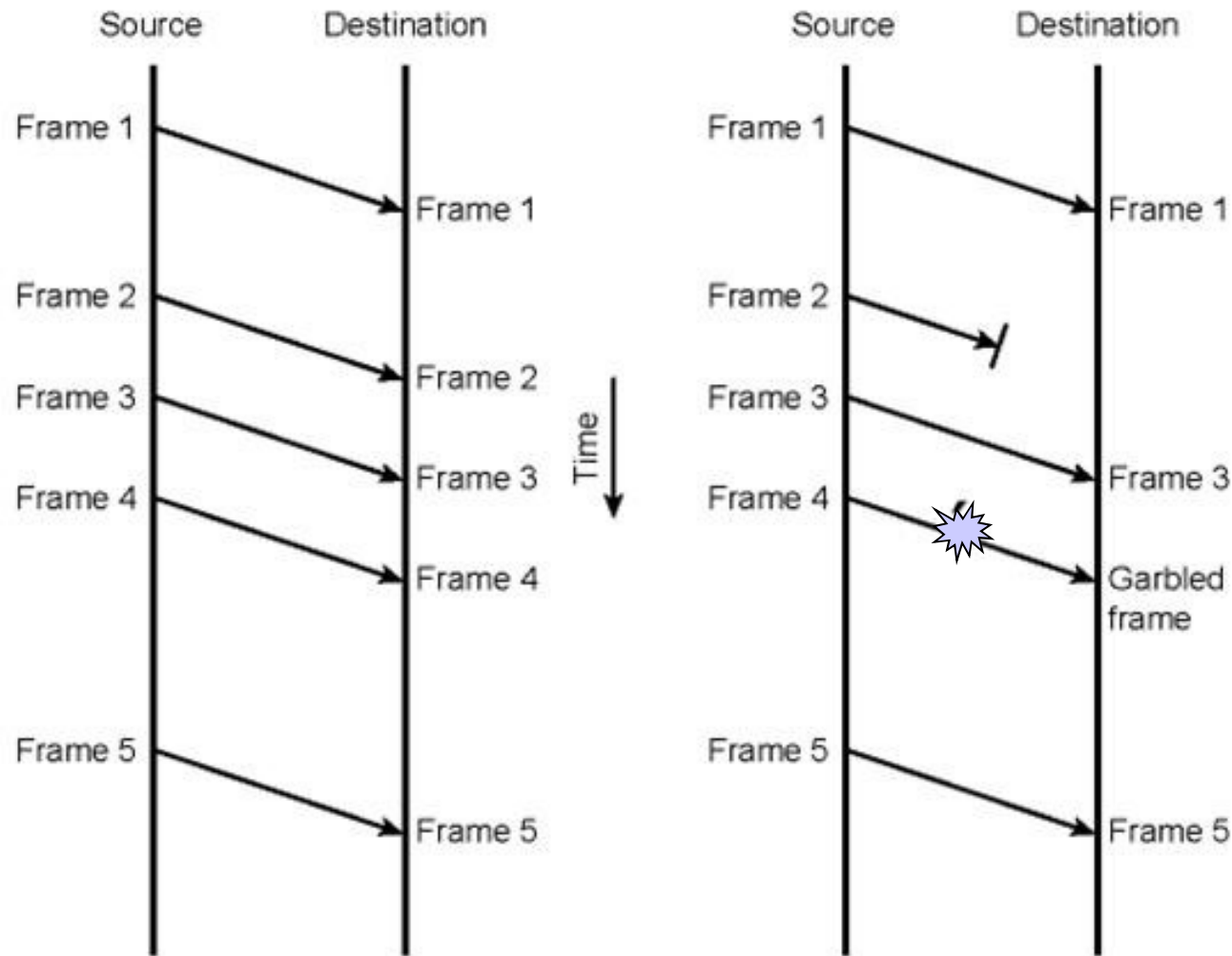
流量控制

- 确保发送实体不会淹没接收实体，即防止接收方缓存溢出
- **基于反馈的流量控制：**接收方向发送方发回信息，允许其发送更多的数据，或者至少告诉发送方其当前情况



- **基于速率的流量控制：**协议有内置的机制来限制发送方的发送速率，不使用接收方的反馈。
 - ◆ 将在网络层学习

为什么需要流量控制?



(a) Error-free transmission

无差错传输

(b) Transmission with losses and errors

传输中有丢失和差错

[Stallings]

基本的流量控制协议

■ 初级协议

- ◆ 乌托邦单工协议
- ◆ 无差错信道上的单工停止等待协议
- ◆ 有噪声信道上的单工协议

■ 滑动窗口协议

- ◆ 一比特滑动窗口协议
- ◆ Go Back N（回退N步）协议
- ◆ 选择重传协议

假定和库函数(1)

■ 一些假定

- ◆ 物理层、数据链路层和网络层是独立的进程
- ◆ **A**设备要发送一长串数据给**B**设备，使用可靠的、面向连级的服务
- ◆ 设备不会死机

假定和库函数(2)

■ 库函数

- ◆ wait_for_event
 - frame_arrival
 - timeout
 - chsum_err...
- ◆ from_network_layer / to_network_layer
- ◆ to_physical_layer / from_physical_layer
- ◆ timer operations
 - start_timer
 - stop_timer
 - timeout...
- ◆ enable_network_layer / disable_network_layer
- ◆ sequence number operation
 - inc (+1),

协议定义

```
#define MAX_PKT 1024                                /* determines packet size in bytes */

typedef enum {false, true} boolean;                  /* boolean type */
typedef unsigned int seq_nr;                          /* sequence or ack numbers */
typedef struct {unsigned char data[MAX_PKT];} packet; /* packet definition */
typedef enum {data, ack, nak} frame_kind;             /* frame_kind definition */

typedef struct {                                      /* frames are transported in this layer */
    frame_kind kind;                                  /* what kind of a frame is it? */
    seq_nr seq;                                       /* sequence number */
    seq_nr ack;                                       /* acknowledgement number */
    packet info;                                      /* the network layer packet */
} frame;
```

这些定义包含在文件 `protocol.h` 中

```

/* Wait for an event to happen; return its type in event. */
void wait_for_event(event_type *event);

/* Fetch a packet from the network layer for transmission on the channel. */
void from_network_layer(packet *p);

/* Deliver information from an inbound frame to the network layer. */
void to_network_layer(packet *p);

/* Go get an inbound frame from the physical layer and copy it to r. */
void from_physical_layer(frame *r);

/* Pass the frame to the physical layer for transmission. */
void to_physical_layer(frame *s);

/* Start the clock running and enable the timeout event. */
void start_timer(seq_nr k);

/* Stop the clock and disable the timeout event. */
void stop_timer(seq_nr k);

/* Start an auxiliary timer and enable the ack_timeout event. */
void start_ack_timer(void);

/* Stop the auxiliary timer and disable the ack_timeout event. */
void stop_ack_timer(void);

/* Allow the network layer to cause a network_layer_ready event. */
void enable_network_layer(void);

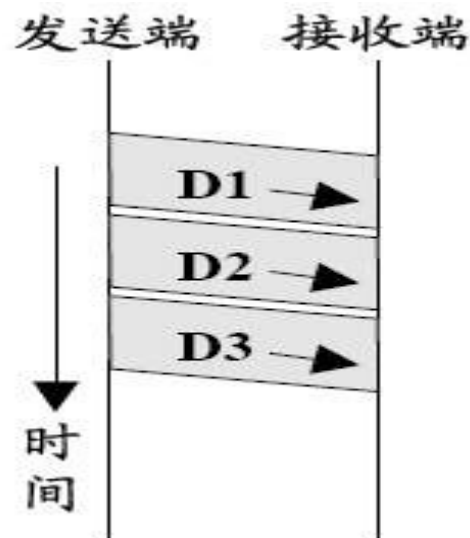
/* Forbid the network layer from causing a network_layer_ready event. */
void disable_network_layer(void);

/* Macro inc is expanded in-line: Increment k circularly. */
#define inc(k) if (k < MAX_SEQ) k = k + 1; else k = 0

```

协议1：乌托邦单工协议

- 假定
 - ◆ 信道是单向数据传输的
 - ◆ 信道是无差错的
 - ◆ 接收方有无限大的缓存
- 不需要差错控制和流量控制



协议1: 乌托邦 单工协议

/* Protocol 1 (utopia) provides for data transmission in one direction only, from sender to receiver. The communication channel is assumed to be error free, and the receiver is assumed to be able to process all the input infinitely quickly. Consequently, the sender just sits in a loop pumping data out onto the line as fast as it can. */

```
typedef enum {frame arrival} event type;
#include "protocol.h"
```

```
void sender1(void)
{
    frame s;                                /* buffer for an outbound frame */
    packet buffer;                          /* buffer for an outbound packet */

    while (true) {
        from_network_layer(&buffer); /* go get something to send */
        s.info = buffer;             /* copy it into s for transmission */
        to_physical_layer(&s);       /* send it on its way */
    }                                /* Tomorrow, and tomorrow, and tomorrow,
                                   Creeps in this petty pace from day to day
                                   To the last syllable of recorded time
                                   - Macbeth, V, v */
}

void receiver1(void)
{
    frame r;
    event_type event;                    /* filled in by wait, but not used here */

    while (true) {
        wait_for_event(&event);        /* only possibility is frame_arrival */
        from_physical_layer(&r);       /* go get the inbound frame */
        to_network_layer(&r.info);    /* pass the data to the network layer */
    }
}
```


协议2：无差错信道上的单工停止等待协议

■ 假定

- ◆ 信道是单工单向数据传输
- ◆ 信道是无差错的
- ◆ 接收方的缓存有限

■ 无需差错控制

■ 需要流量控制

- ◆ **Stop-and-wait**（停止等待/停等）
 - 发送方发出一帧，停下来等待
 - 接收方发回一个空帧（确认/应答，**ACK**）
 - 接收到**ACK**后，发送方发送下一帧

协议2： 单工停止 等待协议

/* Protocol 2 (stop-and-wait) also provides for a one-directional flow of data from sender to receiver. The communication channel is once again assumed to be error free, as in protocol 1. However, this time, the receiver has only a finite buffer capacity and a finite processing speed, so the protocol must explicitly prevent the sender from flooding the receiver with data faster than it can be handled. */

```
typedef enum {frame_arrival} event_type;
#include "protocol.h"
```

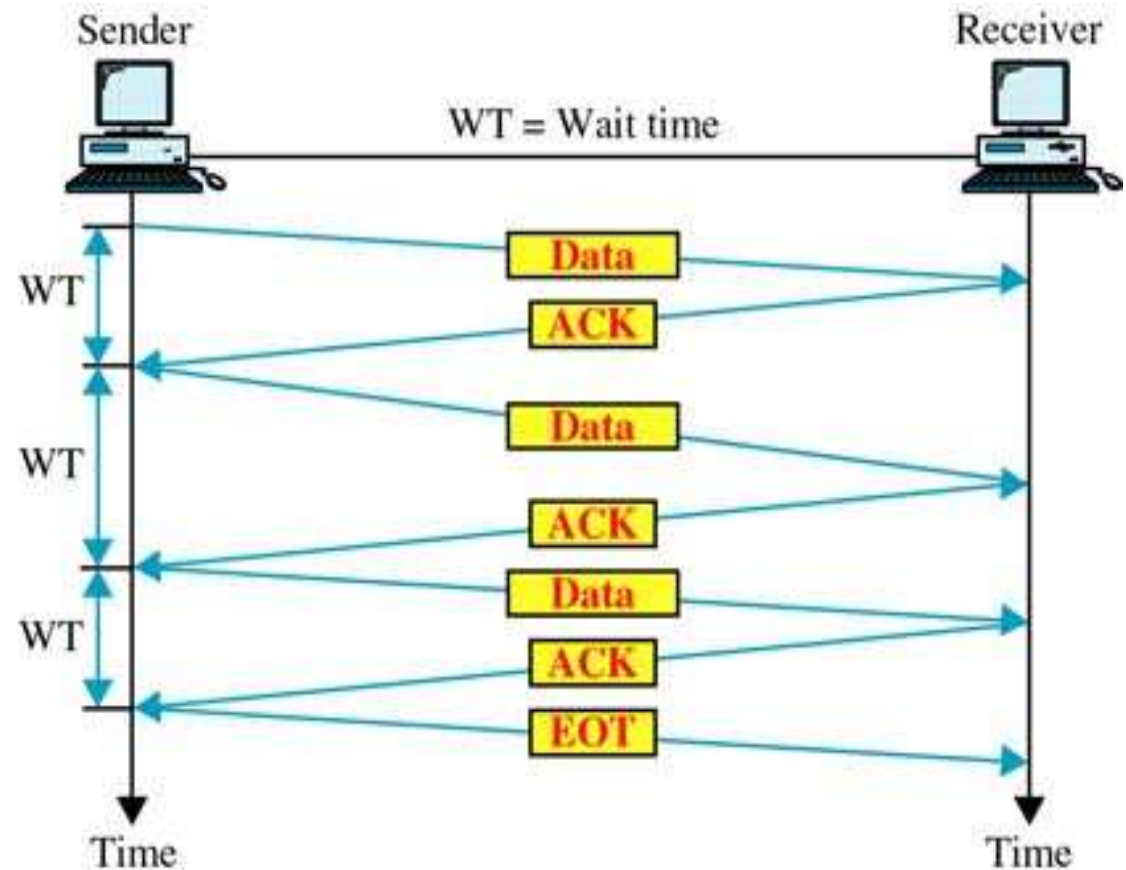
```
void sender2(void)
{
    frame s;                                /* buffer for an outbound frame */
    packet buffer;                          /* buffer for an outbound packet */
    event_type event;                       /* frame_arrival is the only possibility */

    while (true) {
        from_network_layer(&buffer);        /* go get something to send */
        s.info = buffer;                    /* copy it into s for transmission */
        to_physical_layer(&s);              /* bye bye little frame */
        wait_for_event(&event);             /* do not proceed until given the go ahead */
    }
}
```

```
void receiver2(void)
{
    frame r, s;                             /* buffers for frames */
    event_type event;                       /* frame_arrival is the only possibility */
    while (true) {
        wait_for_event(&event);             /* only possibility is frame_arrival */
        from_physical_layer(&r);            /* go get the inbound frame */
        to_network_layer(&r.info);          /* pass the data to the network layer */
        to_physical_layer(&s);              /* send a dummy frame to awaken sender */
    }
}
```

停止等待

- 发送方发出一帧
- 接收方收到帧，以 **ACK** 应答
- 发送方在发送下一帧之前要等待 **ACK**
- 接收方可以通过不发 **ACK** 来停止数据流



基本的流量控制协议

- 流量控制功能：限制发送方的速度，防止淹没接收方
- 流量控制方法：停止等待——发送方每发完一帧，即停下来等待；收到接收方反馈（**ACK**），发送方才能继续发送
- 协议1：理想信道（无错不丢）
 - ◆ 无需流量控制
- 协议2：信道无错，但接收方缓存有限
 - ◆ 流量控制：停止等待

协议2的问题：如果信道不可靠？

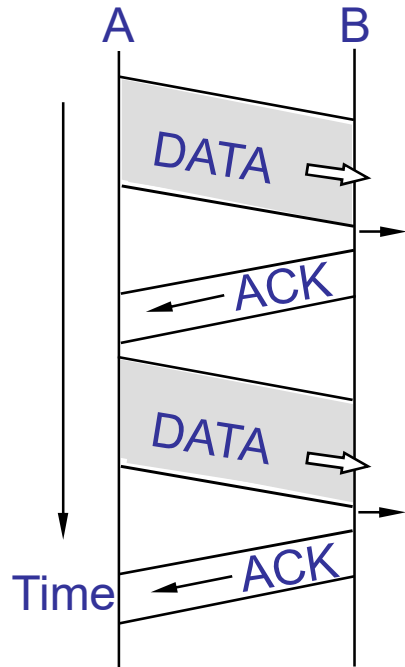
■ 如果一帧被毁坏

- ◆ 接收方通过 **CRC** 检测出差错，丢弃该帧
- ◆ 发送方**超时**，**重发**该帧

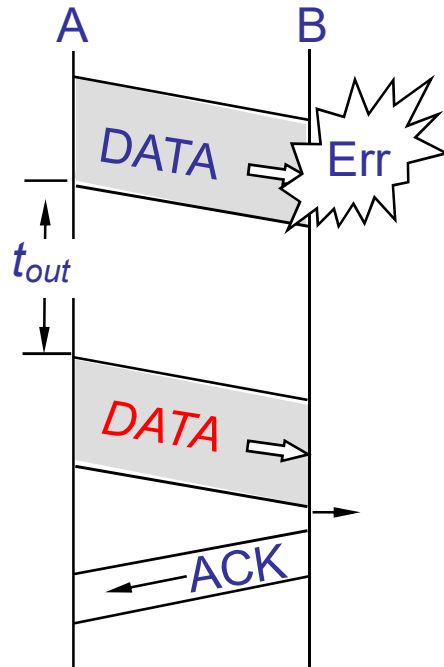
■ 如果一帧丢失

- ◆ 数据帧丢失
 - 发送方**超时重发**
- ◆ **ACK**丢失
 - 发送方**超时重发**

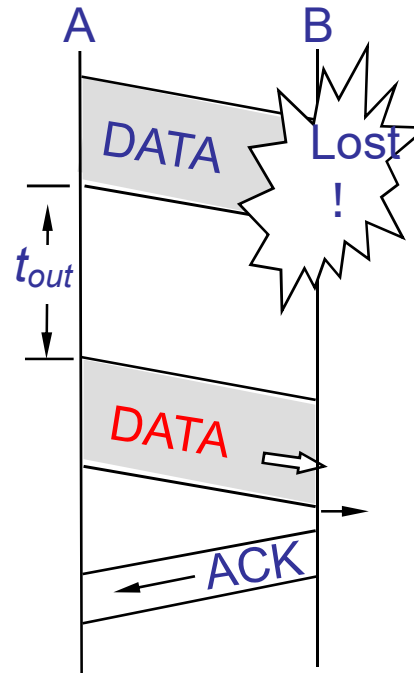
停止等待



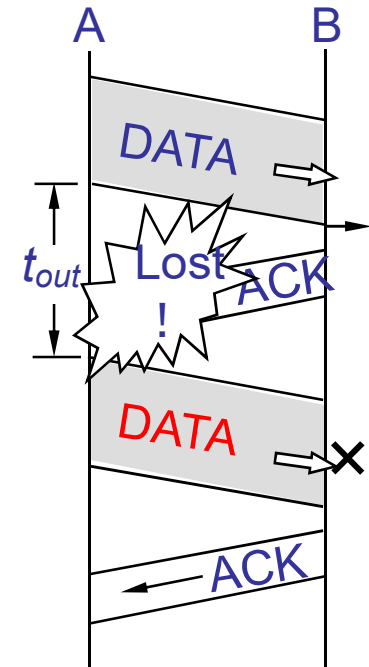
(a) Normal



(b) Data Error



(c) Data Lost



(d) ACK Lost

更多的问题

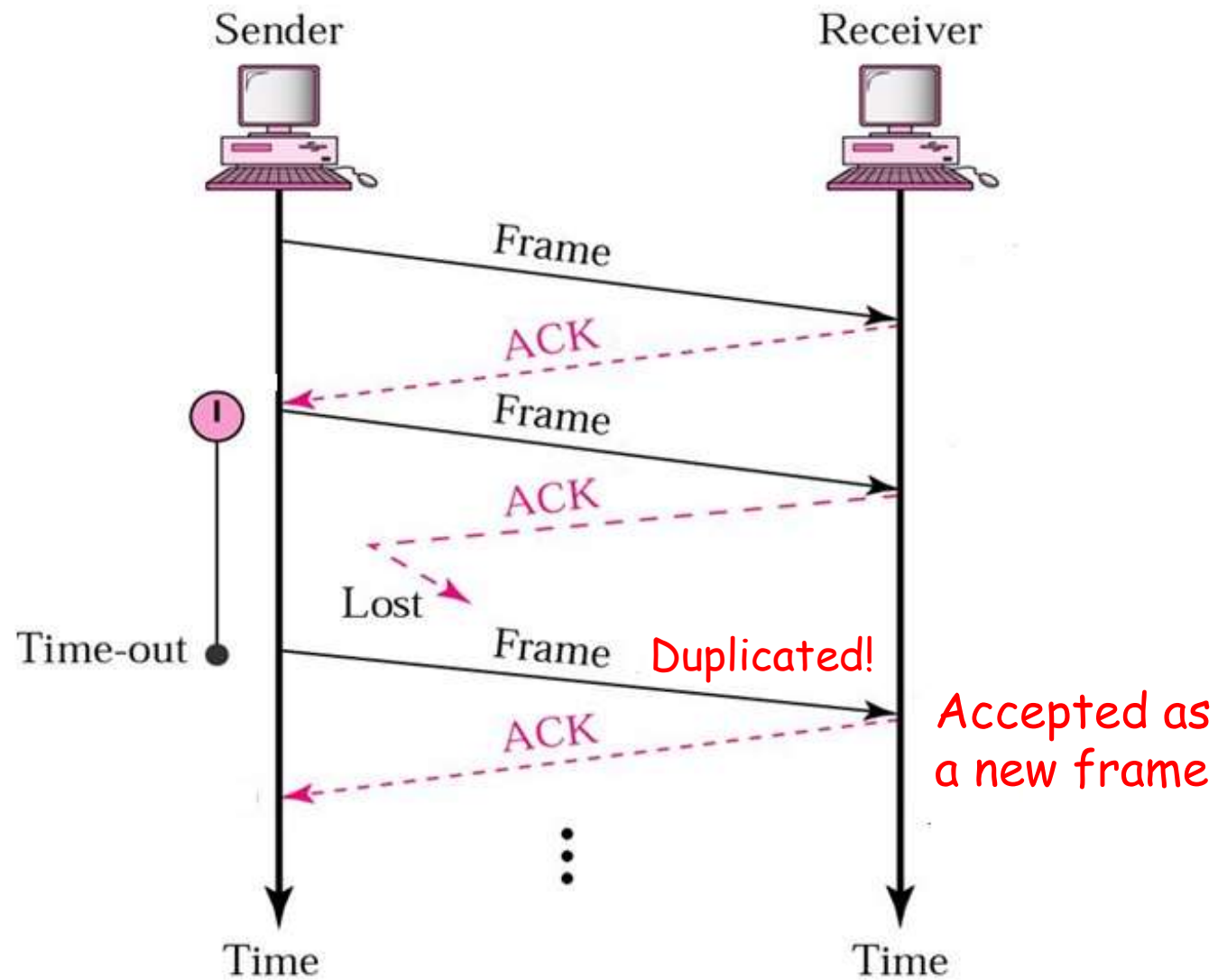
■ 情形1：协议在接收方失败

- ◆ 发送方**A**的网络层将数据包**1**交给数据链路层
接收方**B**正确接收到该包，并交给**B**的网络层
B发回**ACK**给**A**
- ◆ **ACK**丢失
- ◆ **A**的数据链路层最终超时。由于没有收到确认，**A**重发包含数据包**1**的帧
- ◆ 这个**重复帧** 到达**B**，**B**把重复的包**1**交给网络层

■ 情形2：协议在发送方失败

- ◆ 被延误的**ACK**

出错的情形1：ACK丢失



解决方案：使用序号

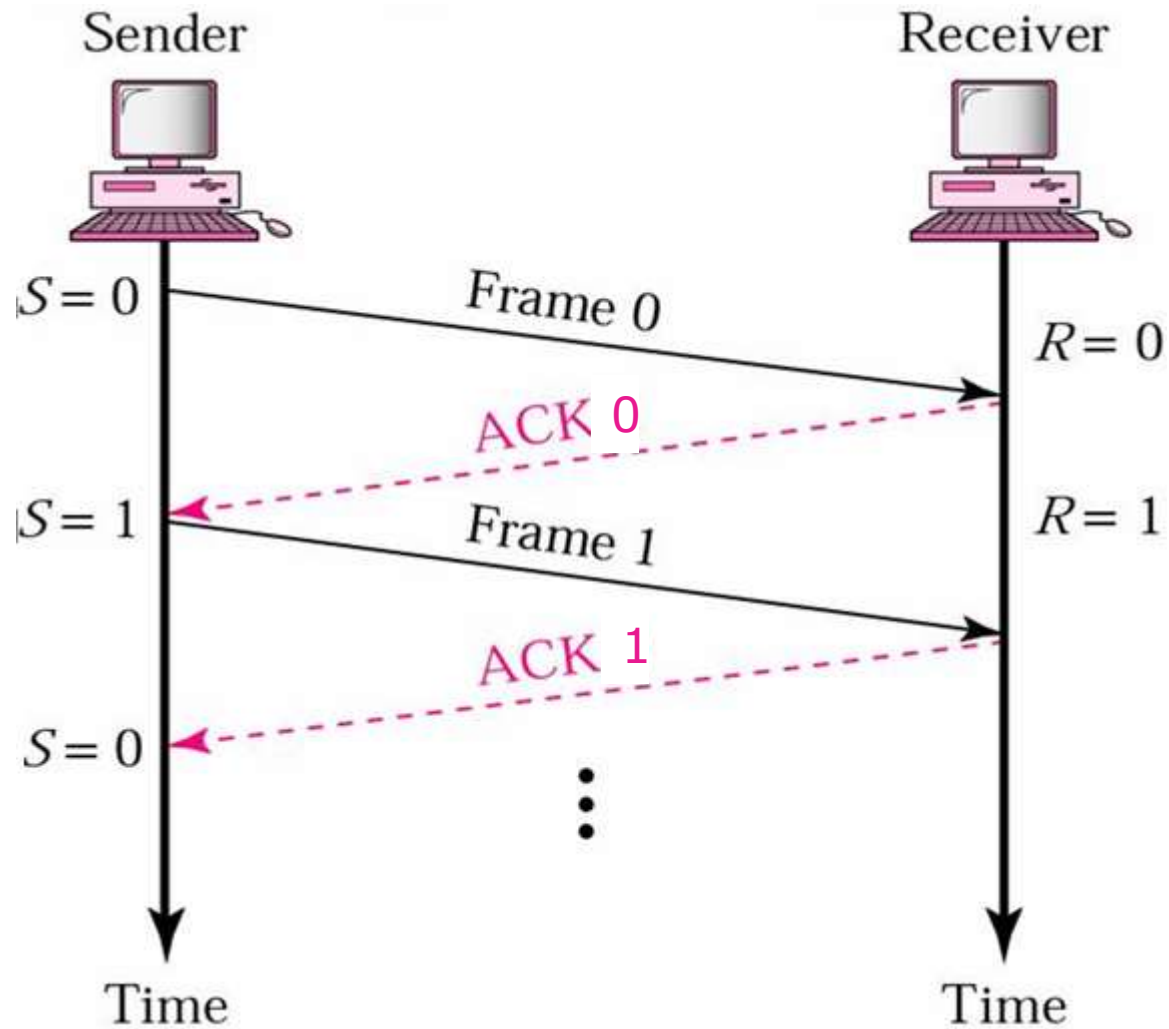
- 用于区分第一次收到的帧和重传的帧
 - ◆ 在每个帧的头部放一个序号
- 序号最少需要多少位？
 - ◆ 数据链路层：1位，一共有两个序号（0 和 1）
 - ◆ 传输层：需要更大的序号空间，如16位

 ARQ: Automatic Repeat reQuest
(自动请求重传)

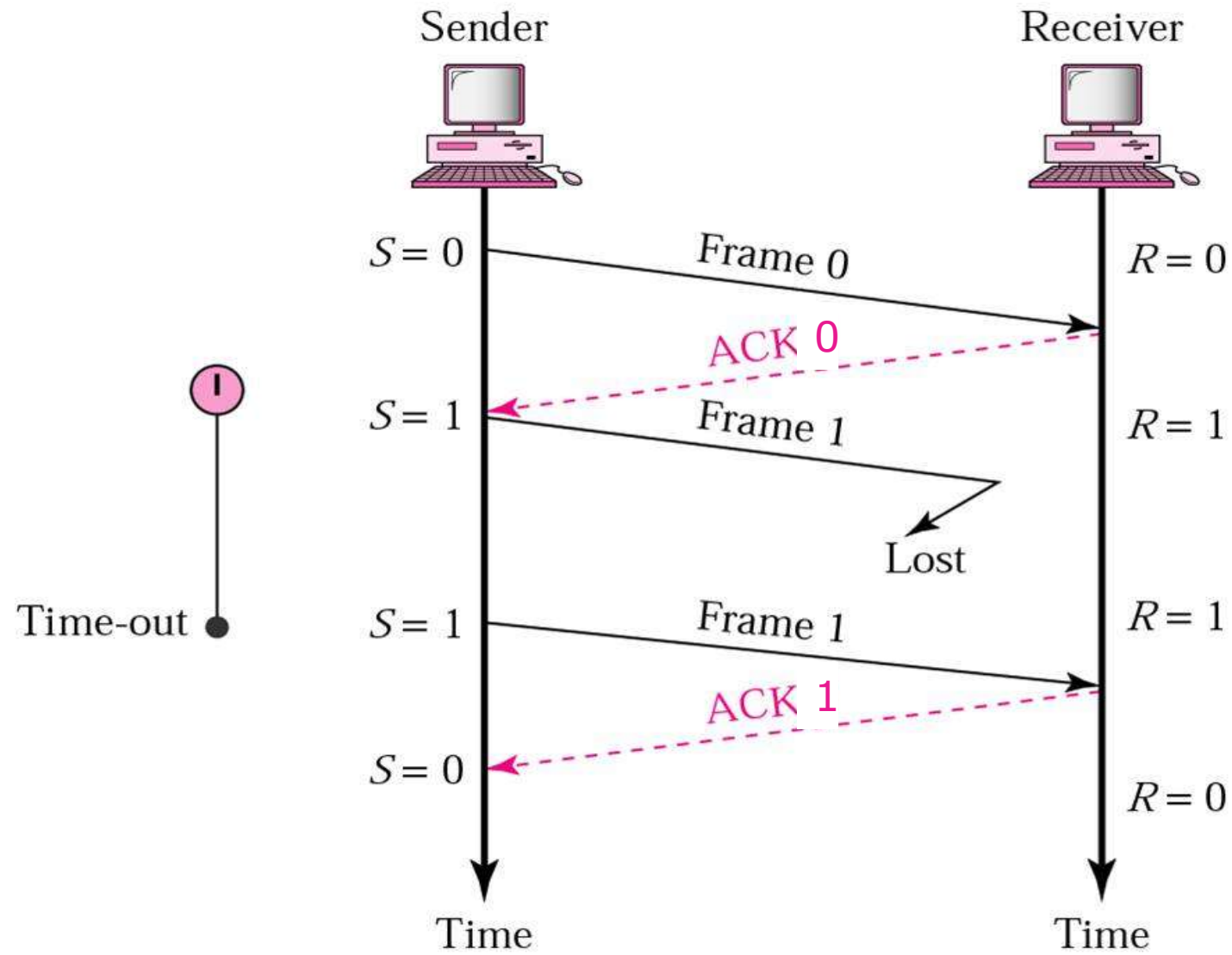
ARQ的工作过程

- 差错检测
 - ◆ **CRC/FCS**校验错 或 帧丢失
- 确认
 - ◆ 接收方返回**ACK**，可能不是每个帧都返回**ACK**
- 发送方超时重传
 - ◆ 使用**序号** 来检测重复帧
- 否认（可选）
 - ◆ 接收方返回**NAK**，发送方重传

停止等待ARQ: 正常操作

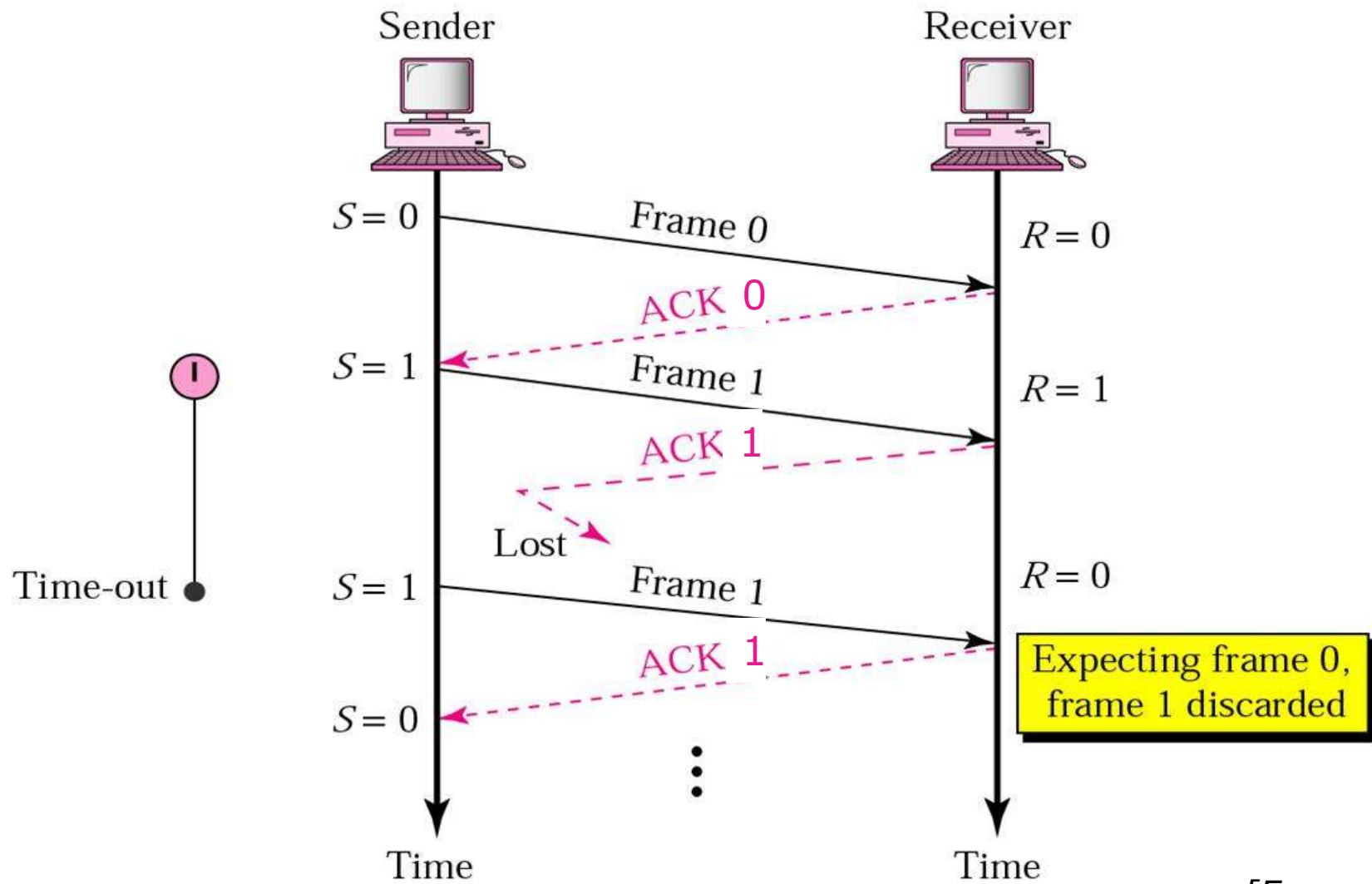


传输差错情形1：数据帧丢失



[Forouzan]

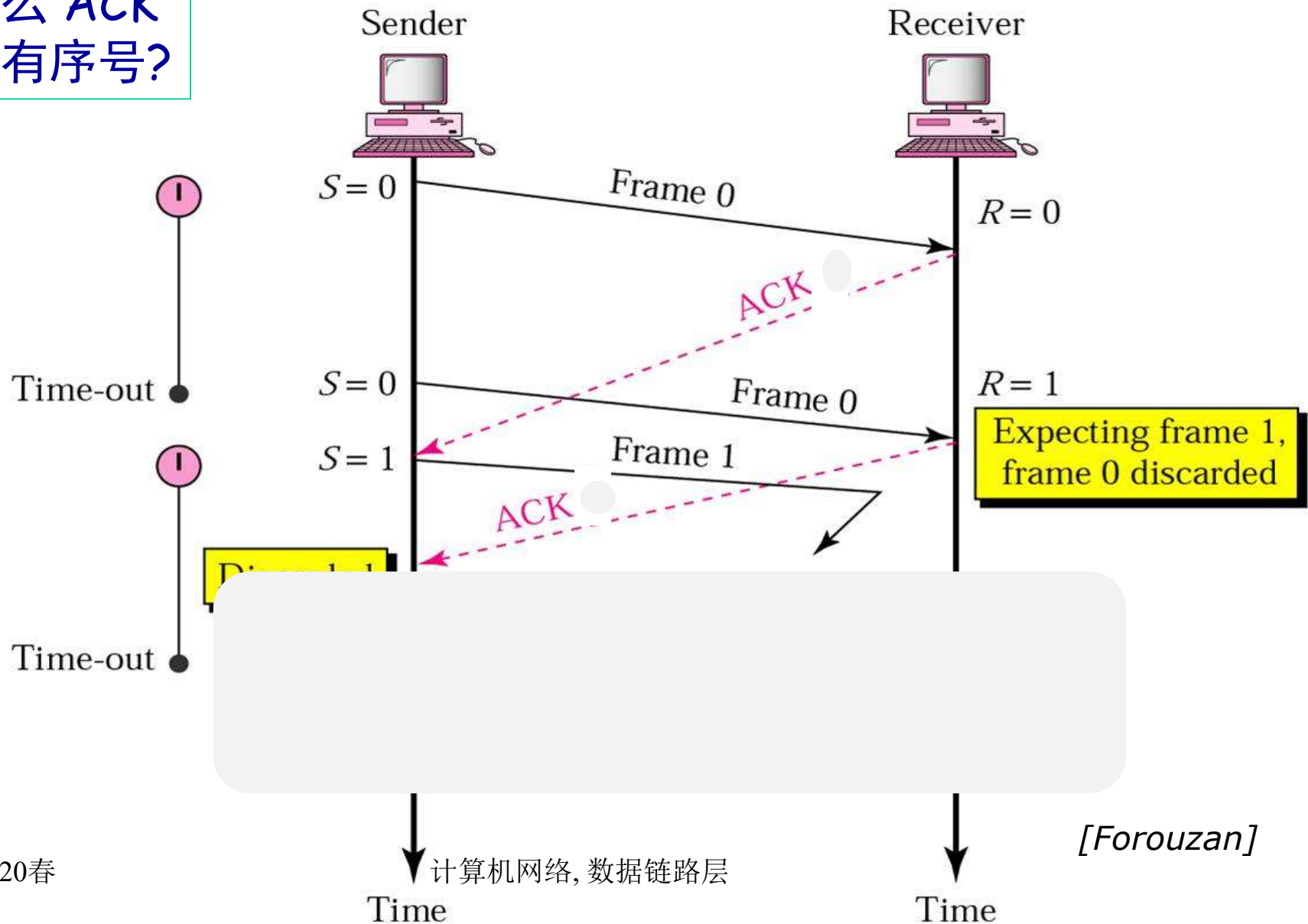
传输差错情形2: ACK丢失



[Forouzan]

传输差错情形3: ACK被延误

为什么 ACK
也要有序号?



单向传输数据的停止等待ARQ协议

- 发送方在数据帧中增加**校验**字段，发送帧之后启动**超时定时器**
- 接收方检错，出错则丢弃此帧；如果校验无错，接收方发送确认帧（**ACK**）
- 收到ACK后，发送方发送下一帧，重启定时器
- 如果定时器超时，发送方**重传**刚发送的数据帧
- **数据帧**中携带**序号**，接收方根据序号来判断是否是重复的数据帧
- **ACK**中携带**序号**，发送方根据序号判断是否是重复的ACK

协议3：带重传的ACK (PAR)

■ 假定

- ◆ 信道是单工单向数据传输
- ◆ 信道是不可靠的
- ◆ 接收方的缓存有限

■ 比协议2增强了什么？

- ◆ 帧序号
- ◆ 超时和重传


```

/* Protocol 3 (par) allows unidirectional data flow over an unreliable channel. */
#define MAX_SEQ 1 /* must be 1 for protocol 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
    seq_nr next_frame_to_send; /* seq number of next outgoing frame */
    frame s; /* scratch variable */
    packet buffer; /* buffer for an outbound packet */
    event_type event;

    next_frame_to_send = 0; /* initialize outbound sequence numbers */
    from_network_layer(&buffer); /* fetch first packet */
    while (true) {
        s.info = buffer; /* construct a frame for transmission */
        s.seq = next_frame_to_send; /* insert sequence number in frame */
        to_physical_layer(&s); /* send it on its way */
        start_timer(s.seq); /* if answer takes too long, time out */
        wait_for_event(&event); /* frame_arrival, cksum_err, timeout */
        if (event == frame_arrival) {
            from_physical_layer(&s); /* get the acknowledgement */
            if (s.ack == next_frame_to_send) {
                stop_timer(s.ack); /* turn the timer off */
                from_network_layer(&buffer); /* get the next one to send */
                inc(next_frame_to_send); /* invert next_frame_to_send */
            }
        }
    }
}

```

Sender

```

void receiver3(void)
{
    seq_nr frame_expected;
    frame r, s;
    event_type event;

    frame_expected = 0;
    while (true) {
        wait_for_event(&event);
        if (event == frame_arrival) {
            from_physical_layer(&r);
            if (r.seq == frame_expected) {
                to_network_layer(&r.info);
                inc(frame_expected);
            }
            s.ack = 1 - frame_expected;
            to_physical_layer(&s);
        }
    }
}

```

Ack序号 = 刚收到的帧的序号

```

/* possibilities: frame_arrival, cksum_err */
/* a valid frame has arrived. */
/* go get the newly arrived frame */
/* this is what we have been waiting for. */
/* pass the data to the network layer */
/* next time expect the other sequence nr */

/* tell which frame is being acked */
/* send acknowledgement */

```

Receiver

带重传的ACK协议(PAR)

考虑差错控制:

Sender

```
从网络层获取一个分组放入buffer
发送buffer中的数据, 新启动定时器
label1:
wait_for_event()
switch (event) {
case 收到了坏帧(校验和错):
    重发缓冲在buffer里的数据, 重新启动定时器
case 定时器超时:
    重发缓冲在buffer里的数据, 重新启动定时器
case 收到校验和正确的帧:
    if (ack序号正确) {
        关闭旧定时器
        从网络层获取下一个分组放入buffer
        发送buffer中的数据, 启动新的定时器
    } else
        重发缓冲在buffer里的数据, 重新启动定时器
}
goto label1
```

考虑差错控制: Receiver

```
frame_expected=0
while(true) {
    wait_for_event()
    switch(event) {
    case 坏帧:
        do_nothing; or send NAK
    case 收到校验和正确的数据帧:
        if(序号==frame_expected) {
            向网络层上交分组
            回ACK(序号为frame_expected)
            inc(frame_expected)
        } else {
            回ACK (序号为1-frame_expected)
        }
    }
}
```

关于定时器

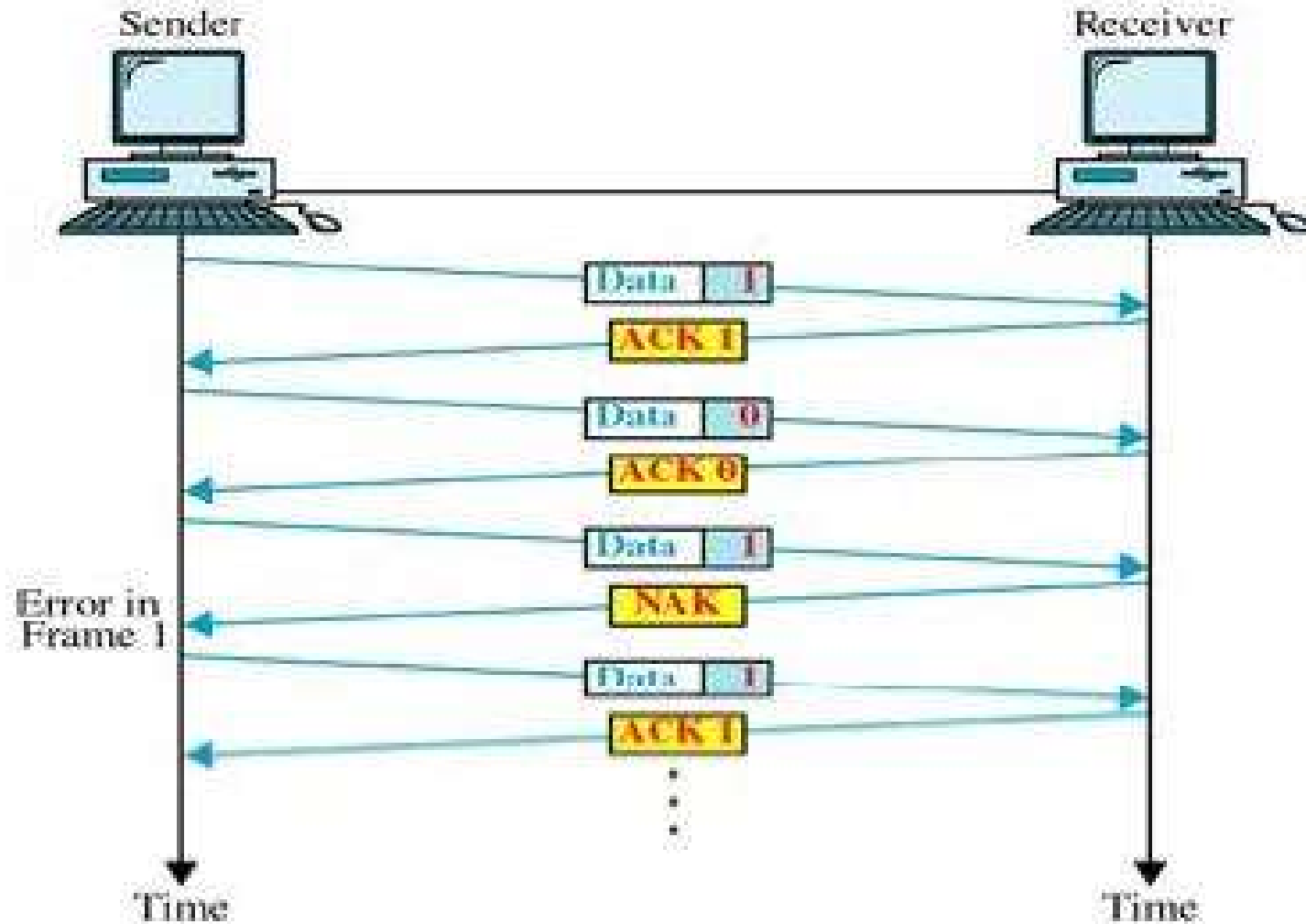
■ 超时

- ◆ 如果超时时间间隔设置过短，发送方将产生不必要的重传。虽然这些额外的帧不会影响协议的正确性，但会影响性能。

■ 关于NAK

- ◆ NAK是必须的吗？
- ◆ 使用NAK有什么优点？

停止等待ARQ: 可选的NAK



基本的流量控制协议

■ 初级协议

- ◆ 乌托邦单工协议
- ◆ 无差错信道上的单工停止等待协议
- ◆ 有噪声信道上的单工协议

■ 滑动窗口协议

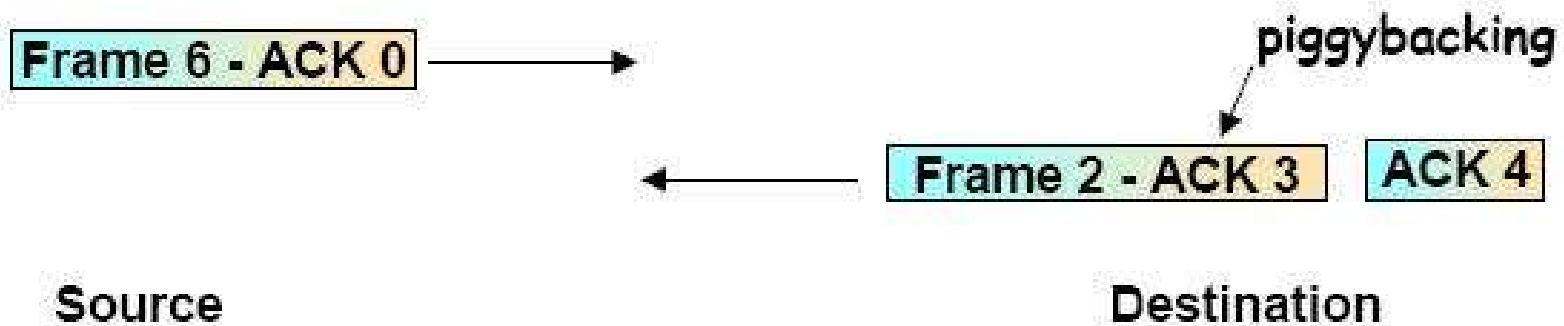
- ◆ 一比特滑动窗口协议
- ◆ Go Back N（回退N步）协议
- ◆ 选择重传协议

滑动窗口流量控制：原则

- 全双工数据发送, 允许捎带应答(piggybacking)
- 允许一次发送多个数据帧
- 在收到ACK之前, 发送方可以发送最多W个帧
 - ◆ W是发送窗口的大小
- 接收方也许可以缓存多个帧
- 序号循环使用 (n比特)
 - ◆ 帧按照模 2^n 编号 ($0 \dots 2^n - 1$)
 - ◆ 窗口大小不一定要达到最大值

捎带应答/确认

- When a data frame arrives, instead of immediately sending a separate control frame (ACK), the receiver restrains itself and waits until the network layer passes it the next packet
- ACK is attached to the outgoing data frame (using the ACK field in the frame header)



发送窗口 & 接收窗口

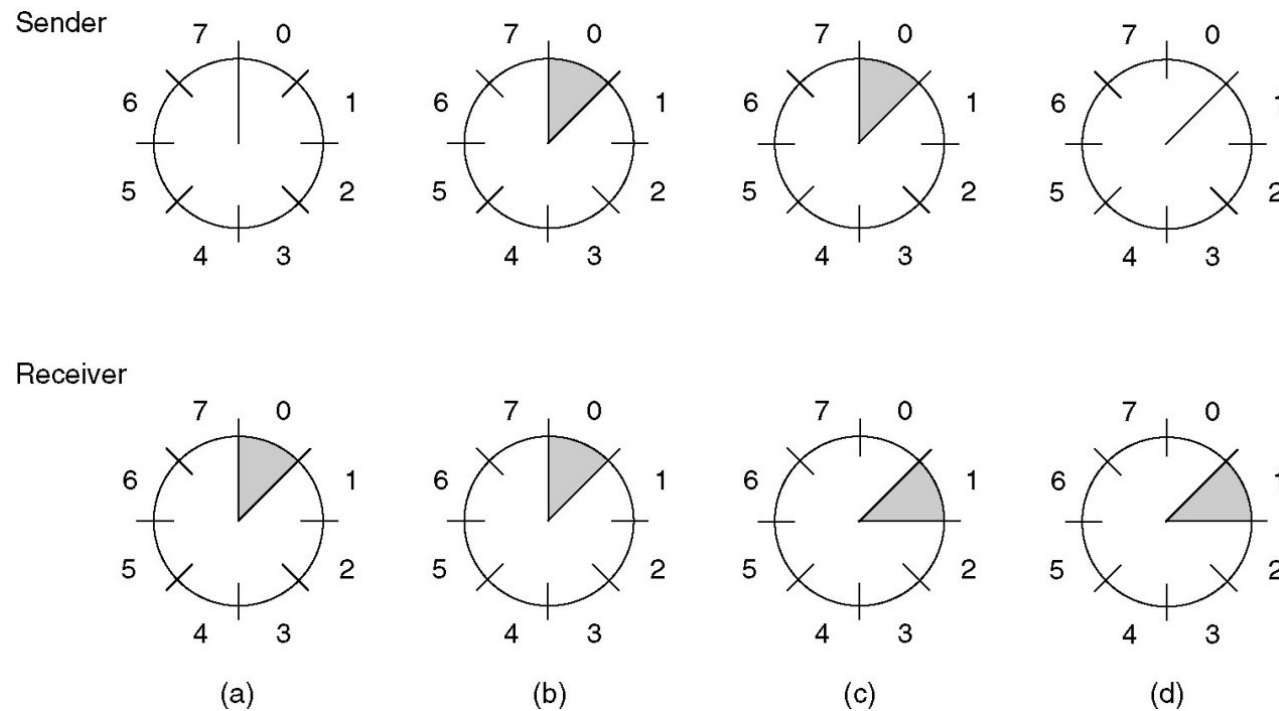
■ 发送窗口

- ◆ 任何瞬间，发送方维护一个序号集合，这些序号对应着可以发送的帧
- ◆ 可以说，这些帧落在发送窗口内

■ 接收窗口

- ◆ 接收方也维护一个接收窗口，窗口内的序号对应着可以接收的帧

大小为1的滑动窗口



滑动窗口大小为1，使用3位序号

(a) 初始窗口

(b) 发出第一帧

(c) 收到第一帧

(d) 收到第一个ACK

一比特滑动窗口协议 (1)

```
/* Protocol 4 (sliding window) is bidirectional. */
#define MAX_SEQ 1 /* must be 1 for protocol 4 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"
void protocol4 (void)
{
    seq_nr next_frame_to_send; /* 0 or 1 only */
    seq_nr frame_expected; /* 0 or 1 only */
    frame r, s; /* scratch variables */
    packet buffer; /* current packet being sent */
    event_type event;

    next_frame_to_send = 0; /* next frame on the outbound stream */
    frame_expected = 0; /* frame expected next */
    from_network_layer(&buffer); /* fetch a packet from the network layer */
    s.info = buffer; /* prepare to send the initial frame */
    s.seq = next_frame_to_send; /* insert sequence number into frame */
    s.ack = 1 - frame_expected; /* piggybacked ack */
    to_physical_layer(&s); /* transmit the frame */
    start_timer(s.seq); /* start the timer running */
}
```

一比特滑动窗口协议 (2)

```
while (true) {  
    wait_for_event(&event);  
    if (event == frame_arrival) {  
        from_physical_layer(&r);  
        if (r.seq == frame_expected) {  
            to_network_layer(&r.info);  
            inc(frame_expected);  
        }  
        if (r.ack == next_frame_to_send) {  
            stop_timer(r.ack);  
            from_network_layer(&buffer);  
            inc(next_frame_to_send);  
        }  
        s.info = buffer;  
        s.seq = next_frame_to_send;  
        s.ack = 1 - frame_expected;  
        to_physical_layer(&s);  
        start_timer(s.seq);  
    }  
}
```

收到正确序号帧，提交网络层，接收窗口前移1格

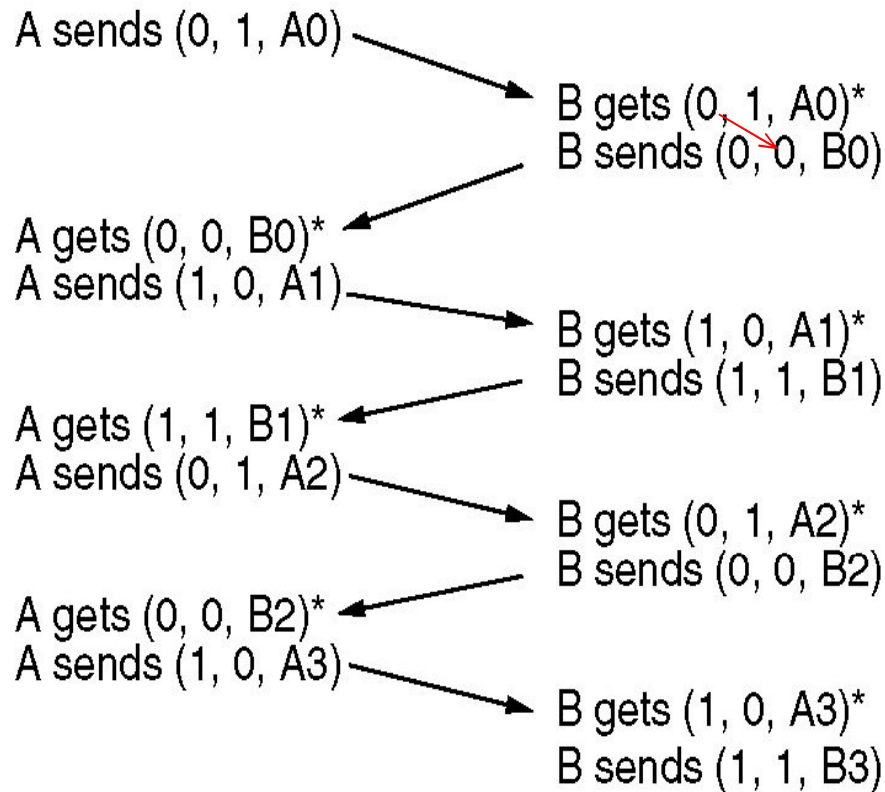
上帧已正确到达，获取网络层新包，帧序号+1

/* frame_arrival, cksum_err, or timeout */
/* a frame has arrived undamaged. */
/* go get it */
/* handle inbound frame stream. */
/* pass packet to network layer */
/* invert seq number expected next */
/* handle outbound frame stream. */
/* turn the timer off */
/* fetch new pkt from network layer */
/* invert sender's sequence number */
/* construct outbound frame */
/* insert sequence number into it */
/* seq number of last received frame */
/* transmit a frame */
/* start the timer running */

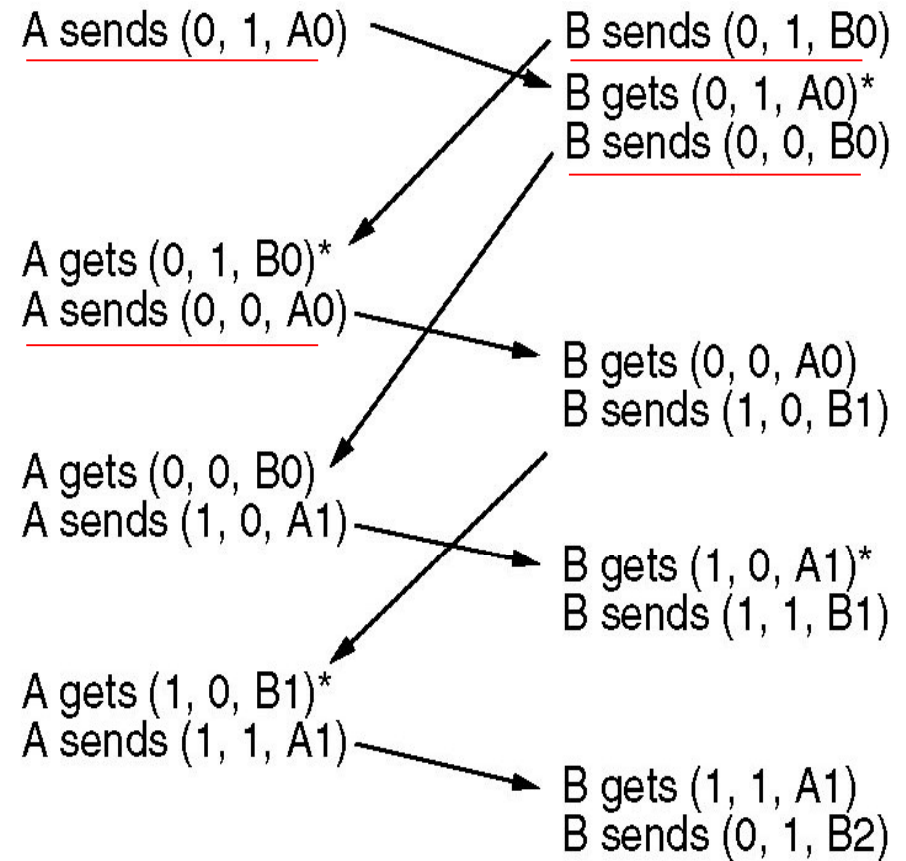
2020春

计算机网络, 数据链路层

协议4的特定情形：同时发送



正常情形



异常情形：每帧发送两次

括号的记法：(帧序号, ACK序号, 包编号)

* 表示网络层接受一个数据包

ACK序号 = 刚收到的帧的序号

(U)

实用的停止等待ARQ协议

- 发送方在数据帧中增加**校验**字段，发送帧之后启动**超时定时器**
- 接收方检错，出错则丢弃此帧；如果校验无错，且为非重复帧，则提交网络层并发送确认帧（**ACK**）或者在回传的数据帧中**捎带确认**；
- 收到确认后，发送方发送下一帧，重启定时器
- 如果定时器超时，发送方**重传**刚发送的数据帧
- **数据帧**中携带**发送序号**，接收方根据序号来判断是否是重复的数据帧
- 发送方根据**ACK序号**判断是否是重复的ACK

协议4的问题：效率低

- 对于带宽为**50kbps**的卫星信道，往返传播时延为 **500**毫秒，发送多个长度为**1000**位的帧
 - ◆ $t=0$: 发送方开始发送
 - ◆ $t=20\text{ msec}$: 该帧发送完毕
 - ◆ $t=270\text{ msec}$: 该帧到达接收方
 - ◆ $t=520\text{ msec}$: 最好的情况下（帧在接收方没有等待，**ACK**帧很短，即忽略接收方的排队/处理时延和**ACK**的发送时延），**ACK**到达发送方
- 结论
 - ◆ 在**500/520** 或**96%** 的时间内，发送方都处于阻塞状态（即等待**ACK**），只用到**4%**的带宽

基本的流量控制协议

■ 初级协议

- ◆ 乌托邦单工协议
- ◆ 无差错信道上的单工停止等待协议
- ◆ 有噪声信道上的单工协议

■ 滑动窗口协议

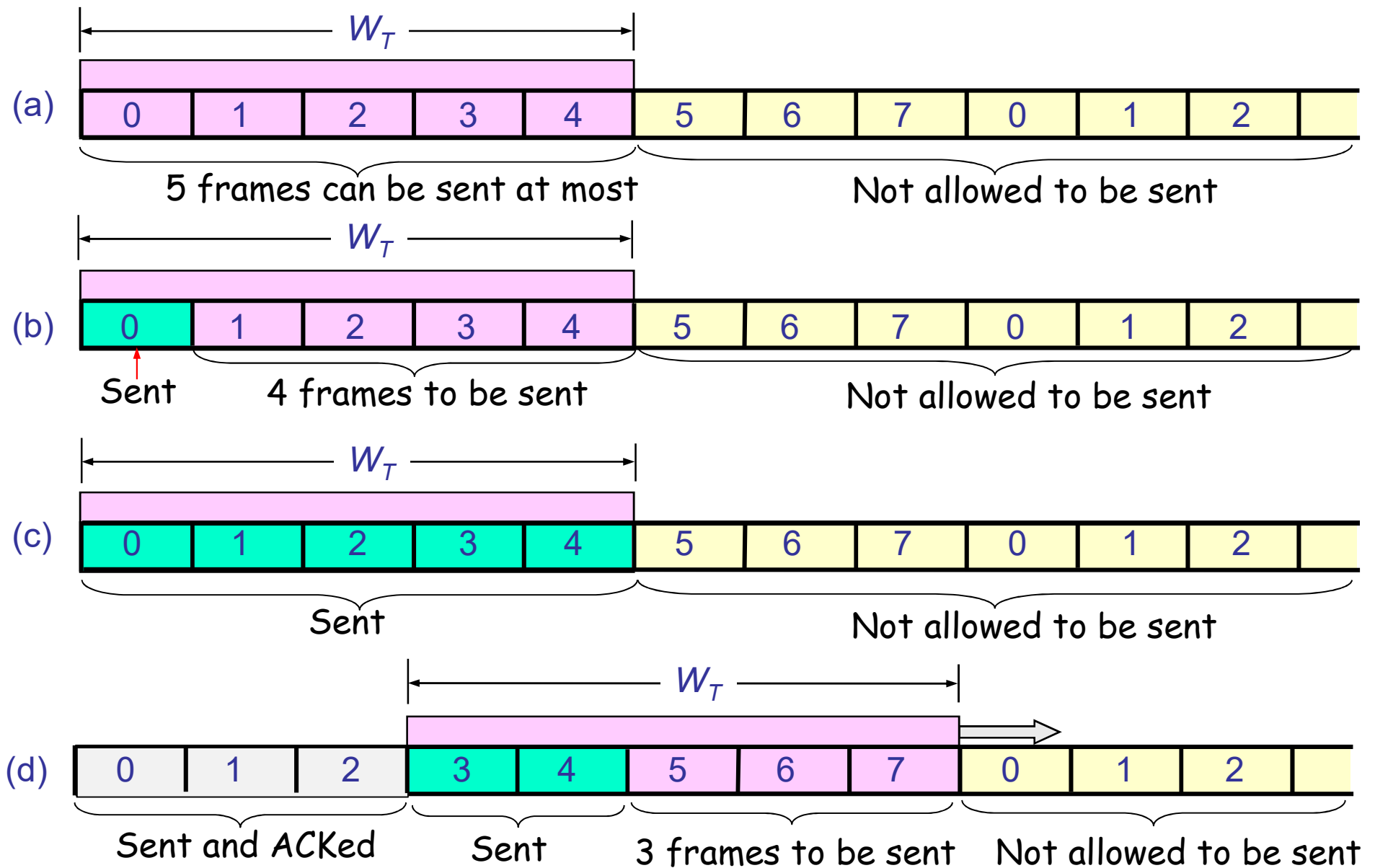
- ◆ 一比特滑动窗口协议
- ◆ Go Back N (回退N步) 协议
- ◆ 选择重传协议

Go Back N（回退N步）：原则

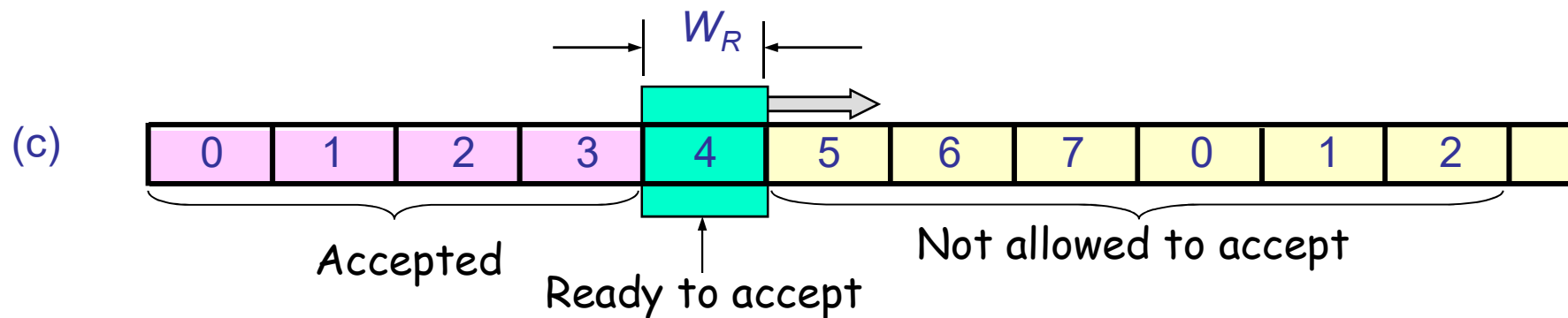
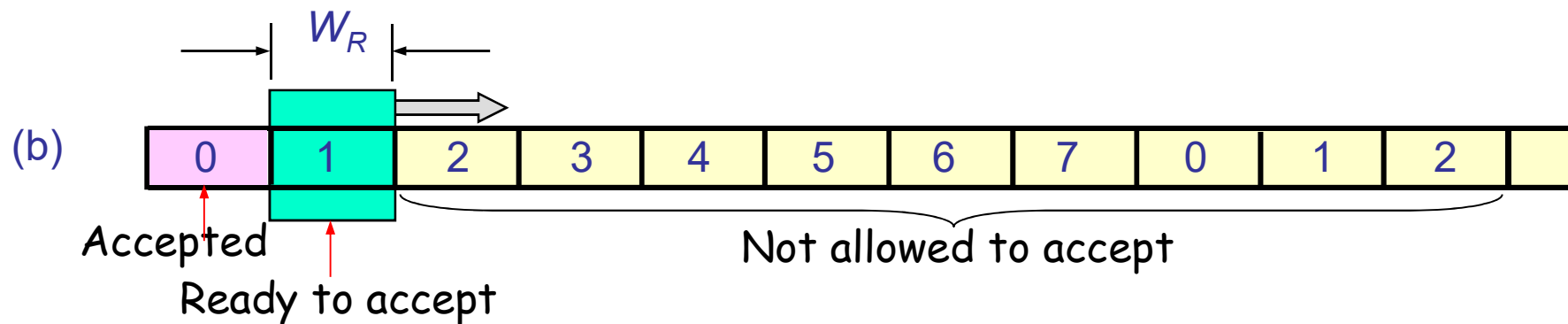
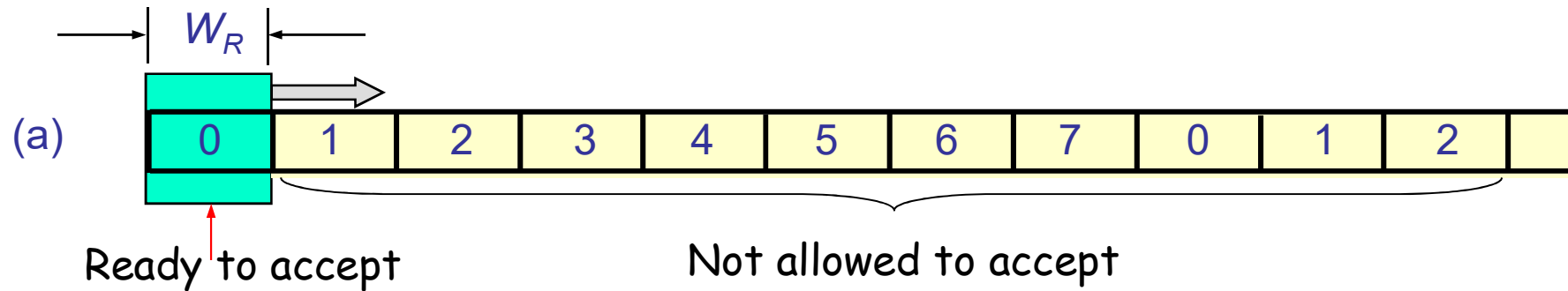
基于滑动窗口

- ◆ 发送窗口：缓存未解决的帧（待发帧及已发送未确认的帧）
- ◆ 接收窗口：接收帧的缓存
- 使用窗口来控制未解决的帧数
 - ◆ 发送方在收到**ACK**之前，可以发送多个帧
 - ◆ 流水线（**pipelining**）
- 如果接收方检测到差错，将丢弃出错帧及后续所有到达的帧，直到正确收到重发的原出错帧
- 发送方必须 回退、重传出错帧及后续的全部帧

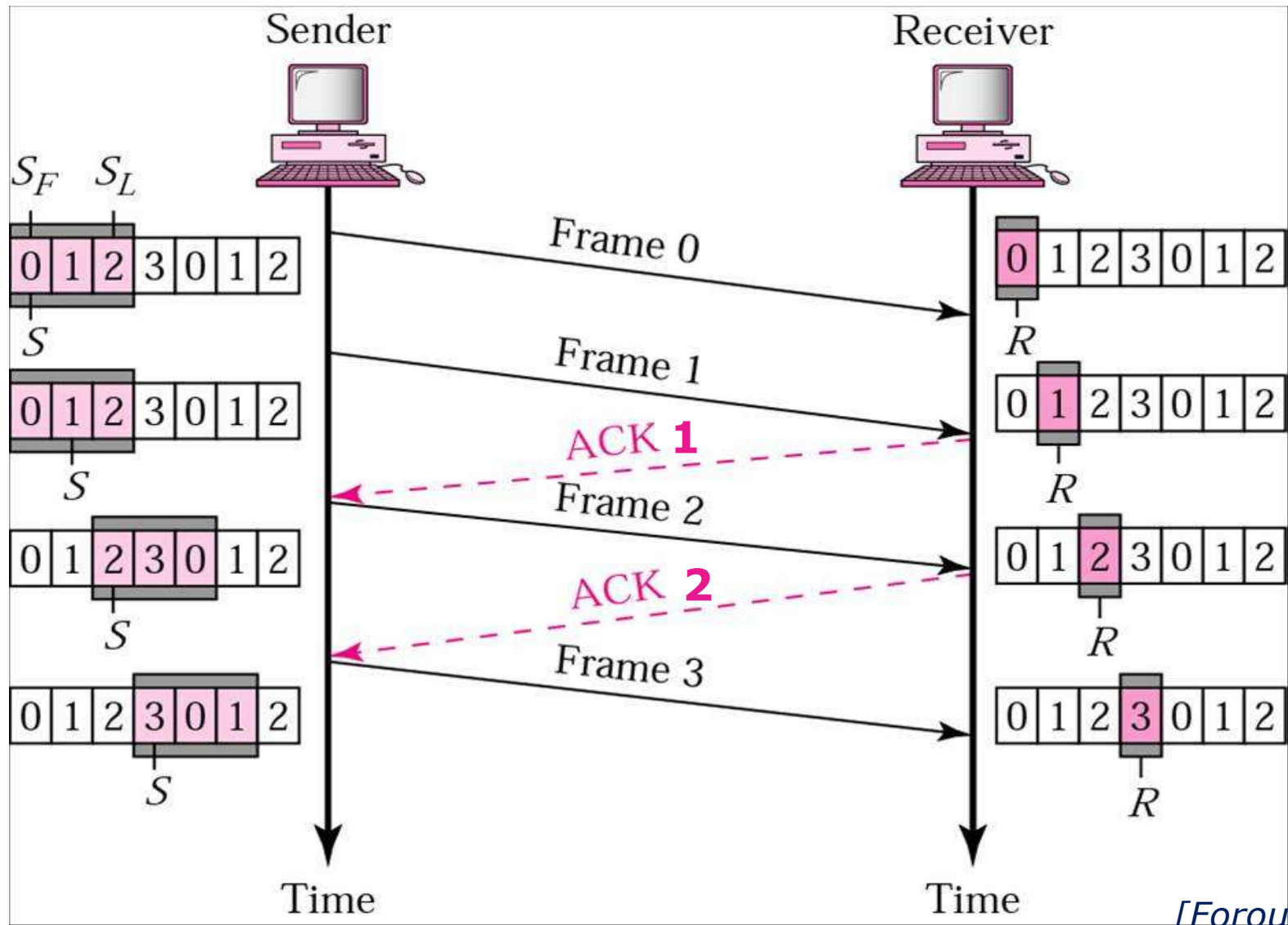
GBN的发送窗口: $W_T > 1$



GBN的接收窗口: $W_R=1$

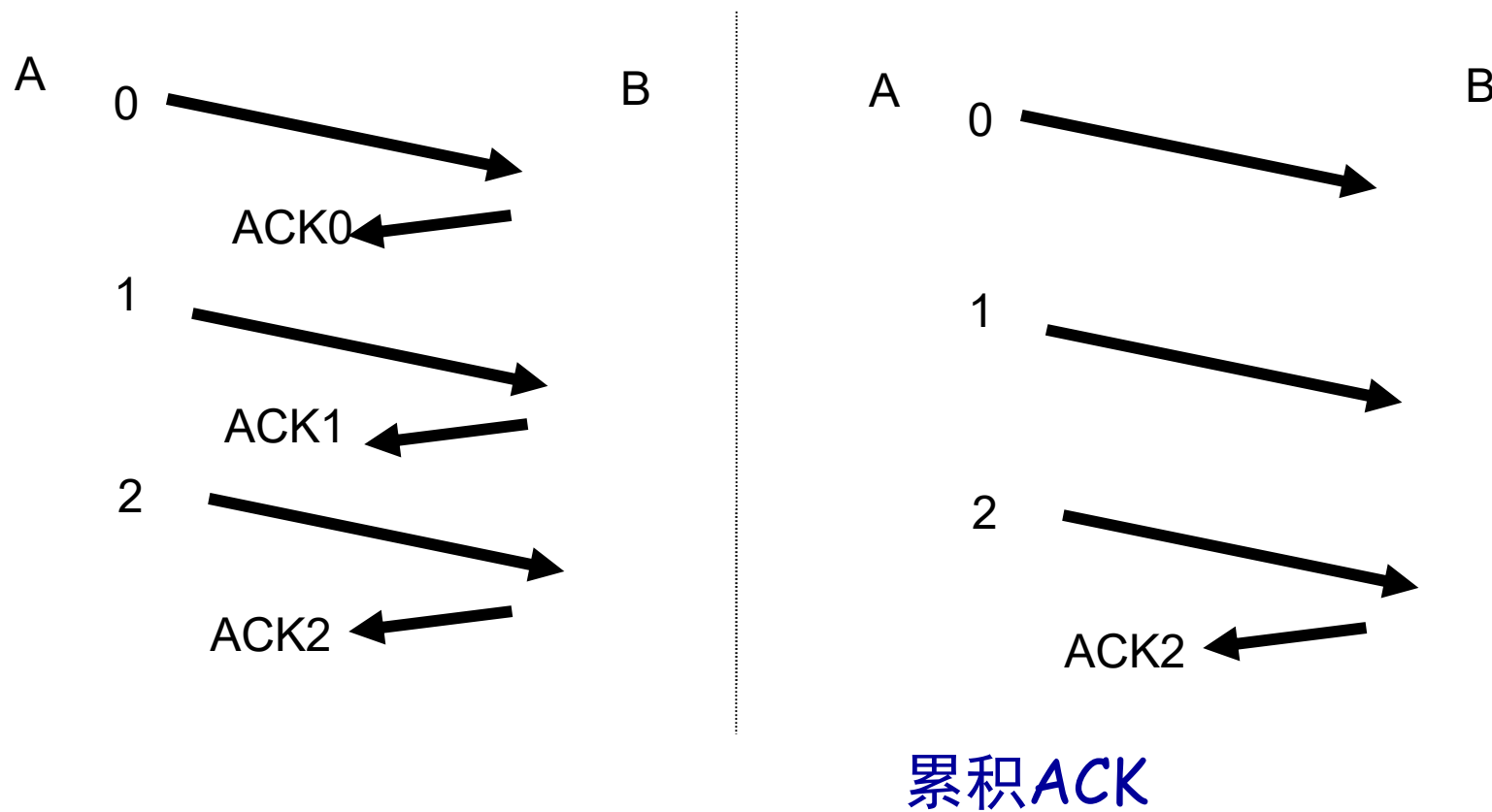


Go Back N: 正常操作

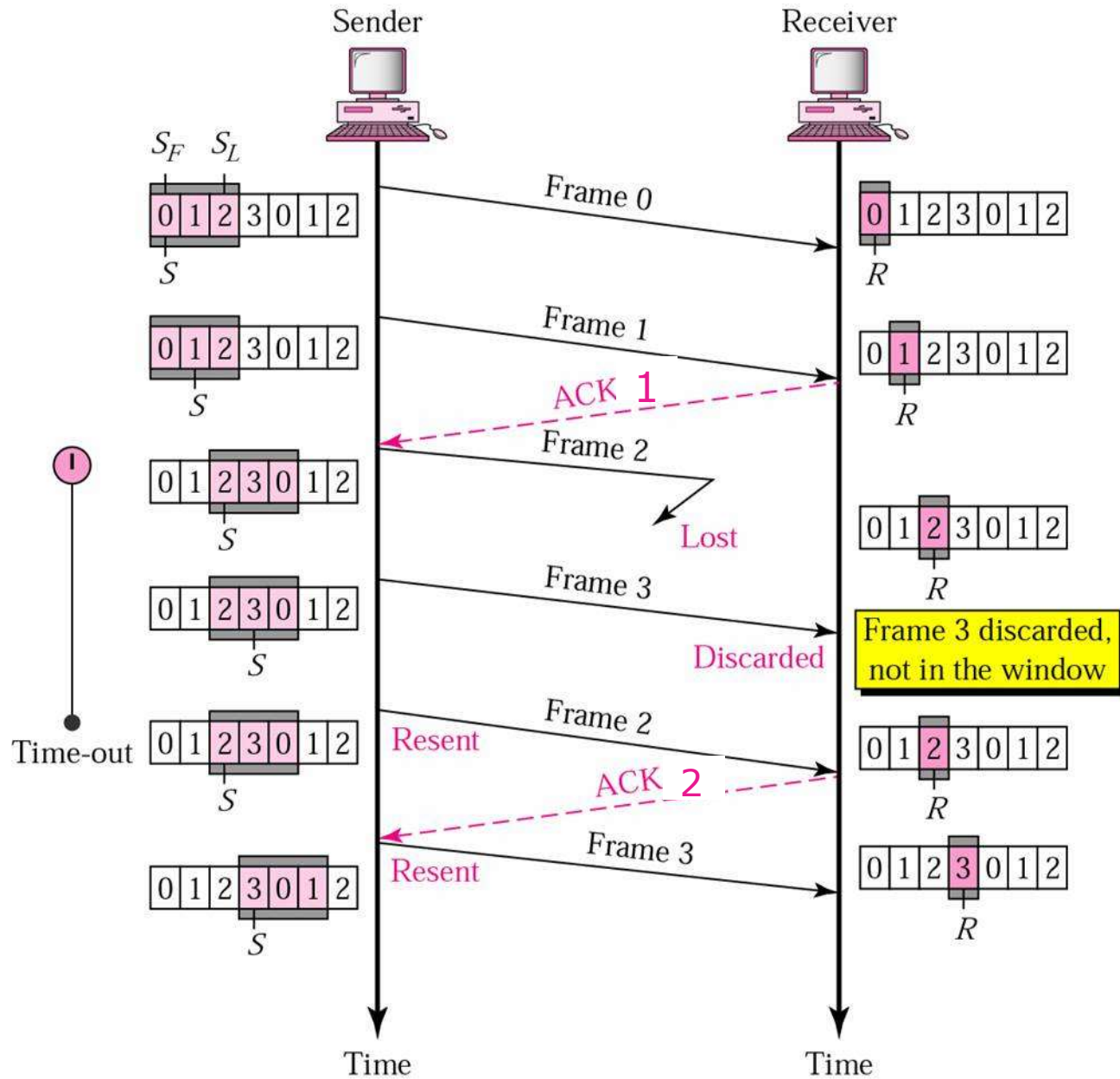


[Forouzan]

Go Back N: 不需要对每个帧都回复ACK

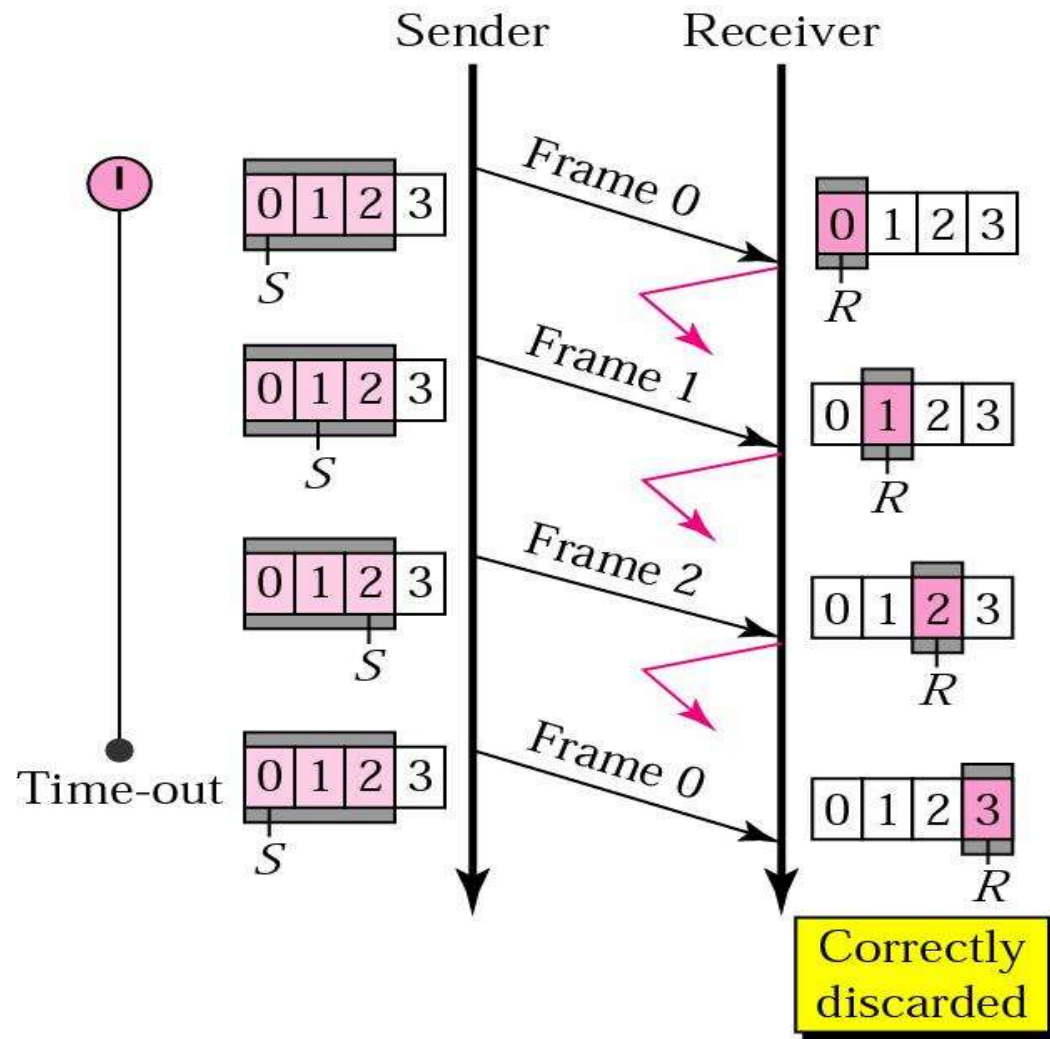


Go Back N: 帧丢失



[Forouzan]

Go Back N: ACK丢失



Go Back N: 帧被毁坏

- 如果数据帧被毁坏
 - ◆ 接收方检测为 i 号帧有错
 - ◆ 丢弃 i 号帧及后面收到的所有帧
 - ◆ 发送方超时重发 i 号帧及后面的所有已发帧
- 如果ACK被毁坏
 - ◆ 接收方收到 i 号帧，发送 $ACK(i)$ ，ACK丢失
 - ◆ 由于ACK是累积的，下一个 $ACK(i+n)$ 可能在发送方超时之前到达，因此不需要重传
 - ◆ 如果发送方超时，重发 i 号帧及后面的所有已发帧

Go Back N的信道效率

- 对于带宽为50kbps的卫星信道，往返传播时延为500毫秒，发送多个长度为1000位的帧，发送窗口 $W_T=26$

- ◆ $t=0$: 发送方开始发送
- ◆ $t=20\text{ ms}$: 第一帧发完
- ◆ $t=270\text{ ms}$: 第一帧到达接收方
- ◆ $t=520\text{ ms}$: 第26帧发出
- ◆ $t=520\text{ ms}$: ACK 到达发送方

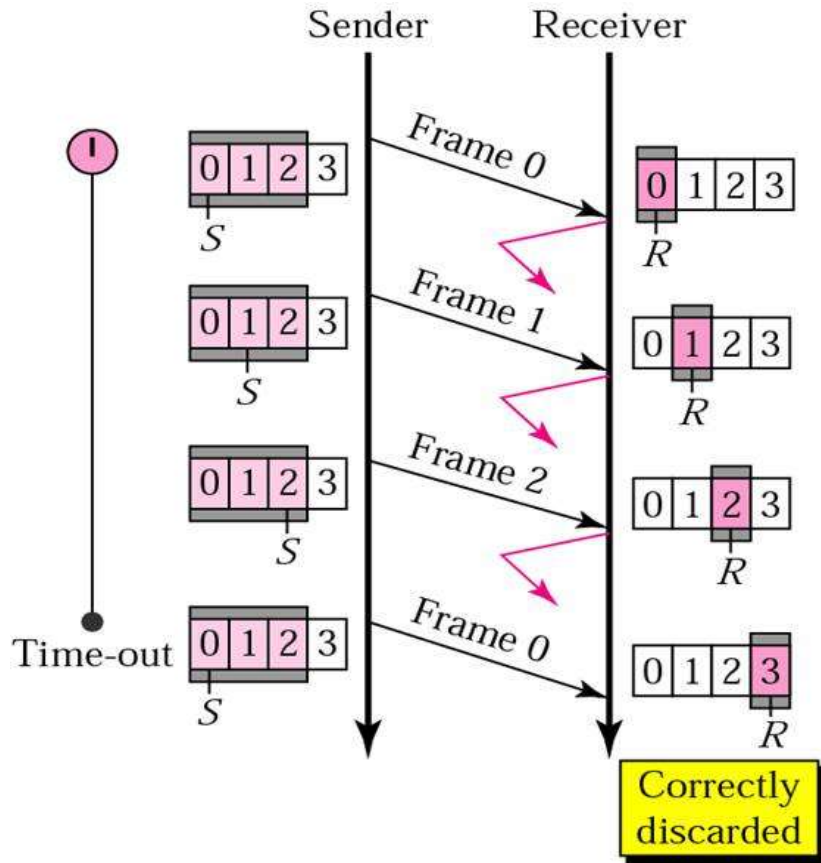
- 信道效率

- ◆ $520/520=100\%$
- ◆ 流水线

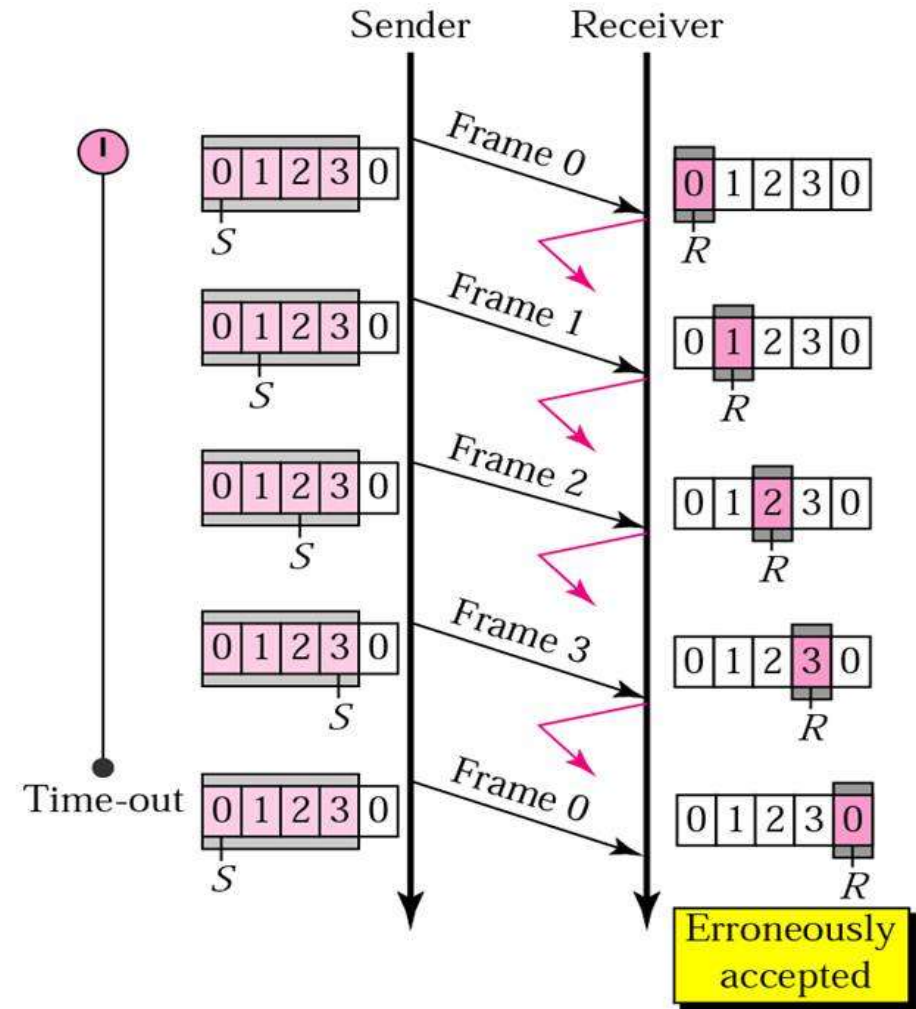
最大发送窗口

- 使用n位序号，窗口大小可以达到 2^n 吗？
- 假设使用3位序号，考虑下列情况：
 - 发送方发送0-7号帧
 - 接收方收到全部8个帧，发回ACK
 - 所有的ACK都丢失了
 - 发送方重传0号帧
- 问题：接收方能够检测出这是重复帧吗？
- 原则
 - ◆ 当发送方发送完发送窗口内所有帧，在未收到ACK之前，接收方 正确接收到所有发送来的帧，接收窗口向前推移，此时，必须保证发送窗口与接收窗口在序号上不能重叠
 - ◆ 对于n位序号， $W_T \leq 2^n - 1$

最大发送窗口：2位序号示例 ($m=2$)



a. Window size $< 2^m$

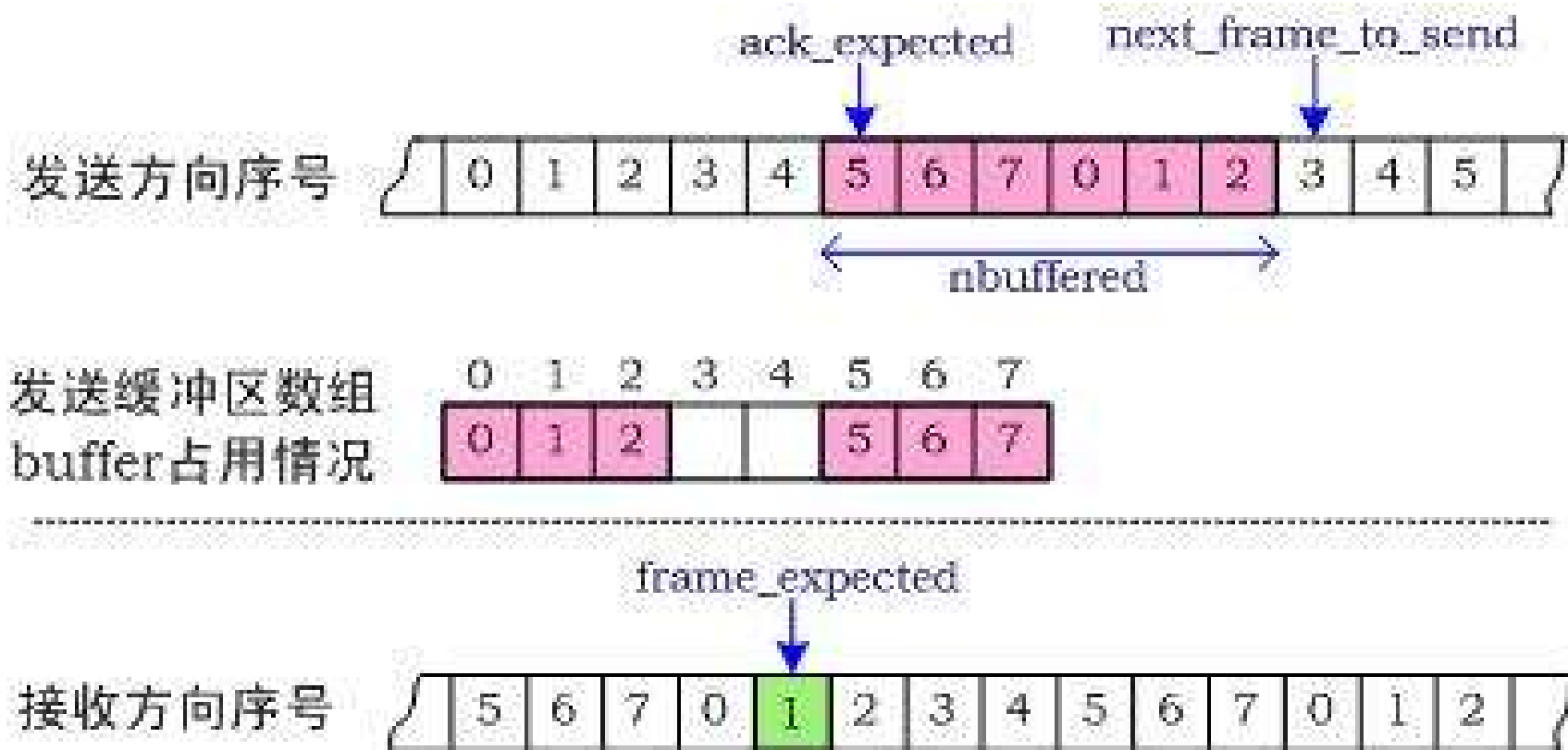


b. Window size $= 2^m$

[Forouzan]

协议5: Go Back N (回退N步) 协议

3位序号, 最大发送窗口 = 7



/* Protocol 5 (Go-back-n) allows multiple outstanding frames. The sender may transmit up to MAX_SEQ frames without waiting for an ack. In addition, unlike in the previous protocols, the network layer is not assumed to have a new packet all the time. Instead, the network layer causes a network_layer_ready event when there is a packet to send. */

```
#define MAX_SEQ 7
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready} event_type;
#include "protocol.h"
```

```
static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
    /* Return true if a <= b < c circularly; false otherwise. */
    if (((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a)))
        return(true);
    else
        return(false);
}
```

```
static void send_data(seq_nr frame_nr, seq_nr frame_expected, packet buffer[])
{
    /* Construct and send a data frame. */
    frame s;                                /* scratch variable */

    s.info = buffer[frame_nr];               /* insert packet into frame */
    s.seq = frame_nr;                         /* insert sequence number into frame */
    s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1); /* piggyback ack */
    to_physical_layer(&s);                   /* transmit the frame */
    start_timer(frame_nr);                   /* start the timer running */
}
```

2020春

计算机网络, 数据链路层

窗口的上边界+1

窗口下边界

```
void protocol5(void)
{
    seq_nr next_frame_to_send;
    seq_nr ack_expected;
    seq_nr frame_expected;
    frame r;
    packet buffer[MAX_SEQ + 1];
    seq_nr nbuffered;
    seq_nr i;
    event_type event;

    enable_network_layer();
    ack_expected = 0;
    next_frame_to_send = 0;
    frame_expected = 0;
    nbuffered = 0;

    /* MAX_SEQ > 1; used for outbound stream */
    /* oldest frame as yet unacknowledged */
    /* next frame expected on inbound stream */
    /* scratch variable */
    /* buffers for the outbound stream */
    /* # output buffers currently in use */
    /* used to index into the buffer array */

    /* allow network_layer_ready events */
    /* next ack expected inbound */
    /* next frame going out */
    /* number of frame expected inbound */
    /* initially no packets are buffered */
}
```

```

while (true) {
    wait_for_event(&event);          /* four possibilities: see event_type above */

    switch(event) {
        case network_layer_ready:    /* the network layer has a packet to send */
            /* Accept, save, and transmit a new frame. */
            from_network_layer(&buffer[next_frame_to_send]); /* fetch new packet */
            nbuffered = nbuffered + 1; /* expand the sender's window */
            send_data(next_frame_to_send, frame_expected, buffer); /* transmit the frame */
            inc(next_frame_to_send); /* advance sender's upper window edge */
            break;

        case frame_arrival:          /* a data or control frame has arrived */
            from_physical_layer(&r); /* get incoming frame from physical layer */

            if (r.seq == frame_expected) {
                /* Frames are accepted only in order. */
                to_network_layer(&r.info); /* pass packet to network layer */
                inc(frame_expected); /* advance lower edge of receiver's window */
            }
    }
}

```



```

/* Ack n implies n - 1, n - 2, etc. Check for this. */
while (between(ack_expected, r.ack, next_frame_to_send)) {
    /* Handle piggybacked ack. */
    nbuffered = nbuffered - 1; /* one frame fewer buffered */
    stop_timer(ack_expected); /* frame arrived intact; stop timer */
    inc(ack_expected); /* contract sender's window */
}
break;

```

```

case cksum_err: break; /* just ignore bad frames */

```

```

case timeout: /* trouble; retransmit all outstanding frames */
    next_frame_to_send = ack_expected; /* start retransmitting here */
    for (i = 1; i <= nbuffered; i++) {
        send_data(next_frame_to_send, frame_expected, buffer); /* resend 1 frame */
        inc(next_frame_to_send); /* prepare to send the next one */
    }

```

```

}

```

```

if (nbuffered < MAX_SEQ)
    enable_network_layer();
else
    disable_network_layer();

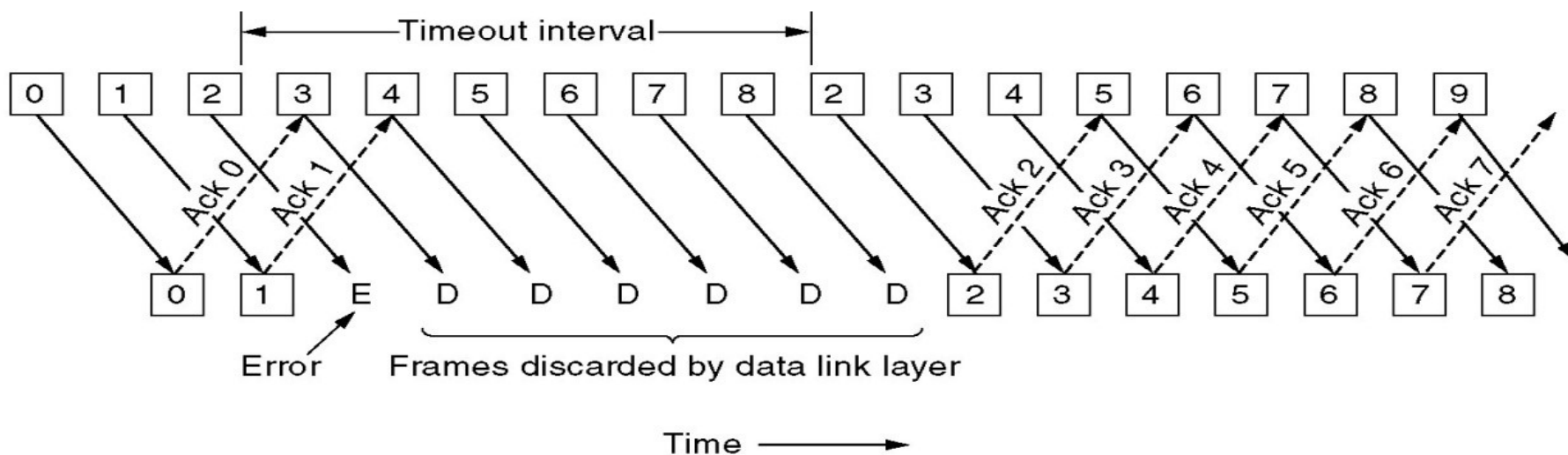
```

```

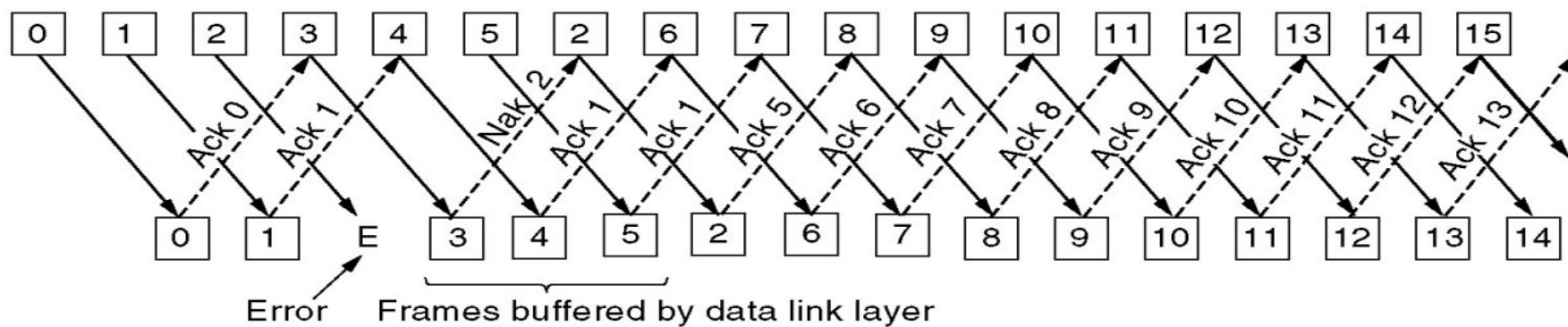
}
}

```

Go Back N的问题



接收窗口 = 1



接收窗口 = 4.

基本的流量控制协议

■ 初级协议

- ◆ 乌托邦单工协议
- ◆ 无差错信道上的单工停止等待协议
- ◆ 有噪声信道上的单工协议

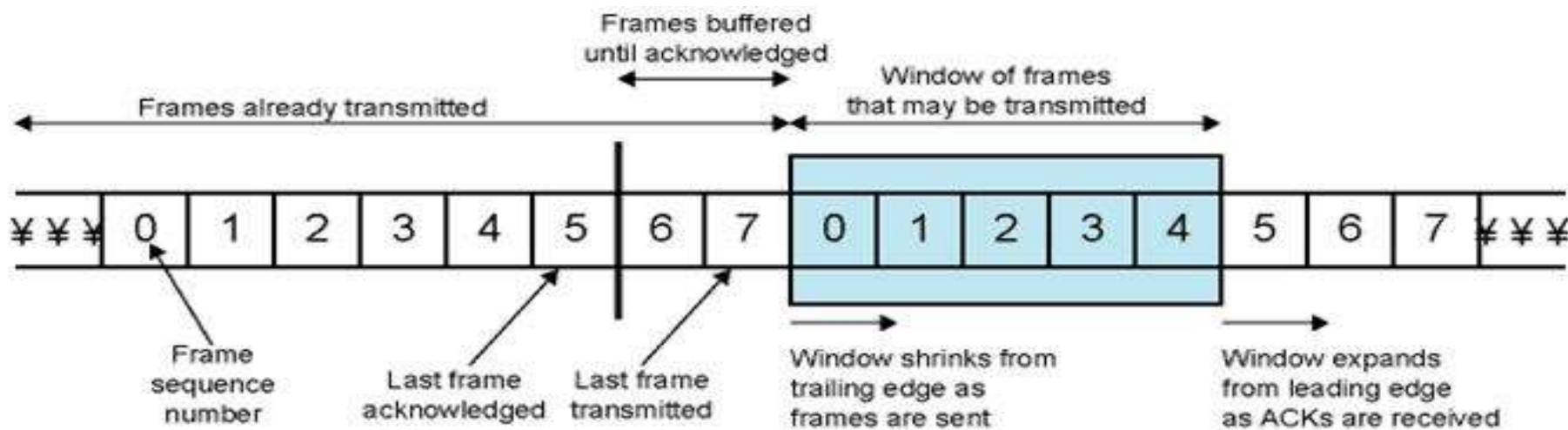
■ 滑动窗口协议

- ◆ 一比特滑动窗口协议
- ◆ Go Back N（回退N步）协议
- ◆ 选择重传协议

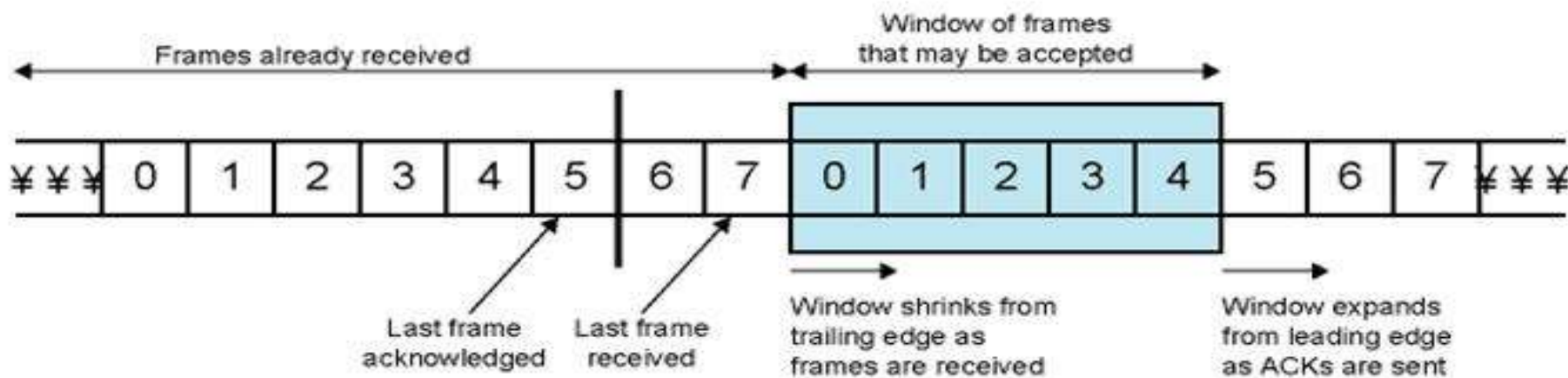
选择重传ARQ协议

- 也称为 选择拒绝协议
- 只有被拒绝的帧重传
- 后续的无错帧将被接收方接收并缓存
- 减少重传
- 接收方必须维护较大的缓存
- 更复杂

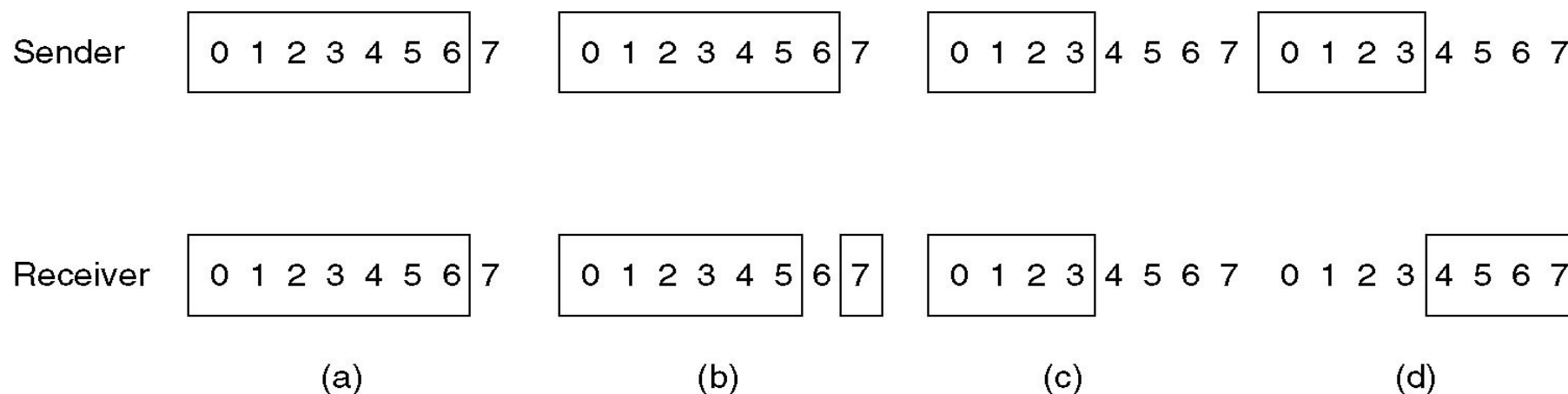
滑动窗口：概貌



(a) Sender's perspective



选择重传的滑动窗口大小



(a) 窗口大小为7，初始状态

(b) 7帧已经发出、收到，但没有确认

(c) 窗口大小为4，初始状态

(d) 4帧已经发出、收到，但没有确认

选择重传协议的最大窗口大小

■ 最大窗口大小不应超过序号范围的一半

◆ $W_T + W_R \leq 2^n$

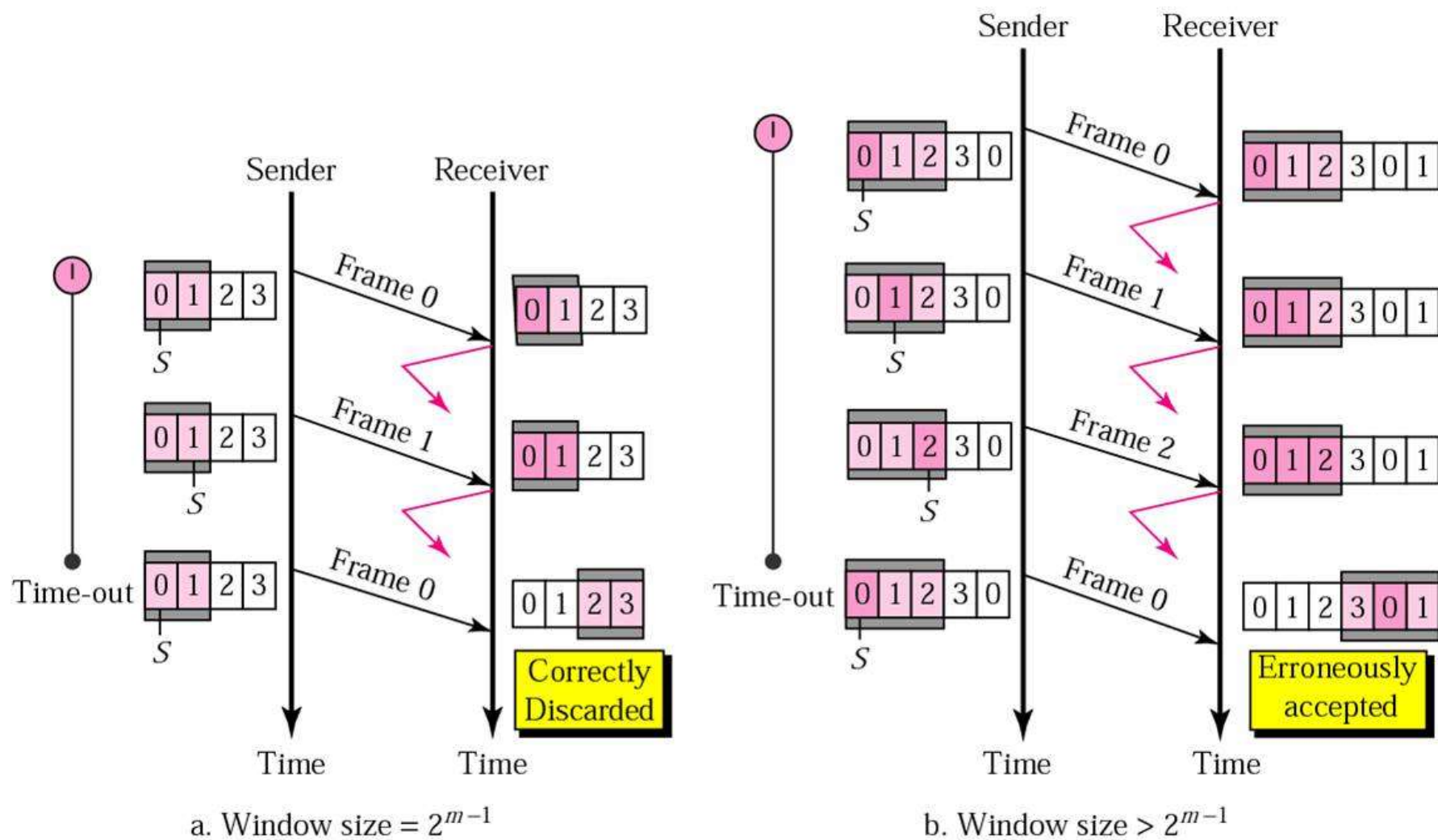
◆ $W_T \geq W_R$

◆ 在实现中，一般 $W_T = W_R = 2^{n-1}$

■ 接收方需要有多少缓存？

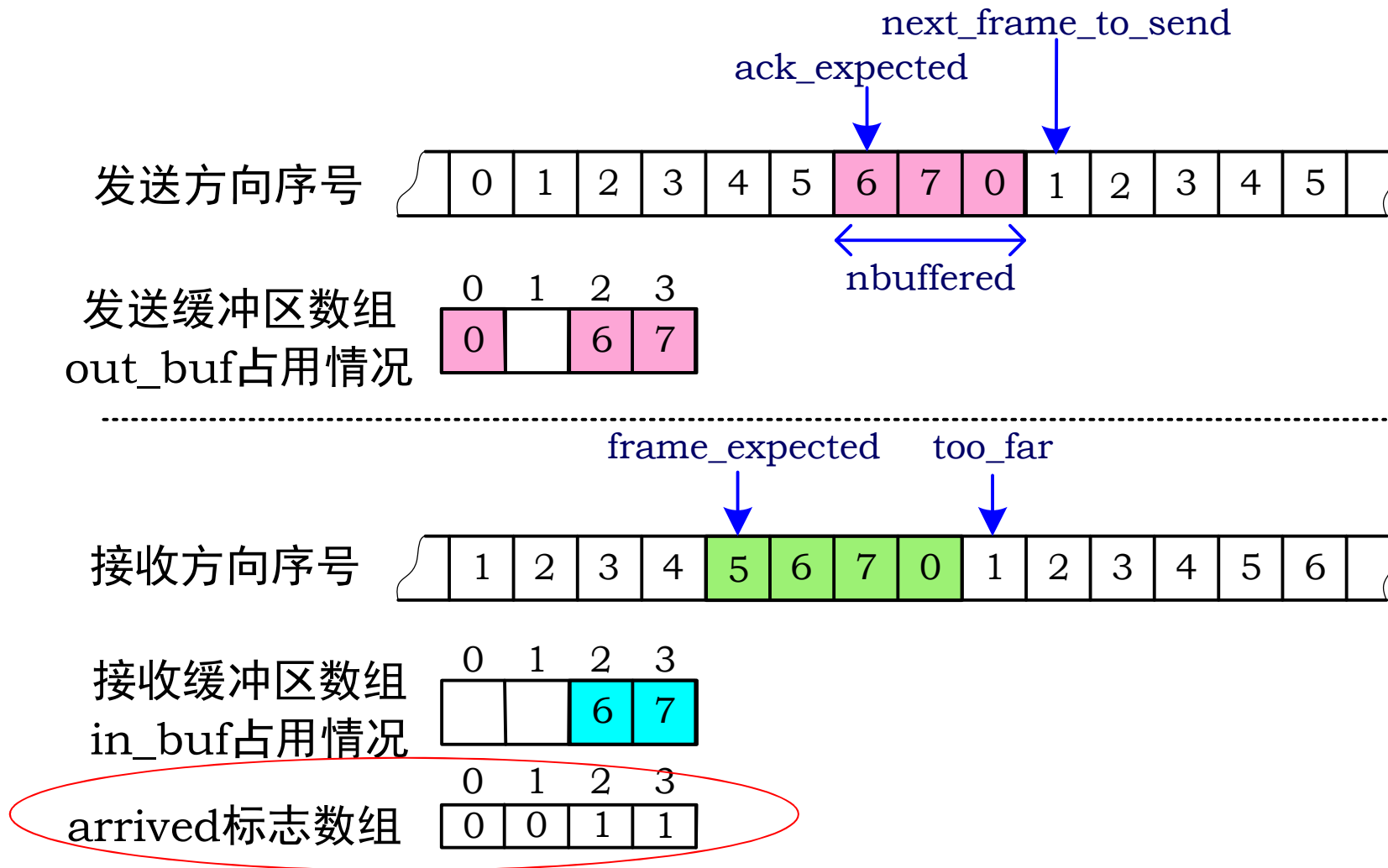
◆ 缓存数量=窗口大小，不等于序号范围

最大窗口大小，2位序号示例($m=2$)



协议6：选择重传

发送窗口最大为4，接收窗口最大为4



对Protocol 5的改进和效率

■ 设置ACK定时器

- ◆ 在没有数据要发送时，发送ACK帧
- ◆ 用于回送ACK的定时器操作
 - start_ack_timer和stop_ack_timer,
- ◆ 数据重传所用的定时器操作
 - start_timer与stop_timer
- ◆ ACK定时器时限的设计

■ NAK帧

■ 协议参数与线路利用率

- ◆ 超时定时器时限的设计（考虑线路往返时间延迟，ACK定时器时限，对方物理层发送排队时延）
- ◆ 滑动窗口，即MAX_SEQ大小的设置

ACK定时器

■ ACK定时器

- ◆ 序号在接收窗口内的帧到达后, *start_ack_timer* 将启动一个辅助定时器。在该定时器超时、还没有反向数据帧时, 发送一个单独的**ACK**帧
- ◆ 辅助定时器的超时时间比数据帧的超时时间要短

NAK

- 当接收方有理由怀疑发生差错时，就发送**NAK**帧给发送方
 - ◆ **NAK**帧用于请求重传
- 在下列两种情况下，接收方应怀疑出错
 - ◆ 收到一个被破坏的帧
 - ◆ 收到一个不是所期待的序号的帧
- 避免对同一帧的多次重发请求
 - ◆ 接收方记录是否对某个帧发送过**NAK**

/* Protocol 6 (Selective repeat) accepts frames out of order but passes packets to the network layer in order. Associated with each outstanding frame is a timer. When the timer expires, only that frame is retransmitted, not all the outstanding frames, as in protocol 5. */

```
#define MAX_SEQ 7                                /* should be  $2^n - 1$  */
#define NR_BUFS ((MAX_SEQ + 1)/2)
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready, ack_timeout} event_type;
#include "protocol.h"
boolean no_nak = true;                          /* no nak has been sent yet */
seq_nr oldest_frame = MAX_SEQ + 1;              /* initial value is only for the simulator */

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
    /* Same as between in protocol 5, but shorter and more obscure. */
    return ((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a));
}

static void send_frame(frame_kind fk, seq_nr frame_nr, seq_nr frame_expected, packet buffer[])
{
    /* Construct and send a data, ack, or nak frame. */
    frame s;                                     /* scratch variable */

    s.kind = fk;                                /* kind == data, ack, or nak */
    if (fk == data) s.info = buffer[frame_nr % NR_BUFS];
    s.seq = frame_nr;                           /* only meaningful for data frames */
    s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);
    if (fk == nak) no_nak = false;              /* one nak per frame, please */
    to_physical_layer(&s);                      /* transmit the frame */
    if (fk == data) start_timer(frame_nr % NR_BUFS);
    stop_ack_timer();                          /* no need for separate ack frame */
}
```

```

void protocol6(void)
{
    seq_nr ack_expected;           /* lower edge of sender's window */
    seq_nr next_frame_to_send;     /* upper edge of sender's window + 1 */
    seq_nr frame_expected;         /* lower edge of receiver's window */
    seq_nr too_far;                /* upper edge of receiver's window + 1 */
    int i;                         /* index into buffer pool */
    frame r;                       /* scratch variable */
    packet out_buf[NR_BUFS];       /* buffers for the outbound stream */
    packet in_buf[NR_BUFS];        /* buffers for the inbound stream */
    boolean arrived[NR_BUFS];      /* inbound bit map */
    seq_nr nbuffered;              /* how many output buffers currently used */
    event_type event;

    enable_network_layer();         /* initialize */
    ack_expected = 0;               /* next ack expected on the inbound stream */
    next_frame_to_send = 0;         /* number of next outgoing frame */
    frame_expected = 0;
    too_far = NR_BUFS;
    nbuffered = 0;                  /* initially no packets are buffered */
    for (i = 0; i < NR_BUFS; i++) arrived[i] = false;

```



```

while (true) {
    wait_for_event(&event);                /* five possibilities: see event_type above */
    switch(event) {
        case network_layer_ready:          /* accept, save, and transmit a new frame */
            nbuffered = nbuffered + 1;      /* expand the window */
            from_network_layer(&out_buf[next_frame_to_send % NR_BUFS]); /* fetch new packet */
            send_frame(data, next_frame_to_send, frame_expected, out_buf); /* transmit the frame */
            inc(next_frame_to_send);        /* advance upper window edge */
            break;

        case frame_arrival:                /* a data or control frame has arrived */
            from_physical_layer(&r);         /* fetch incoming frame from physical layer */
            if (r.kind == data) {
                /* An undamaged frame has arrived. */
                if ((r.seq != frame_expected) && no_nak)
                    send_frame(nak, 0, frame_expected, out_buf); else start_ack_timer();
                if (between(frame_expected, r.seq, too_far) && (arrived[r.seq % NR_BUFS] == false)) {
                    /* Frames may be accepted in any order. */
                    arrived[r.seq % NR_BUFS] = true;    /* mark buffer as full */
                    in_buf[r.seq % NR_BUFS] = r.info;  /* insert data into buffer */
                    while (arrived[frame_expected % NR_BUFS]) {
                        /* Pass frames and advance window. */
                        to_network_layer(&in_buf[frame_expected % NR_BUFS]);
                        no_nak = true;
                        arrived[frame_expected % NR_BUFS] = false;
                        inc(frame_expected); /* advance lower edge of receiver's window */
                        inc(too_far);       /* advance upper edge of receiver's window */
                        start_ack_timer();   /* to see if a separate ack is needed */
                    }
                }
            }
    }
}

```

Continued →

```

if((r.kind==nak) && between(ack_expected,(r.ack+1)%(MAX_SEQ+1),next_frame_to_send))
    send_frame(data, (r.ack+1) % (MAX_SEQ + 1), frame_expected, out_buf);

while (between(ack_expected, r.ack, next_frame_to_send)) {
    nbuffered = nbuffered - 1;          /* handle piggybacked ack */
    stop_timer(ack_expected % NR_BUFS); /* frame arrived intact */
    inc(ack_expected);                  /* advance lower edge of sender's window */
}
break;

case cksum_err:
    if (no_nak) send_frame(nak, 0, frame_expected, out_buf); /* damaged frame */
    break;

case timeout:
    send_frame(data, oldest_frame, frame_expected, out_buf); /* we timed out */
    break;

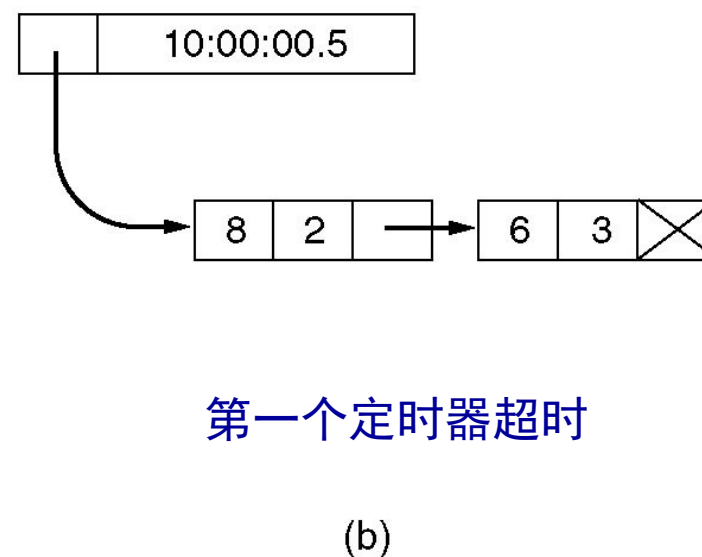
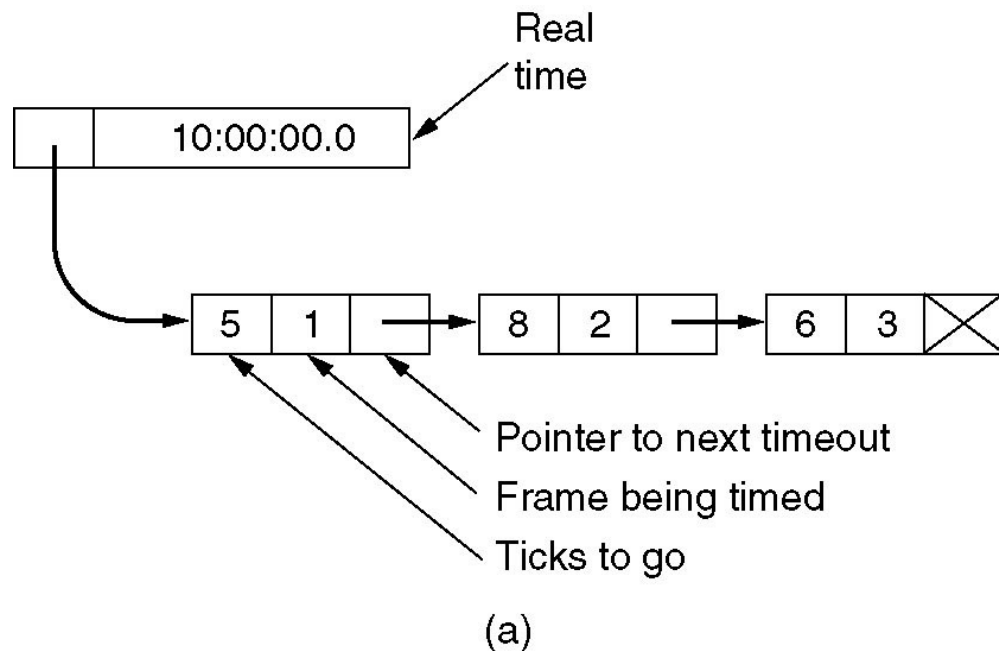
case ack_timeout:
    send_frame(ack,0,frame_expected, out_buf); /* ack timer expired; send ack */
}

if (nbuffered < NR_BUFS) enable_network_layer(); else disable_network_layer();
}
}

```


软件实现定时器队列

软件仿真多个定时器



- 时钟每100毫秒跳一次
- 3个超时分配发生在 10:00:00.5、10:00:01.3 和 10:00:01.9

流量控制协议小结

- 协议1: 理想信道、无需流量控制
- 协议2: 无差错信道、停止等待
 - ◆ 接收方发ACK
- 协议3: 无传输差错但可能丢失
 - ◆ 使用定时器和1位序号
- 协议4: 1比特滑动窗口 (停止等待协议)
 - ◆ 全双工、捎带确认
- 协议5: Go Back N滑动窗口协议
 - ◆ 发送窗口>1、接收窗口=1
 - ◆ 对于n位序号, $W_T \leq 2^n - 1$
- 协议6: 选择重传滑动窗口协议
 - ◆ 发送窗口>1、接收窗口>1
 - ◆ 对于n位序号, 通常 $W_T = W_R = 2^{n-1}$
 - ◆ ACK定时器 和 NAK

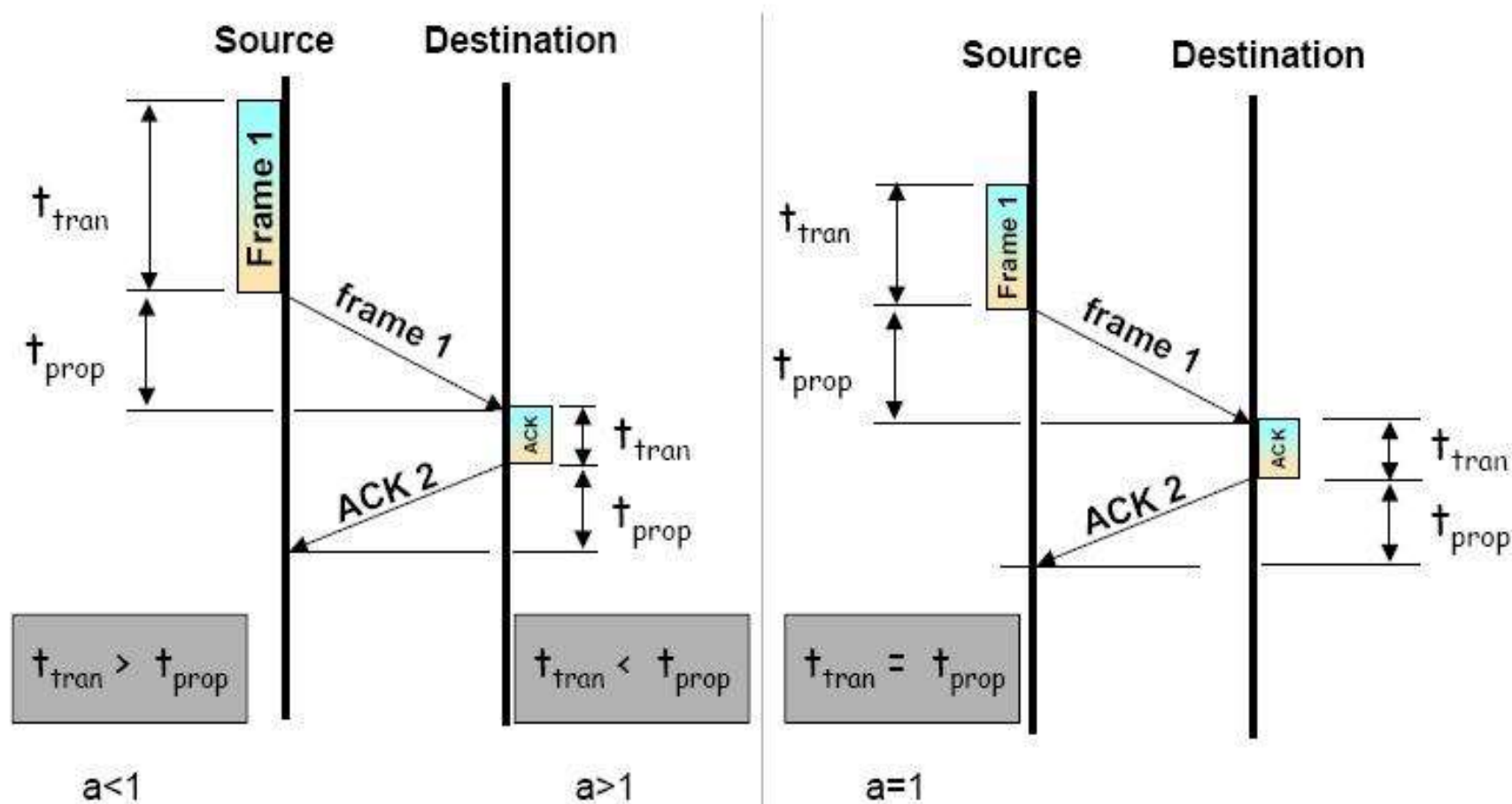
总结

	乌托邦协议	无错信道停等协议	停等协议	1比特滑动窗口协议	GoBackN	选择重传
信道条件	理想信道					
协议功能		流量控制	差错控制、流量控制	差错控制、流量控制	差错控制、流量控制	
通信模式	单工	单工	单工	双工	双工	双工
确认方式						
序号	无	无				
窗口尺寸						
信道效率						
帧种类						
典型协议						
适用场景						

主要内容

- 数据链路层的位置、功能及服务
- 相关问题的设计
 - ◆ 成帧
 - ◆ 差错控制
 - ◆ 流量控制
 - 基本流量控制协议
 - 滑动窗口协议的性能
- 协议示例
 - ◆ HDLC
 - ◆ PPP

帧传输模型



传输时间和 环回时间

■ 传输时间(t_{xfr})

- ◆ 接收方收到整个帧用的时间

$$t_{xfr} = t_{tran} + t_{prop} + t_{queue/switching}$$

$$t_{tran} = \frac{\text{number of frame bits [bits]}}{\text{data rate [bps]}}$$

$$t_{prop} = \frac{\text{link length [m]}}{v_{prop} [m/s]}$$

■ 环回时间(RTT)

- ◆ 如果数据帧和**ACK**的传输路径不同，**RTT**可能不是单程时延的两倍

无差错停止等待协议的性能(1)

- 成功传输一个数据帧的时间为 T_F

$$T_F = t_{\text{tran}} + t_{\text{prop}} + t_{\text{proc}} + t_{\text{ack}} + t_{\text{prop}} + t_{\text{proc}}$$

- ◆ T_F : 从开始发送到收到**ACK**的时间
- ◆ t_{tran} : 发送一个数据帧的时间
- ◆ t_{prop} : 从发送方到接收方的传播时间
- ◆ t_{proc} : 处理时间
- ◆ t_{ack} : **ACK**的发送时间

无差错停止等待协议的性能(2)

- 假定处理时间 t_{proc} 和ACK发送时间 t_{ack} 忽略不计,

$$T_F = t_{tran} + 2t_{prop}$$

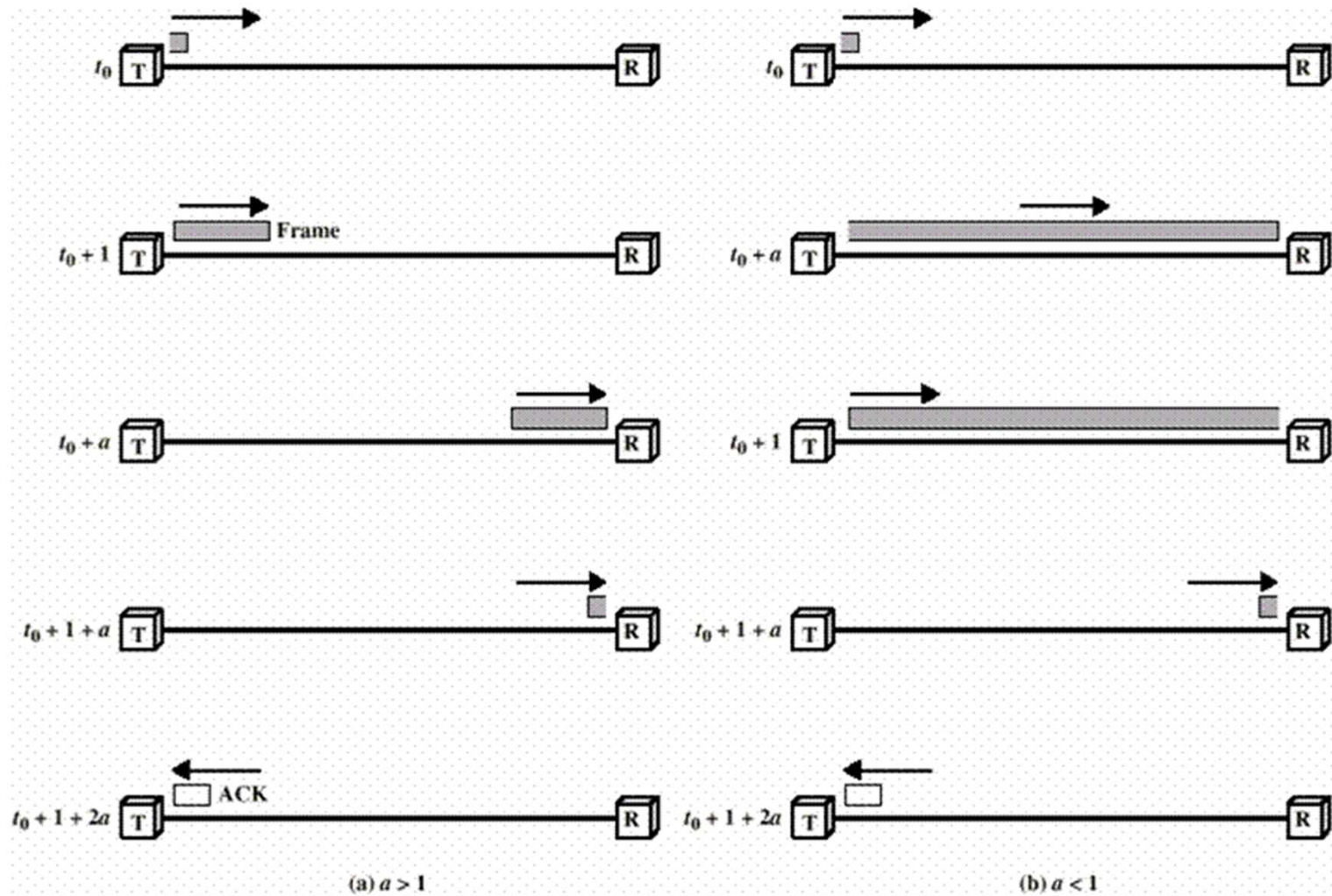
- 信道利用率

$$U = \frac{t_{tran}}{2t_{prop} + t_{tran}} = \frac{1}{1 + 2a}$$

$$\text{where } a = \frac{\text{propagation time}}{\text{transmission time}}$$

停止等待协议：性能

将发送时间归一化为 **1**，则传播时延是 **a**



停止等待协议的性能计算示例

■ 停止等待：无差错

□ Satellite Link: Propagation Delay $t_{\text{prop}} = 270 \text{ ms}$

Frame Size = 4000 bits = 500 bytes

Data rate = 56 kbps $\Rightarrow t_{\text{tran}} = 4/56 = 71 \text{ ms}$

$\alpha = t_{\text{prop}}/t_{\text{tran}} = 270/71 = 3.8$

$U = 1/(2\alpha+1) = 0.12$

□ Short Link: 1 km = 5 μs ,

Rate=10 Mbps,

Frame=500 bytes $\Rightarrow t_{\text{tran}} = 4\text{k}/10\text{M} = 400 \mu\text{s}$

$\alpha = t_{\text{prop}}/t_{\text{tran}} = 5/400 = 0.012 \Rightarrow U = 1/(2\alpha+1) = 0.98$

停止等待：线路利用率

- 信道容量: b bps
- 帧长: l 比特
- 环回时延: R 秒

■ 线路利用率=

$$\frac{\frac{l}{b}}{\frac{l}{b} + R} = \frac{l}{l + bR}$$

◆ 如果 $l < bR$, 利用率低于 50%

停等协议的性能：有差错情形

有传输差错

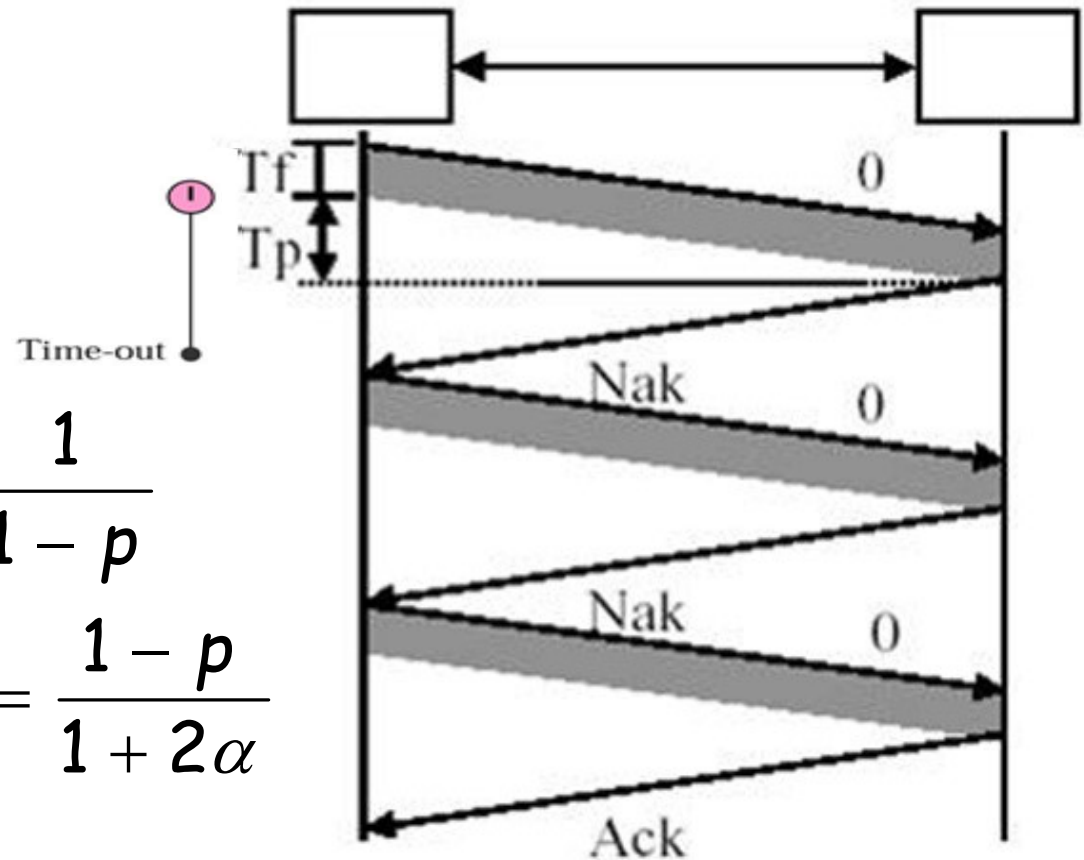
◆ p = 一帧出错的概率

◆ N_r = 帧传输的次数

$$\alpha = \frac{T_{prop}}{T_{tran}}$$

$$N_r = \sum_{i=1}^{\infty} ip^{i-1}(1-p) = \frac{1}{1-p}$$

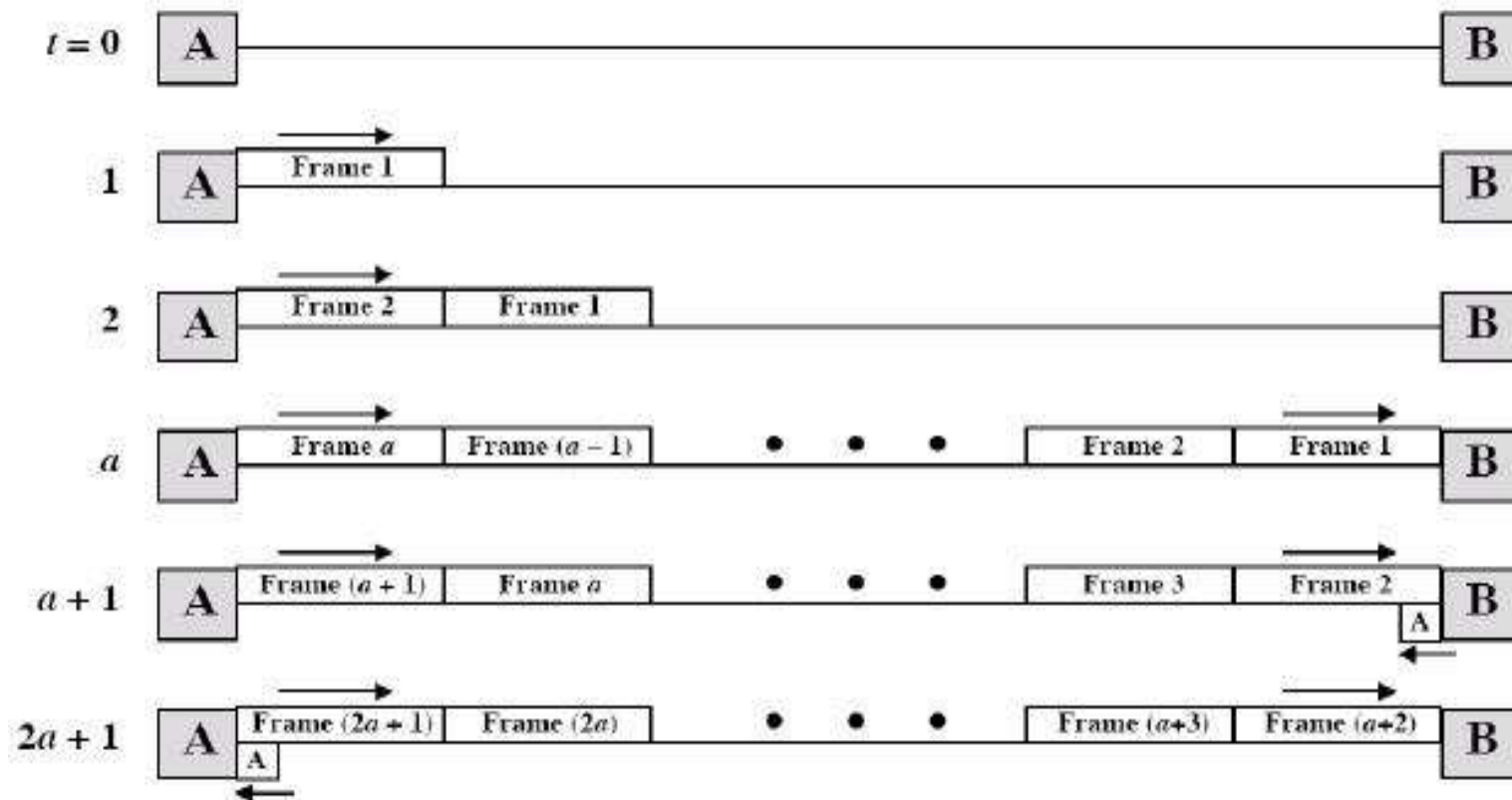
$$U = \frac{T_{tran}}{N_r(T_{tran} + 2T_{prop})} = \frac{1-p}{1+2\alpha}$$



滑动窗口协议的性能：无差错情形 (1)

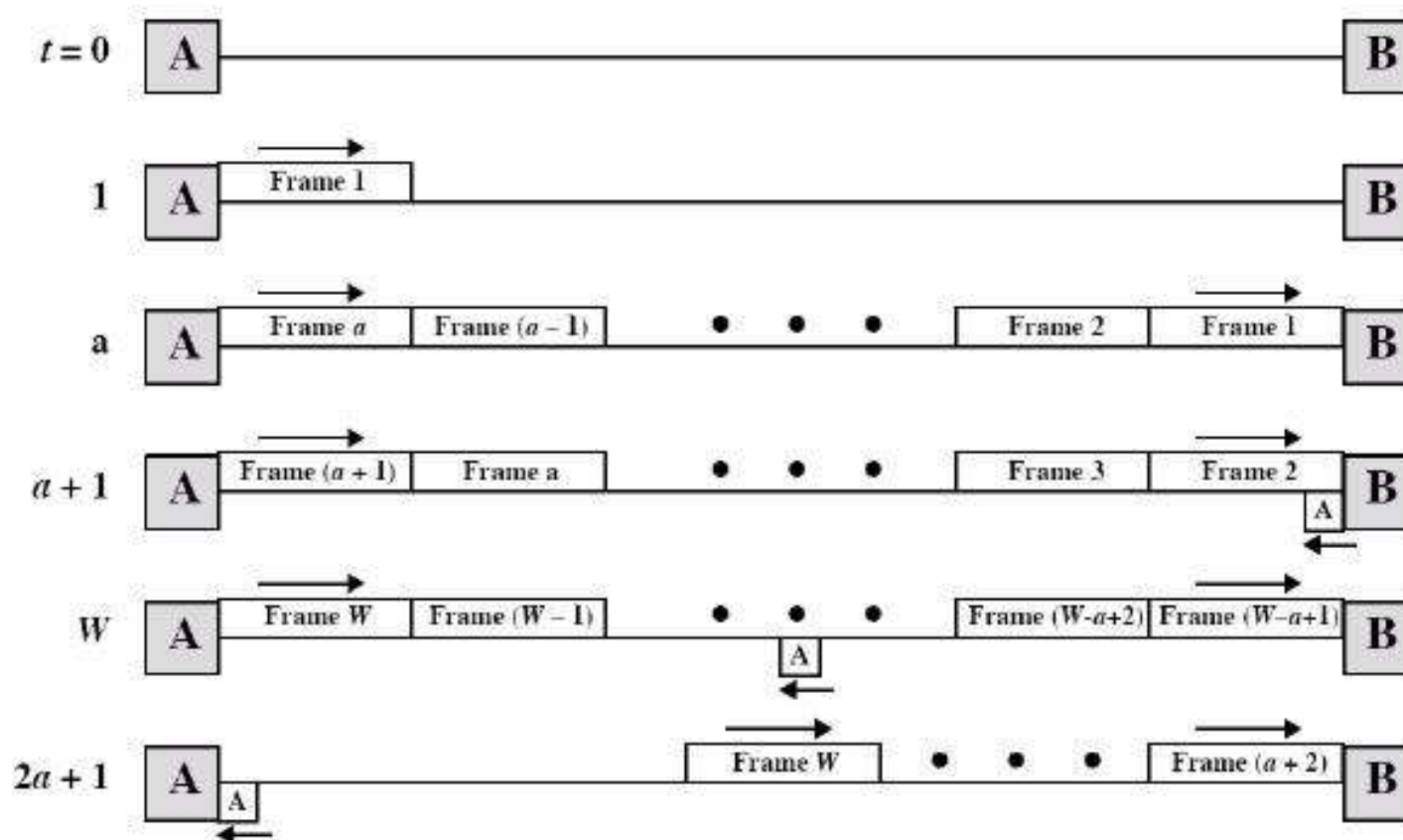
■ 将发送时间归一化为 1，则传播时延是 α

◆ 情形1: $W \geq 2\alpha + 1$ ，其中 W 是发送窗口大小



滑动窗口协议的性能：无差错情形 (2)

情形2: $W < 2a + 1$



滑动窗口协议的性能：无差错情形 (3)

- 因此信道利用率为

$$U = \begin{cases} 1 & W \geq 2a+1 \\ \frac{W}{2a+1} & W < 2a+1 \end{cases}$$

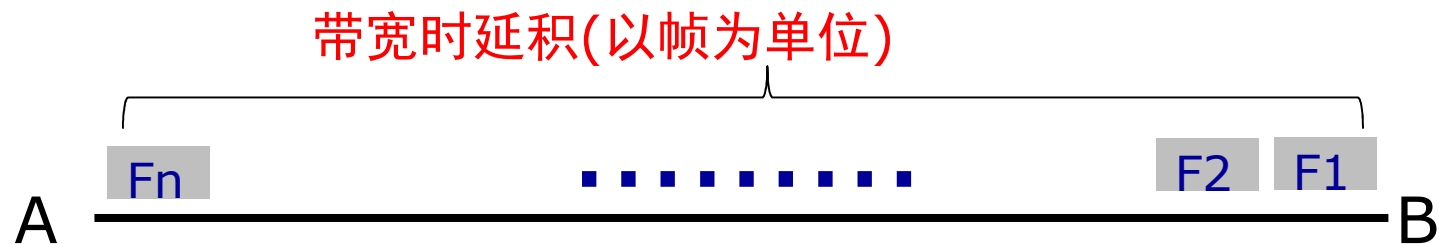
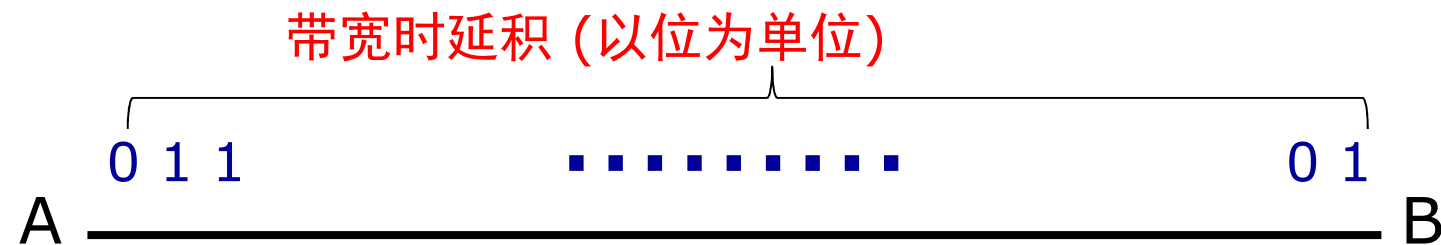
- 教材上的另一个公式

$$\text{link utilization} \leq \frac{w}{1 + 2BD}$$

其实是一样的

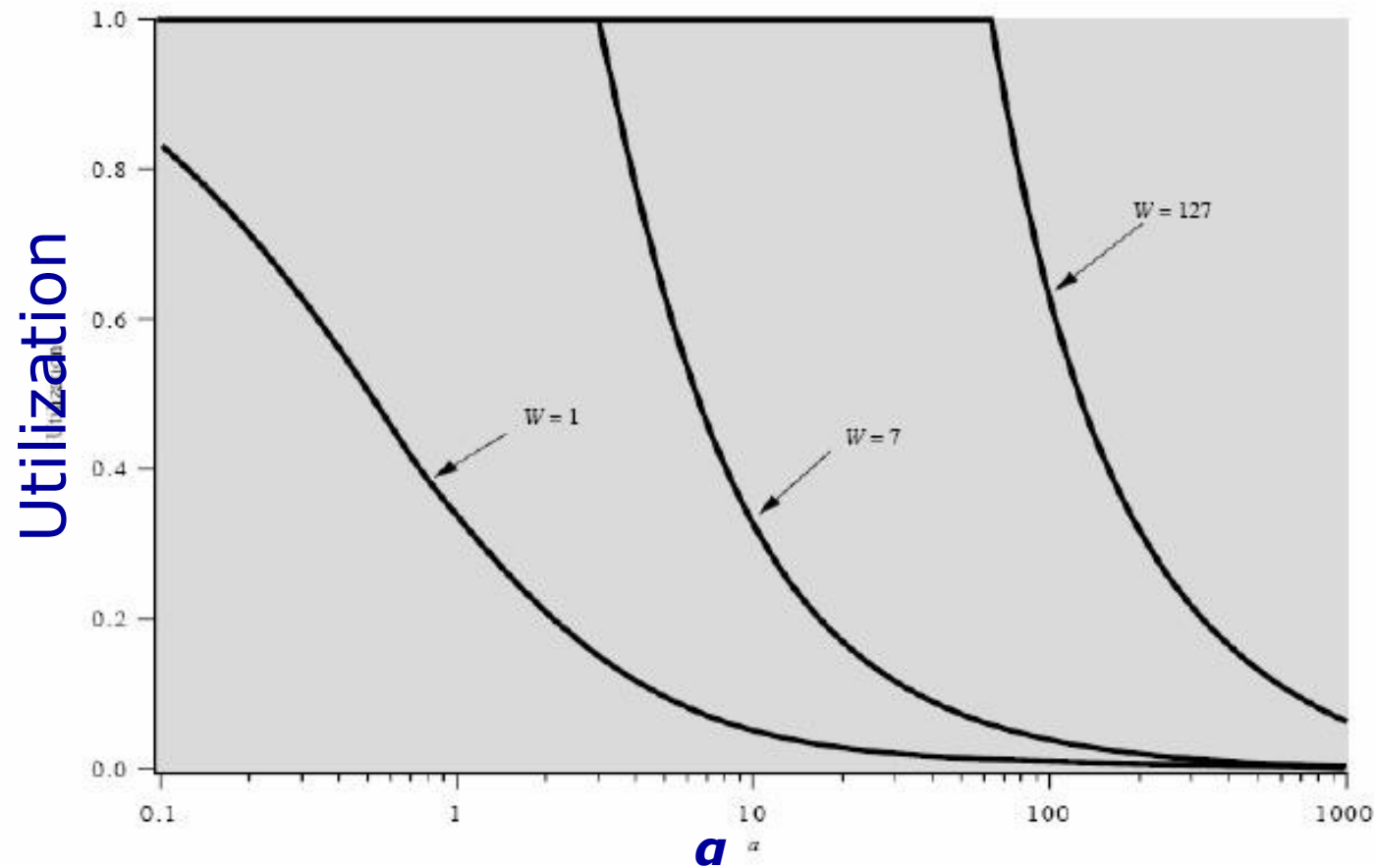
带宽时延积

- 带宽：最大数据率 (bps)
- 时延：传播时延
- 带宽时延积：在单程传播时延的时间内，信道可以容纳多少位数据



滑动窗口协议的性能：无差错情形 (4)

- 滑动窗口的信道利用率是 α 的函数



无差错情况下的滑动窗口协议的性能: Piggybacking

- 捎带确认/搭载ACK
- 必须加上反向数据帧的发送时延

◆ $U = W / (2a + 2) = W / 2(a + 1)$

实用的流量控制协议总结

实用的协议

◆ 协议4: 1比特滑动窗口

➤ $W_T = W_R = 1$

◆ 协议5, Go Back N滑动窗口

➤ 发送窗口 >1 , 接收窗口 $=1$

➤ 对 n -bit序号, $W_T \leq 2^n - 1$

◆ 协议6, 选择重传滑动窗口

➤ 发送窗口 >1 , 接收窗口 >1

➤ 对 n -bit序号, $W_T = W_R \leq 2^n - 1$

协议性能

◆ 停止等待协议

➤ 无差错信道: $U = 1/(1+2a)$

➤ 有差错信道: $U = (1-p)/(1+2a)$

◆ Go-back N 和选择重传协议、无差错捎带确认: $U = W/2(a+1)$

$$U = \begin{cases} 1 & W \geq 2a+1 \\ \frac{W}{2a+1} & W < 2a+1 \end{cases}$$

主要内容

- 数据链路层的位置、功能及服务
- 相关问题的设计
 - ◆ 成帧
 - ◆ 差错控制
 - ◆ 流量控制
- 协议示例
 - ◆ HDLC
 - ◆ PPP

数据链路的类型

■ 点到点链路

- ◆ 一个发送方、一个接收方
- ◆ 地址不是必需的
- ◆ 可靠通信
- ◆ 协议示例：HDLC, PPP...

■ 广播链路

- ◆ 多个发送方、潜在多个接收方
- ◆ 访问共享介质
- ◆ 地址是必需的
- ◆ 协议示例：CSMA/CD、CSMA/CA...

点到点的数据链路协议示例

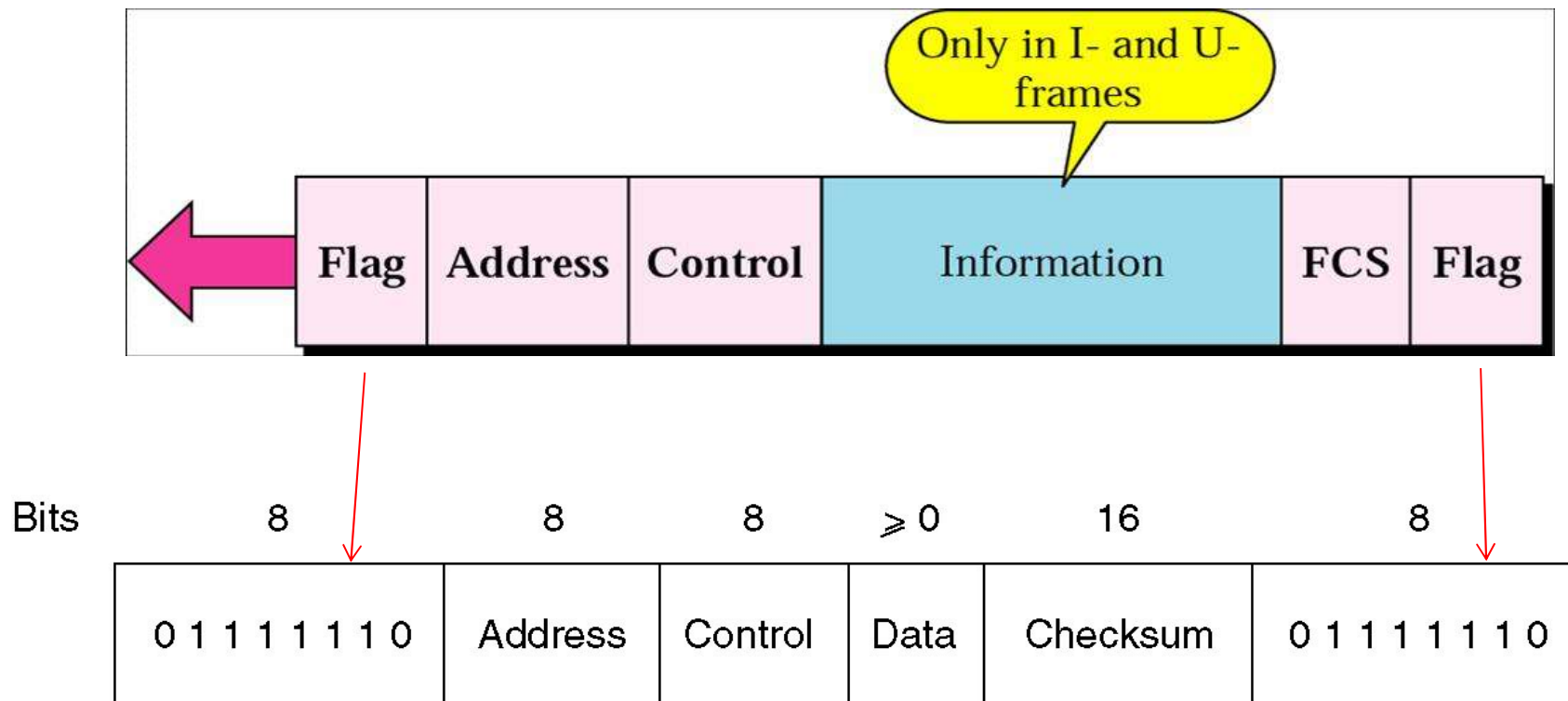
- 高级数据链路控制HDLC
- 因特网的数据链路层
 - ◆ 串行线IP协议 SLIP
 - ◆ 点对点协议PPP

HDLC: 历史

- IBM SDLC
 - ◆ 同步数据链路控制
- ANSI ADCCP
 - ◆ 先进的数据通信控制规程
- ISO HDLC
 - ◆ 高级数据链路控制
- CCITT(ITU-T) LAPB
 - ◆ 链路接入规程 LAP
 - ◆ 链路接入规程平衡模式 LAPB
- LAPD (ISDN D信道协议)
- LAN LLC子层协议

HDLC: 帧结构

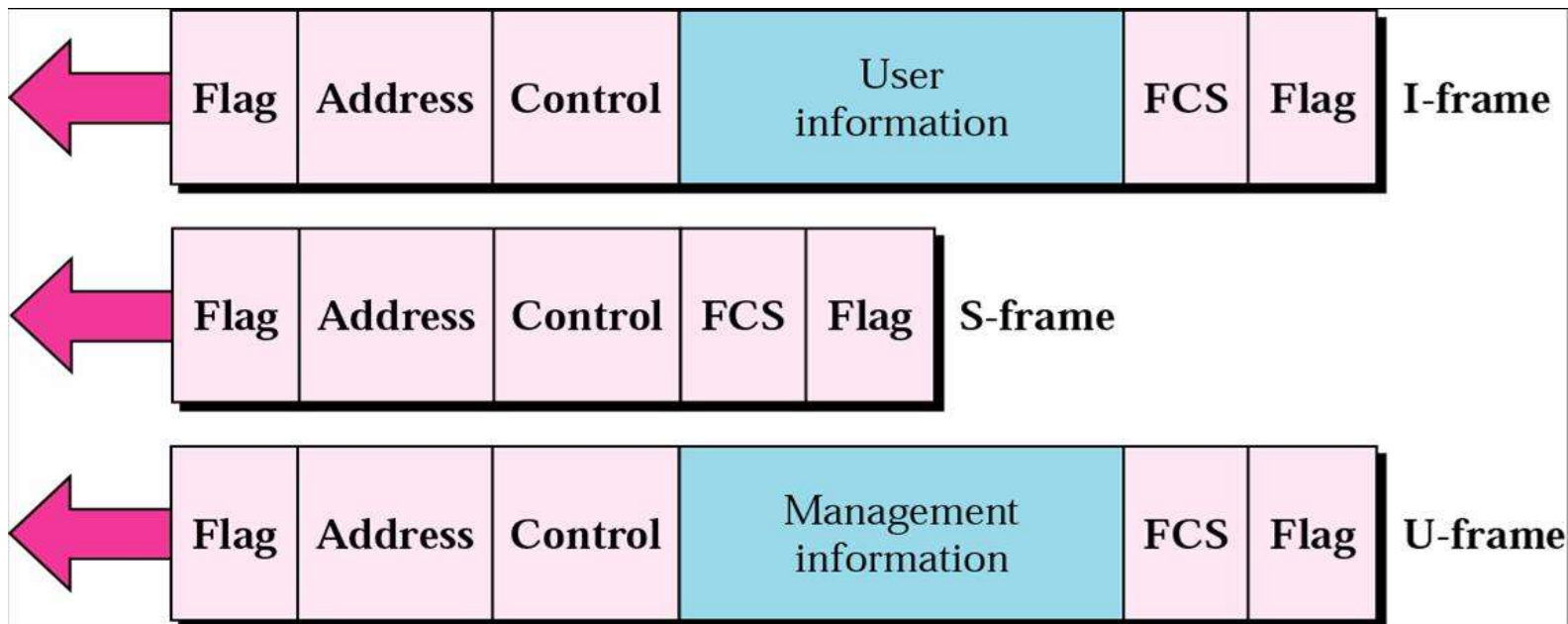
- 同步传输
- 面向比特、零比特填充实现透明传输



[Forouzan]

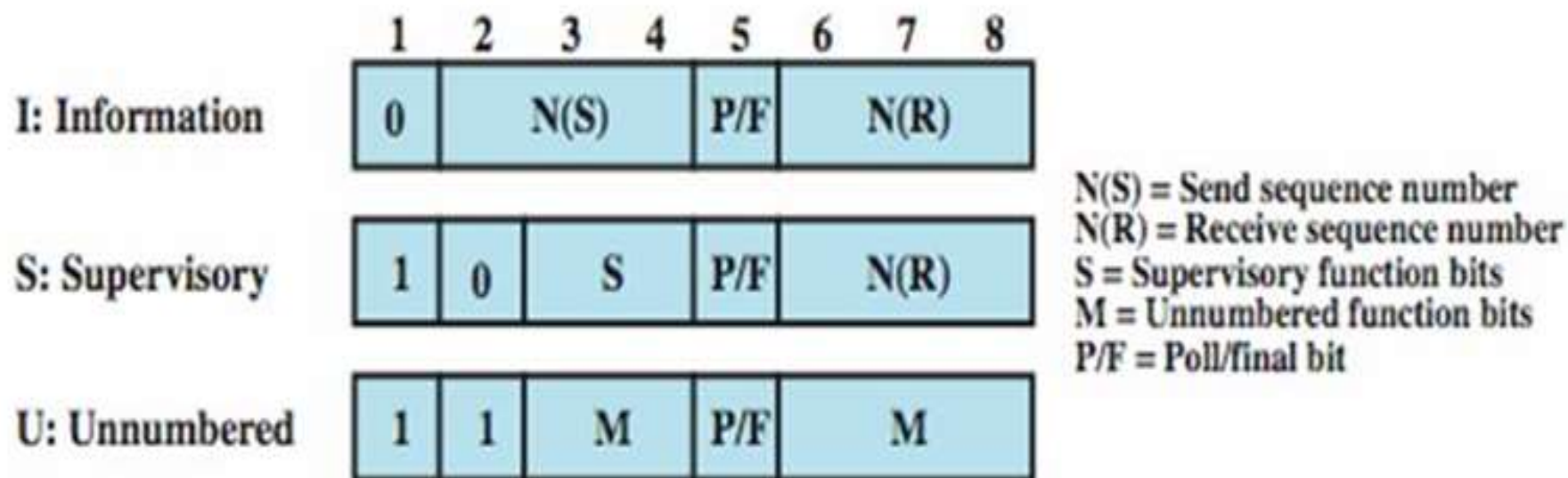
HDLC：帧类型

- 信息帧：包含上层数据
 - ◆ 捎带确认、包含流量控制和差错控制信息
- 监督帧：**ACK** 或 **NAK**
- 无编号帧：数据链路层的管理和链路控制



[Forouzan]

HDLC: 控制字段



点到点的数据链路协议示例

■ HDLC

- ◆ 帧首尾标志：“01111110”，零比特填充
- ◆ CRC校验
- ◆ 流量控制和差错控制：
 - GoBackN ARQ和选择重传ARQ

■ 因特网的数据链路层

- ◆ 串行线IP协议 SLIP
- ◆ 点对点协议PPP

SLIP —— 串行线IP

- 用于连接远程服务器
- RFC1055
- 成帧：字节填充
 - 首尾标志：0xC0
 - 使用0xDB 0xDC 替代帧中数据0xC0
 - 使用 0xDB 0xDB 替代帧中数据0xDB
- 问题
 - 没有差错控制
 - 只支持IP
 - 不能动态分配IP地址
 - 没有身份认证

点到点的数据链路协议示例

- HDLC
- 因特网的数据链路层
 - ◆ 串行线IP协议 SLIP
 - ◆ 点对点协议PPP

PPP vs. HDLC

- 点对点链路 **vs.** 多种配置（点对点、一对多）
- 面向字节 **vs.** 面向比特
 - ◆ 字节填充 **vs** 比特填充
- 无连接不确认的服务 **vs.** 可靠的面向连接的服务

与HDLC相比, PPP

- 更简单

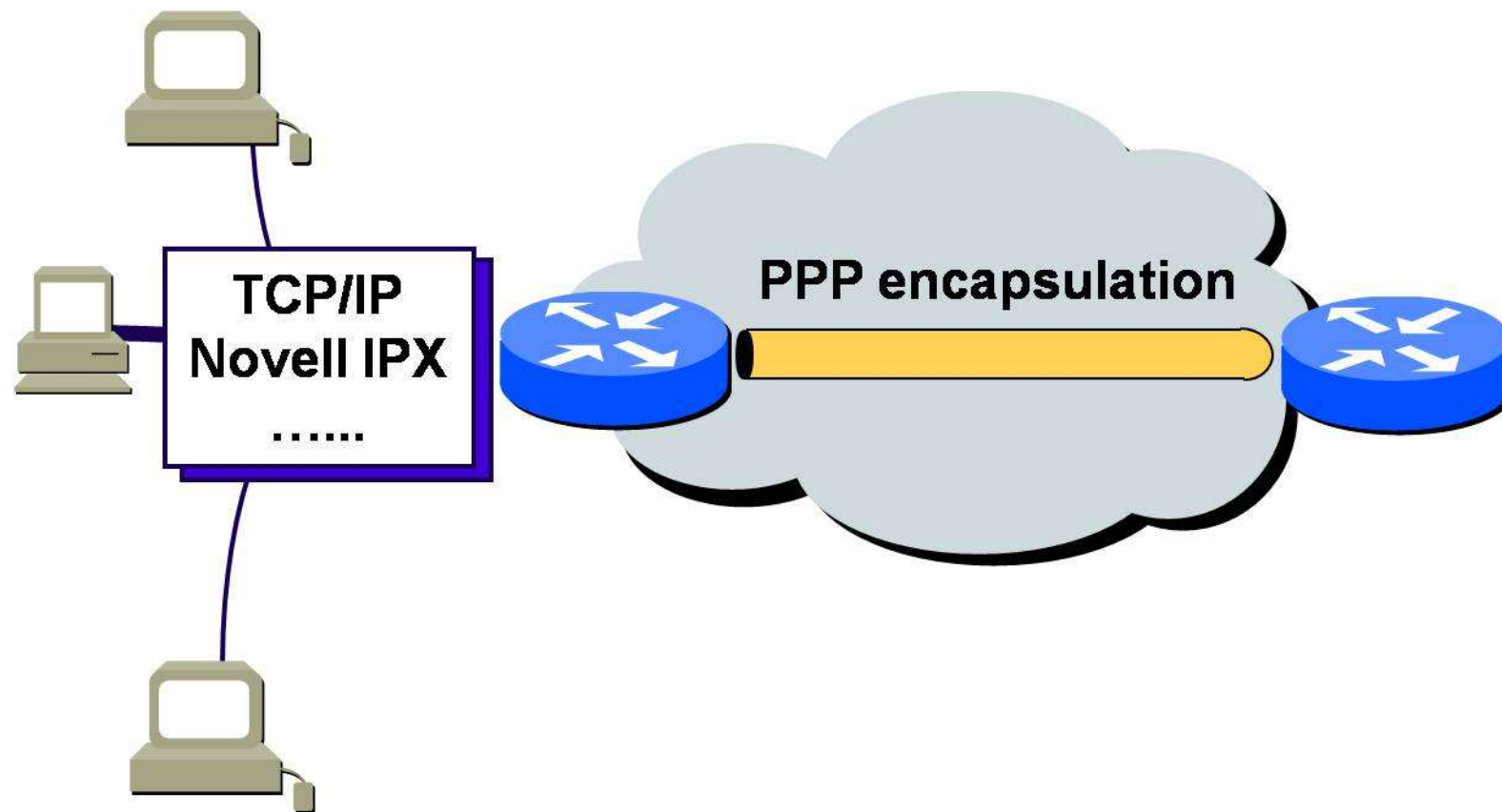
- 增强了

- ◆ 支持多种网络层协议
- ◆ 连接协商
- ◆ 用户身份认证
- ◆ 协商数据压缩
- ◆ 物理层可以是同步的或异步的

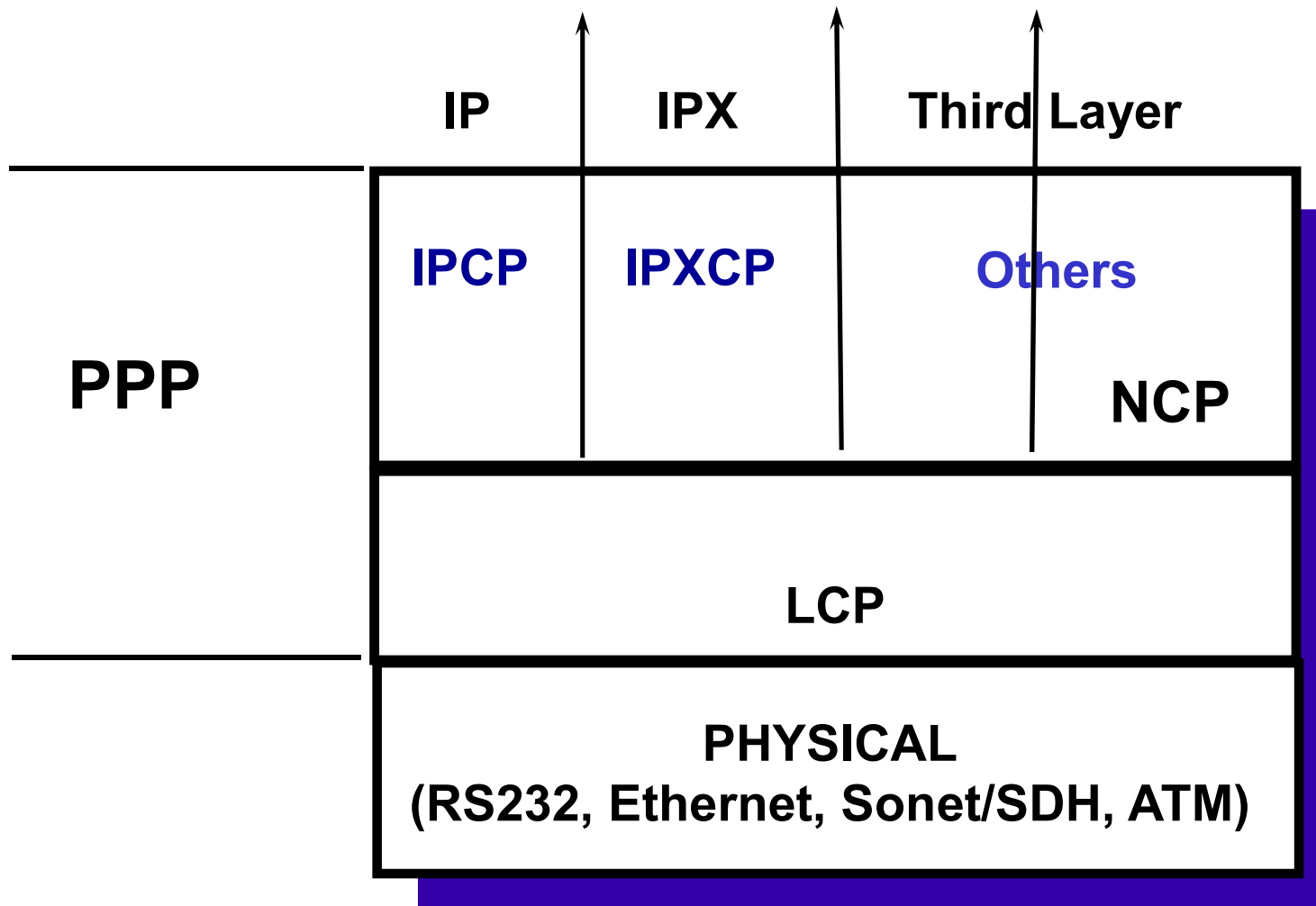
- 去除了

- ◆ 差错纠正
- ◆ 流量控制
- ◆ 序号

PPP: 支持多种网络层协议



PPP: 子层 (1)



PPP: 子层 (2)

- 协议标准

- ◆ RFC1661,1662,1663

- 第一层

- ◆ 同步电路 (面向比特编码, 如PPP over SONET)

- ◆ 异步电路 (面向字节编码)

- ◆ PPPoE (PPP over Ethernet, ADSL)

- LCP: 链路控制协议

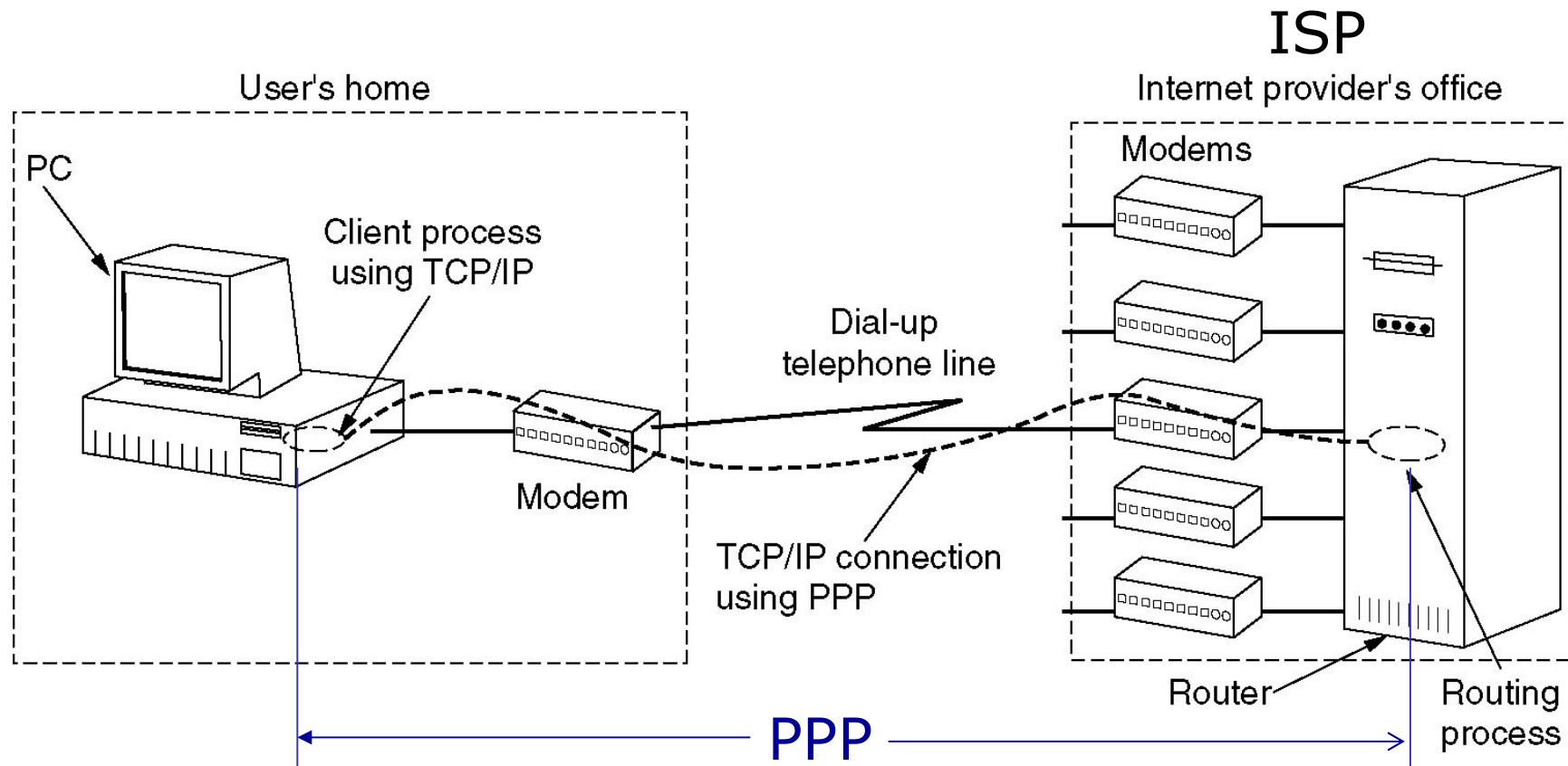
- ◆ 差错检测 (CRC)

- ◆ 身份认证

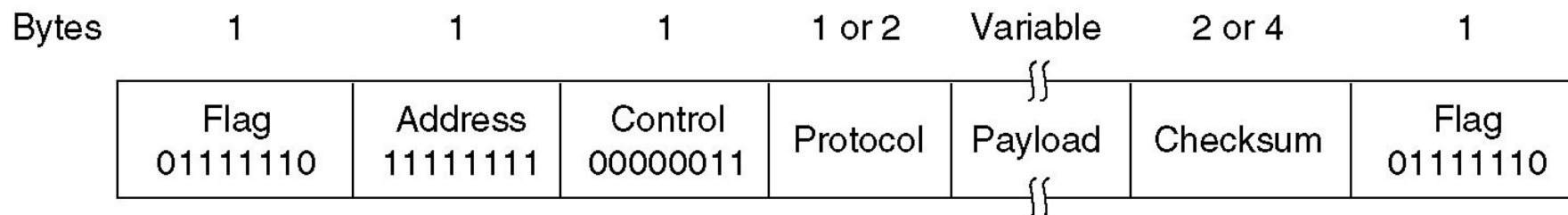
- NCP: 网络控制协议

- ◆ 协商网络层选项, 如IP地址

PPP用于因特网的拨号接入



PPPoE: 帧格式 (1)



■ 无编号模式

■ 字节填充

◆ 转移字符: 0x7D

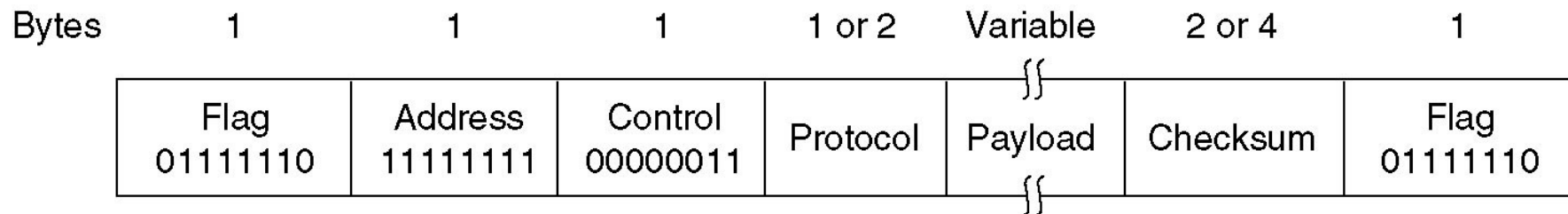
◆ 0x7E → 0x7D 0x5E

◆ 0x7D → 0x7D 0x5D

◆ 对于ASCII控制字符($\leq 0x20$), 也在前面加上 0x7D

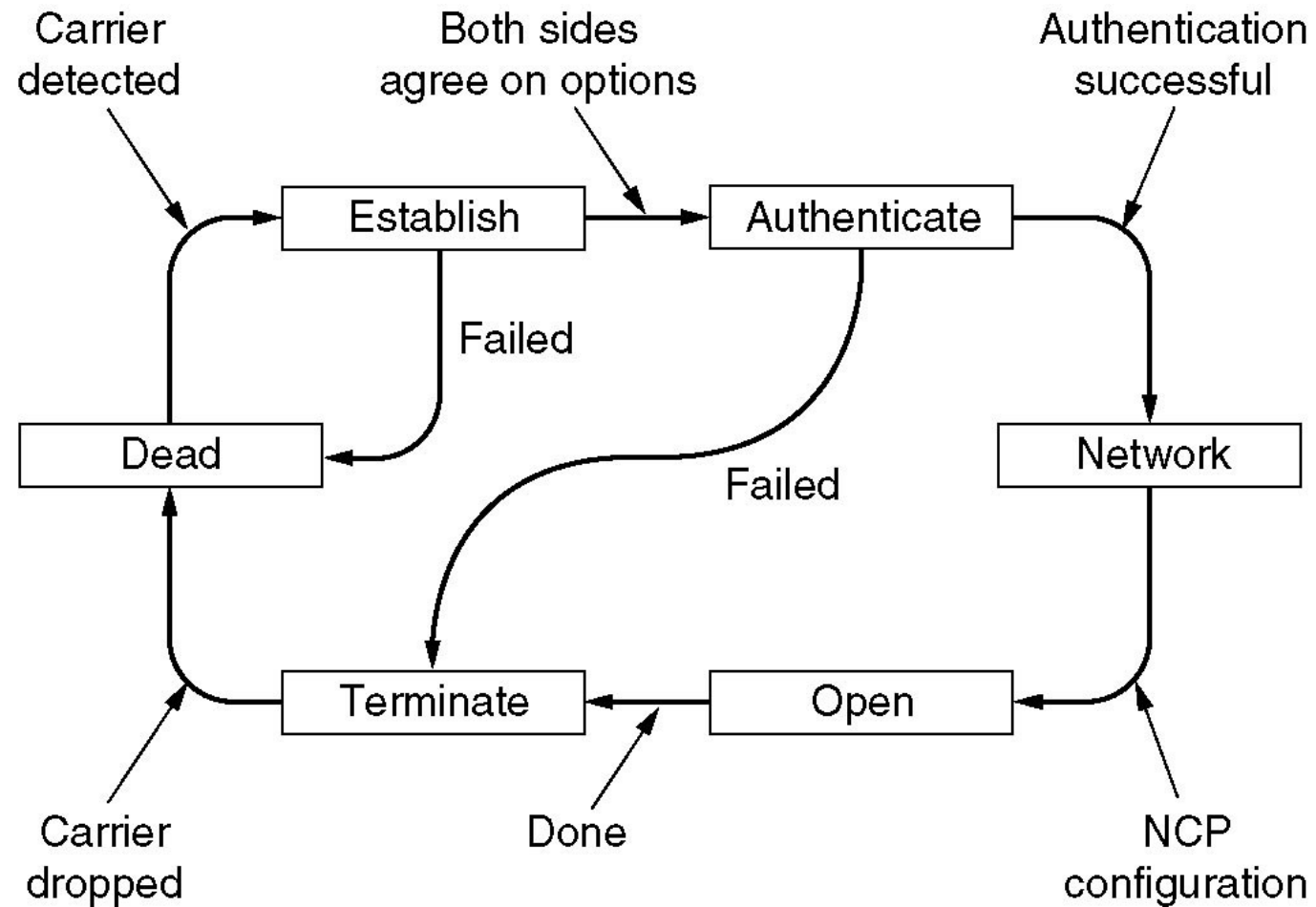
➤ 如: 0x03 → 0x7D 0x03

PPP for PPPoE: Frame Format(2)

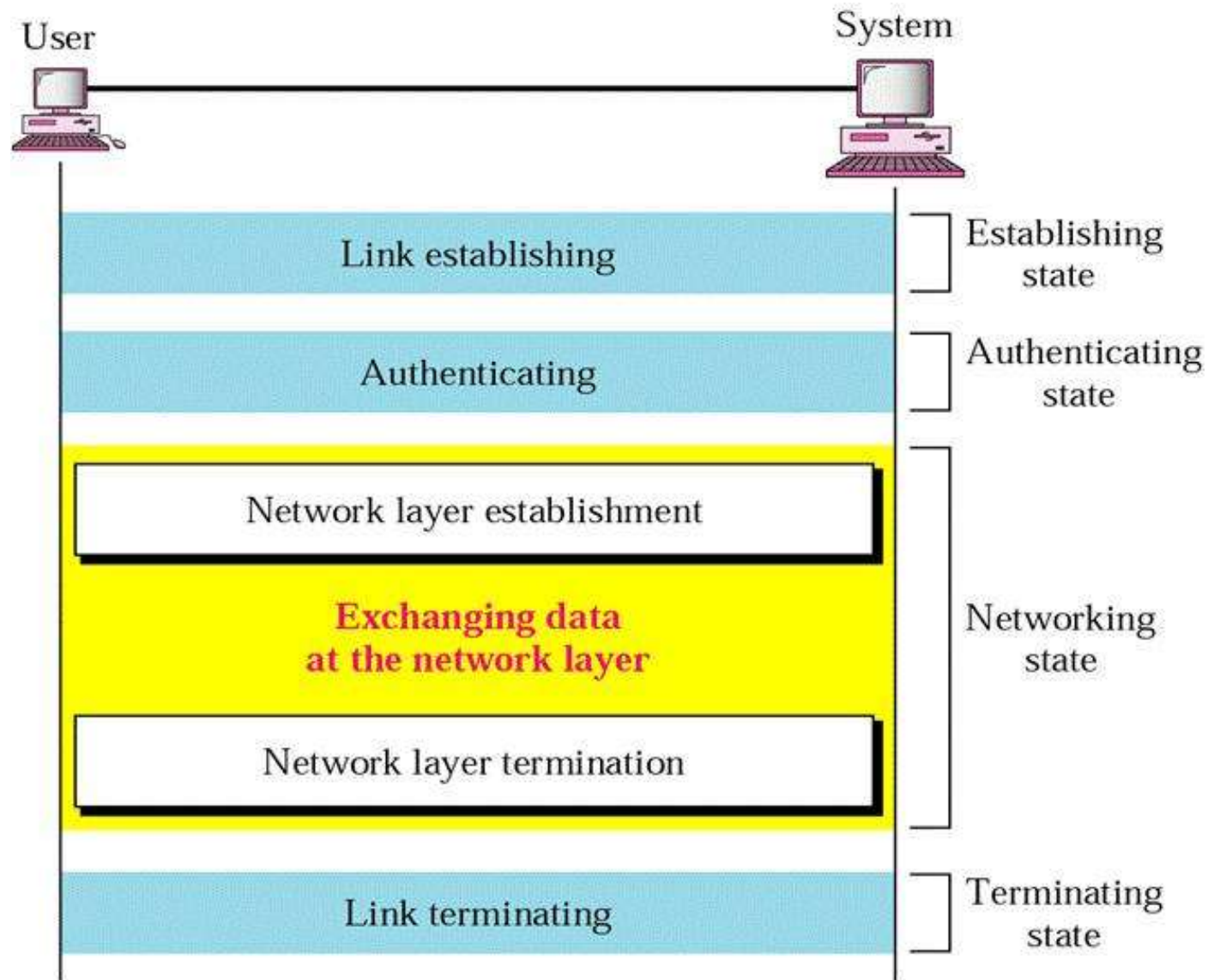


- 地址字段: 0xFF表示“所有站点”
- 控制字段: 0x03表示无编号模式
- 协议字段: 表示帧中的净荷（数据）类型，如 LCP, NCP, IP, IPX, AppleTalk...
- 校验和: 使用CRC进行检错

PPP: 状态转移图



PPP: 状态转移示例



数据链路层要点

■ 数据链路层的功能与服务

■ 成帧方法

- ◆ 字节填充
- ◆ 比特填充
- ◆ 物理层编码违例法

■ 差错控制

- ◆ 纠错码: 汉明码
- ◆ 检错码: CRC

■ 流量控制: ARQ

- ◆ 停止等待协议: $W_T = W_R = 1, U = 1/(1+2a)$
- ◆ Go-back-N协议: $1 < W_T < 2^m, W_R = 1 \rightarrow U = \begin{cases} 1 & W \geq 2a+1 \\ \frac{W}{2a+1} & W < 2a+1 \end{cases}$
- ◆ 选择重传协议: $W_T = W_R = 2^{m-1}$

■ 实际协议示例

- ◆ HDLC: 用途、帧类型
- ◆ PPP: 用途、帧结构

Terms

缩写	英文全称	中文
	access control	访问(接入)控制
ACK	Acknowledgement	确认(帧)
	addressing	寻址
ARQ	Automatic Repeat reQuest	自动请求重发
ATM	Asynchronous Transfer Mode	异步传输模式
	bandwidth-delay product	带宽时延积
BER	Bit Error Rate	误码率
	bit rate	数据率(比特率)
	bit stuffing	(零)比特填充
	byte stuffing	字节填充
	burst error	突发差错
	carrier	载波
	character count	字符计数法
	Character Stuffing	字符填充
	codeword	码字

缩写	英文全称	中文
	cumulative acknowledgement	累积 ACK
CRC	Cyclic Redundancy Check	循环冗余校验
	data link layer	数据链路层
	dial-up	拨号
	encapsulation	封装
	error control	差错控制
	error syndrome	差错综合症
	error correction	差错纠正
	Error Correcting Code	纠错码
	error detection	差错检测
	Ethernet	以太网
	flag byte	标志字节
FCS	Frame Check Sequence	帧校验序列
FDM	Frequency Division Multiplexing	频分复用

缩写	英文全称	中文
	flow control	流量控制
FEC	Forward Error Correction	前向纠错
	Feedback-based flow control	基于反馈的流量控制
	frame	帧
	framing	成帧
	frame header	帧头
	full-duplex	全双工
	Generator polynomial	生成多项式
	Go-Back-N	回退N步(协议)
	Hamming code	汉明(海明)码
	hamming distance	汉明距离
	HUB	集线器
LAN	Local Area Network	局域网
LAP	Link Access Procedure	链路接入规程

缩写	英文全称	中文
LAPB	Link Access Procedure - Balanced mode	链路接入规程——平衡模式
LAPD	Link Access Procedure, on D channel	D信道上的链路接入规程
LCP	Link Control Protocol	链路控制协议
	Line Utilization	线路利用率
LLC	Logical Link Control	逻辑链路控制
MAC	Media Access Control	媒体(介质)访问控制
	Manchester Encoding	曼彻斯特编码
HDLC	High-Level Data Link Control	高级数据链路控制
	hop	跳
	hop by hop	逐跳
NAK	Negative acknowledgement	否认(帧)
NCP	Network Control Protocol	网络控制协议
NIC	Network Interface Card	网络接口卡
	Odd & Even Parity	奇偶校验

缩写	英文全称	中文
PAR	Positive Acknowledgement with Retransmission	带确认的重传
	packet	分组/包
	Physical layer coding violations	物理层编码违例法
	Packet Switching	分组交换
	Parity check	奇偶校验
	payload	净荷（数据）
	piggybacking	捎带确认(应答)
	pipelining	流水线
PPP	Point-to-Point Protocol	点对点协议
PPPoE	PPP over Ethernet	以太网之上的点对点协议
	Polynomial Code	多项式编码
	preamble	前导码
	propagation delay	传播时延
	rate-based flow control	基于速率的流量控制
	round-trip propagation time	环回传播时间

缩写	英文全称	中文
SLIP	Serial Line IP	串行线IP
SONET	Synchronous Optical Network	同步光网络
	Selective Repeat	选择重传(协议)
	sliding window	滑动窗口
	Stop-and-wait	停止等待(协议)
	store-and-forward switching	存储转发交换
	subnet	子网
	switch	交换机
	switched Ethernet	交换式以太网
TDM	Time Division Multiplexing	时分复用
	Timeout Timer	超时定时器
	transparent transmission	透明传输
	transmission delay	发送时延
	transport layer	传输层
WAN	Wide Area Network	广域网
WLAN	Wireless Local Area Network	无线局域网

Copyright Information

- Some figures used in this presentation have been either directly copied or adapted from following materials
 - ◆ Lecture notes of book “Computer Networks, a Top-down Approach”, J.Kurose and W.Ross, 3rd Edition, which are marked with *[Kurose]*
 - ◆ “Data communications and networking”, B.A.Forouzan, which are marked with *[Forouzan]*
 - ◆ “Data and Computer Communications”, William Stallings, which are marked with *[Stallings]*
 - ◆ “计算机网络”, 谢希仁, 第五版, 电子工业出版社, 2008年, which are marked with *[谢]*

软件实现CRC计算

■ Software (from RFC1662)

◆ FCS: Frame Check Sequence, 16bits

```
static u16 fcstab[256] = {
    0x0000, 0x1189, 0x2312, 0x329b, 0x4624, 0x57ad, 0x6536,
    0x74bf, 0x8c48, 0x9dc1, 0xaf5a, 0xbed3, 0xca6c, 0xdbe5,
    .....,
    0xd49d, 0xc514, 0xb1ab, 0xa022, 0x92b9, 0x8330, 0x7bc7,
    0x6a4e, 0x58d5, 0x495c, 0x3de3, 0x2c6a, 0x1ef1, 0x0f78
};

u16 fcs16(u16 fcs, unsigned char *p, int len)
{
    while (len--)
        fcs = (fcs >> 8) ^ fcstab[(fcs ^ *p++) & 0xff];
    return (fcs);
}
```